# chemical-engineering-ai999x: capability guide

**Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `chemical_engineering_ai999x.about()` for the same answer.

Every capability chemical-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

## Quick start

```
import chemical_engineering_ai999x

result = chemical_engineering_ai999x.solve('chemeng_blending_flows', W=[[0.9, 0.1], [0.2, 0.8]], p=
[0.5, 0.5], P='100 kg/s')
result.show()
# F1 = 42.8571 kg/s, F = [42.8571428571, 57.1428571429]
```

## By topic

| topic | capabilities |
|---|---|
| Energy balances / adiabatic reactors | Adiabatic reactor outlet temperature |
| Energy balances / combustion | Adiabatic flame temperature (complete combustion, NASA polynomials) |
| Energy balances / heat of reaction | Heat of reaction at another temperature (Kirchhoff) |
| Energy balances / sensible heat | Sensible heat with a polynomial heat capacity |
| Energy balances / unsteady heating | Batch heating of a stirred tank: temperature after a time, Batch heating of a stirred tank: time to reach a temperature, Batch cooling (or heating) with a non-isothermal coolant in a coil |
| Fluid transport / pumps | Pump total head and power, Net positive suction head available |
| Material balances / combustion | Combustion with excess air: air requirement and flue-gas composition |
| Material balances / compositions | Mass fractions to mole fractions and mean molar mass |
| Material balances / mixing and blending | Blending: feed flows that make a product of given composition |
| Material balances / recycle and purge | Reactor with recycle and purge |
| Material balances / separators | Two-product separator balance (binary distillation column) |

| topic | capabilities |
|---|---|
| Material balances / unsteady state | Unsteady well-mixed tank (concentration change), Filling or draining well-mixed tank (variable volume) |
| Petroleum engineering / decline curves | Exponential decline curve: rate and cumulative production, Hyperbolic decline curve: rate and cumulative production, Harmonic decline curve: rate and cumulative production |
| Petroleum engineering / gas well testing | Gas-well deliverability from a back-pressure test (Rawlins-Schellhardt) |
| Process control / controller tuning | Ziegler-Nichols closed-loop tuning of lags in series |
| Process control / feedback control | Proportional control: steady-state offset after a set-point step |
| Process control / frequency response | Frequency response of first-order lags in series (amplitude ratio and phase) |
| Process control / PID control | Discrete PID controller output for an error history |
| Process control / process dynamics | First-order-plus-dead-time process: response to a step |
| Process control / process identification | Process gain and time constant from step-test data (least squares) |
| Process control / stability | Closed-loop stability of a P-controlled process (closed-loop poles), Gain and phase margins of a PI loop around lags in series |
| Property estimation / equations of state | Peng-Robinson equation of state: compressibility and fugacity coefficient, Soave-Redlich-Kwong equation of state: compressibility and fugacity coefficient |
| Property estimation / gas compressibility | Gas compressibility from the virial equation (Pitzer-Abbott correlation) |
| Property estimation / heat of vaporisation | Heat of vaporisation at another temperature (Watson), Heat of vaporisation at the normal boiling point (Riedel), Heat of vaporisation from the acentric factor (Pitzer) |
| Property estimation / liquid density | Saturated liquid molar volume (Rackett) |
| Property estimation / vapour pressure | Vapour pressure by the Lee-Kesler correlation |
| Reactor design / batch reactors | Batch reactor time for a conversion (first order), Batch reactor time for a conversion (second order), Batch reactor conversion after a time (first order), Batch reactor conversion after a time (second order) |
| Reactor design / continuous stirred tanks | CSTR volume for a conversion (first order), CSTR conversion for a volume (first order), CSTR conversion for a volume (second order), Equal CSTRs in series: conversion (first order), Equal CSTRs in series: conversion (second order), CSTR volume for a conversion (second order) |
| Reactor design / multiple reactions | Series reactions A -> B -> C: maximum of the intermediate (batch or PFR) |
| Reactor design / plug-flow reactors | PFR volume for a conversion (first order, liquid), PFR volume for a conversion (second order, liquid), PFR conversion for a volume (first order, liquid), PFR conversion for a volume (second order, liquid), Gas-phase PFR volume with volume change (first order), Gas-phase PFR volume with volume change (second order) |
| Reactor design / recycle reactors | Plug-flow reactor with recycle: conversion (first order) |
| Reactor design / residence-time distribution | Residence-time distribution: mean, variance and tanks-in-series number from tracer data |

| topic | capabilities |
|---|---|
| Reactor design / reversible reactions | Reversible first-order reaction A <-> B in a batch reactor |
| Reservoir engineering / gas material balance | Gas initially in place from the p/z plot |
| Reservoir engineering / material balance | Oil initially in place from the material balance (Havlena-Odeh straight line), Oil in place and gas-cap size from the material balance (Havlena-Odeh, gas cap) |
| Reservoir engineering / well inflow | Steady radial flow of oil to a well (Darcy) and productivity index |
| Reservoir engineering / well testing | Pressure build-up test: Horner analysis for permeability and extrapolated pressure, Transient drawdown around a well (line-source solution) |
| Separations / absorption | Absorber with N equilibrium stages (Kremser), Packed absorber: number of transfer units and height (Colburn) |
| Separations / activity coefficients | Activity coefficients from the Wilson equation, Activity coefficients from the NRTL equation, Activity coefficients from the UNIQUAC equation |
| Separations / distillation | Minimum reflux ratio of a binary column (Underwood), McCabe-Thiele number of theoretical stages (binary), Minimum number of stages at total reflux (Fenske), Minimum reflux of a multicomponent column (Underwood) |
| Separations / flash vaporisation | Isothermal flash with given K-values (Rachford-Rice), Isothermal flash of an ideal mixture (Raoult's law), Flash with Wilson K-values (hydrocarbon mixtures) |
| Separations / liquid extraction | Countercurrent liquid-liquid extraction (N stages, immiscible solvents), Crosscurrent (multiple-contact) extraction with fresh solvent |
| Separations / stripping | Stripper with N equilibrium stages (Kremser) |
| Separations / vapour pressure | Vapour pressure from the Antoine equation, Boiling (saturation) temperature from the Antoine equation |
| Separations / vapour-liquid equilibrium | Bubble-point pressure (Raoult's law), Dew-point pressure (Raoult's law), Bubble-point temperature (Raoult's law), Dew-point temperature (Raoult's law), Bubble-point pressure by the modified Raoult's law (activity coefficients) |
| Transport / filtration | Constant-pressure cake filtration: filtrate volume after a time, Specific cake and medium resistance from constant-pressure filtration data |
| Transport / mass transfer | Diffusion of A through stagnant B (Stefan film), Mass-transfer coefficient from a stated Sherwood correlation, Equimolar counterdiffusion flux |
| Transport / packed beds | Packed-bed pressure drop (Ergun equation), Packed-bed pressure drop (Carman correlation) |

## Outcomes

| outcome | meaning | what to do |
|---|---|---|
| SOLVED | a value with its unit, steps, assumptions and verification | use it; read the assumptions |
| NEEDS_INPUT | a required input is missing; `uncertainty['needs']` names it | add it (with its unit) and ask again |
| AMBIGUOUS | two readings or two physical branches give different answers | say which situation is meant |
| INVALID_INPUT | a unit of the wrong kind, an unknown input name, values with no valid solution | correct the input |

| outcome | meaning | what to do |
|---|---|---|
| OUT_OF_SCOPE | no certified capability computes this (or the structure is not modelled yet) | see the catalogue below |
| UNRECOGNIZED_FORMAT | the wording was not recognised; a supported template is offered in `uncertainty['try']` | use the template or solve() |
| INTERNAL_ERROR | a defect of the engine, never a property of the question | report it with the question |

## Result fields

| field | meaning |
|---|---|
| value, unit | the result in SI units |
| values | every output of a procedure (x1, x2, discriminant ...) |
| outcome | one of the seven outcomes above |
| steps | each equation used, its inputs (with where each came from) and its result |
| assumptions | conditions and declared defaults the answer relies on |
| verification | residual of every equation, dimensional consistency, independent checks |
| latex() | the derivation as LaTeX |
| show() | renders in a notebook, prints in a terminal |

## Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

## Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_blending_flows | Blending: feed flows that make a product of given composition | W, p, P | F1, F |
| chemeng_binary_separator_balance | Two-product separator balance (binary distillation column) | F, zF, xD, xB | D, B, recovery_D, recovery_B |
| chemeng_recycle_purge | Reactor with recycle and purge | F0, zA, X, f | R, P, reactor_feed, X_overall, product, y_inert |
| chemeng_unsteady_mixing_tank | Unsteady well-mixed tank (concentration change) | V, q, C_in, C0, t | C |

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_variable_volume_tank | Filling or draining well-mixed tank (variable volume) | V0, q_in, q_out, C_in, C0, t | V, C |
| chemeng_combustion_flue_gas | Combustion with excess air: air requirement and flue-gas composition | fuel, excess_air | O2_theoretical, air, n_CO2, n_H2O, n_SO2, n_O2, n_N2, y_CO2_dry, y_O2_dry, y_H2O_wet, AFR_mass |
| chemeng_sensible_heat_cp_polynomial | Sensible heat with a polynomial heat capacity | n, a, b, c, d, T1, T2 | Q, cp_mean |
| chemeng_reaction_enthalpy_at_temperature | Heat of reaction at another temperature (Kirchhoff) | nu, cp, dH0, T0, T | dH |
| chemeng_adiabatic_reactor_temperature | Adiabatic reactor outlet temperature | n0, nu, cp, dH, T_ref, T_in, key, X | T_out |
| chemeng_adiabatic_flame_temperature | Adiabatic flame temperature (complete combustion, NASA polynomials) | fuel, excess_air, T_in | T_ad |
| chemeng_batch_heating_temperature | Batch heating of a stirred tank: temperature after a time | m, cp, UA, T_s, T0, t | T |
| chemeng_batch_heating_time | Batch heating of a stirred tank: time to reach a temperature | m, cp, UA, T_s, T0, T | t |
| chemeng_batch_coil_cooling | Batch cooling (or heating) with a non-isothermal coolant in a coil | M, cp, W, cp_c, t_in, UA, T0, t | T |
| chemeng_batch_time_first_order | Batch reactor time for a conversion (first order) | k, X | t |
| chemeng_batch_time_second_order | Batch reactor time for a conversion (second order) | k, CA0, X | t |
| chemeng_batch_conversion_first_order | Batch reactor conversion after a time (first order) | k, t | X |
| chemeng_batch_conversion_second_order | Batch reactor conversion after a time (second order) | k, CA0, t | X, CA |
| chemeng_cstr_volume_first_order | CSTR volume for a conversion (first order) | v0, k, X | V, tau |
| chemeng_cstr_conversion_first_order | CSTR conversion for a volume (first order) | v0, V, k | X |
| chemeng_cstr_conversion_second_order | CSTR conversion for a volume (second order) | v0, V, k, CA0 | X, CA |
| chemeng_pfr_volume_first_order | PFR volume for a conversion (first order, liquid) | v0, k, X | V |
| chemeng_pfr_volume_second_order | PFR volume for a conversion (second order, liquid) | v0, k, CA0, X | V |
| chemeng_pfr_conversion_first_order | PFR conversion for a volume (first order, liquid) | v0, V, k | X |
| chemeng_pfr_conversion_second_order | PFR conversion for a volume (second order, liquid) | v0, V, k, CA0 | X, CA |

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_pfr_gas_volume_first_order | Gas-phase PFR volume with volume change (first order) | FA0, CA0, k, eps, X | V |
| chemeng_pfr_gas_volume_second_order | Gas-phase PFR volume with volume change (second order) | FA0, CA0, k, eps, X | V |
| chemeng_cstrs_in_series_first_order | Equal CSTRs in series: conversion (first order) | v0, V, k, N | X |
| chemeng_cstrs_in_series_second_order | Equal CSTRs in series: conversion (second order) | v0, V, k, CA0, N | X, CA |
| chemeng_recycle_reactor_first_order | Plug-flow reactor with recycle: conversion (first order) | v0, V, k, R | X |
| chemeng_reversible_first_order_batch | Reversible first-order reaction A <-> B in a batch reactor | k1, k2, CA0, CB0, t | CA, CB, X, X_eq |
| chemeng_series_reaction_maximum | Series reactions A -> B -> C: maximum of the intermediate (batch or PFR) | k1, k2, CA0 | t_max, CB_max, CA, CC |
| chemeng_rtd_moments | Residence-time distribution: mean, variance and tanks-in-series number from tracer data | t, C, t_unit | t_mean, variance, N_tanks |
| chemeng_antoine_vapour_pressure | Vapour pressure from the Antoine equation | A, B, C, units, base, T | Psat |
| chemeng_antoine_boiling_temperature | Boiling (saturation) temperature from the Antoine equation | A, B, C, units, base, P | Tsat |
| chemeng_raoult_bubble_pressure | Bubble-point pressure (Raoult's law) | x, ant, units, base, T | P, y |
| chemeng_raoult_dew_pressure | Dew-point pressure (Raoult's law) | y, ant, units, base, T | P, x |
| chemeng_raoult_bubble_temperature | Bubble-point temperature (Raoult's law) | x, ant, units, base, P | T, y |
| chemeng_raoult_dew_temperature | Dew-point temperature (Raoult's law) | y, ant, units, base, P | T, x |
| chemeng_rachford_rice_flash | Isothermal flash with given K-values (Rachford-Rice) | z, K | VF, x, y |
| chemeng_raoult_isothermal_flash | Isothermal flash of an ideal mixture (Raoult's law) | z, ant, units, base, T, P | VF, x, y |
| chemeng_wilson_k_flash | Flash with Wilson K-values (hydrocarbon mixtures) | z, Tc, Pc, omega, p_unit, T, P | VF, x, y, K |
| chemeng_binary_minimum_reflux | Minimum reflux ratio of a binary column (Underwood) | alpha, zF, xD, q | Rmin |
| chemeng_mccabe_thiele_stages | McCabe-Thiele number of theoretical stages (binary) | alpha, zF, xD, q, xB, R | N, Rmin |
| chemeng_kremser_absorber | Absorber with N equilibrium stages (Kremser) | N, A, m, y_in, x_in | y_out, recovery |

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_kremser_stripper | Stripper with N equilibrium stages (Kremser) | N, S, m, x_in, y_in | x_out, recovery |
| chemeng_absorber_transfer_units | Packed absorber: number of transfer units and height (Colburn) | y1, y2, x2, m, A, HOG | NOG, Z |
| chemeng_countercurrent_extraction | Countercurrent liquid-liquid extraction (N stages, immiscible solvents) | F, S, K, N, XF, YS | XN, Y1, recovery |
| chemeng_crosscurrent_extraction | Crosscurrent (multiple-contact) extraction with fresh solvent | F, S, K, N, XF | XN, recovery |
| chemeng_wilson_activity_coefficients | Activity coefficients from the Wilson equation | x, Lam | gamma1, gamma, GE_RT |
| chemeng_nrtl_activity_coefficients | Activity coefficients from the NRTL equation | x, tau, alpha | gamma1, gamma, GE_RT |
| chemeng_uniquac_activity_coefficients | Activity coefficients from the UNIQUAC equation | x, r, q, tau | gamma1, gamma |
| chemeng_peng_robinson | Peng-Robinson equation of state: compressibility and fugacity coefficient | Tc, Pc, omega, T, P | Z_vapour, Z_liquid, phi_vapour, phi_liquid, n_roots |
| chemeng_srk | Soave-Redlich-Kwong equation of state: compressibility and fugacity coefficient | Tc, Pc, omega, T, P | Z_vapour, Z_liquid, phi_vapour, phi_liquid, n_roots |
| chemeng_lee_kesler_vapour_pressure | Vapour pressure by the Lee-Kesler correlation | T, Tc, Pc, omega | Psat |
| chemeng_watson_heat_of_vaporization | Heat of vaporisation at another temperature (Watson) | T, T_ref, Hvap_ref, Tc | Hvap |
| chemeng_riedel_heat_of_vaporization | Heat of vaporisation at the normal boiling point (Riedel) | Tb, Tc, Pc | Hvap_b |
| chemeng_pitzer_heat_of_vaporization | Heat of vaporisation from the acentric factor (Pitzer) | T, Tc, omega | Hvap |
| chemeng_rackett_liquid_volume | Saturated liquid molar volume (Rackett) | T, Tc, Pc, Zc | Vm |
| chemeng_virial_compressibility | Gas compressibility from the virial equation (Pitzer-Abbott correlation) | T, P, Tc, Pc, omega | B, Z |
| chemeng_mass_to_mole_fractions | Mass fractions to mole fractions and mean molar mass | w, MW | z, MW_mean |
| chemeng_stefan_diffusion_flux | Diffusion of A through stagnant B (Stefan film) | D, P, T, L, yA1, yA2 | NA |
| chemeng_ergun_pressure_drop | Packed-bed pressure drop (Ergun equation) | dp, eps, vs, rho, mu, L | dP |
| chemeng_carman_pressure_drop | Packed-bed pressure drop (Carman correlation) | dp, eps, vs, rho, mu, L | dP |

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_cake_filtration_volume | Constant-pressure cake filtration: filtrate volume after a time | A, dP, mu, alpha, c, Rm, t | V |
| chemeng_filtration_constants_fit | Specific cake and medium resistance from constant-pressure filtration data | t, V, t_unit, v_unit, A, dP, mu, c | alpha, Rm, slope, intercept |
| chemeng_fopdt_step_response | First-order-plus-dead-time process: response to a step | K, tau, theta, M, t | y |
| chemeng_p_control_offset | Proportional control: steady-state offset after a set-point step | Kc, K, taus, t_unit | y_ss, offset |
| chemeng_lag_frequency_response | Frequency response of first-order lags in series (amplitude ratio and phase) | K, taus, t_unit, w | AR, phase_deg |
| chemeng_pid_controller_output | Discrete PID controller output for an error history | Kc, tauI, tauD, dt, e, bias | u_last, u |
| chemeng_ziegler_nichols_tuning | Ziegler-Nichols closed-loop tuning of lags in series | K, taus, t_unit, controller | Ku, Pu, Kc, tauI, tauD |
| chemeng_closed_loop_stability | Closed-loop stability of a P-controlled process (closed-loop poles) | num, den, Kc | max_real, n_rhp, stable |
| chemeng_pi_loop_margins | Gain and phase margins of a PI loop around lags in series | Kc, tauI, K, taus, t_unit | GM, PM_deg, w_pc, w_gc |
| chemeng_first_order_step_fit | Process gain and time constant from step-test data (least squares) | t, y, t_unit, M, theta | K, tau |
| chemeng_radial_darcy_flow | Steady radial flow of oil to a well (Darcy) and productivity index | k, h, mu, pe, pw, re, rw, S | q, PI |
| chemeng_exponential_decline | Exponential decline curve: rate and cumulative production | qi, Di, t | q, Np |
| chemeng_hyperbolic_decline | Hyperbolic decline curve: rate and cumulative production | qi, Di, b, t | q, Np |
| chemeng_harmonic_decline | Harmonic decline curve: rate and cumulative production | qi, Di, t | q, Np |
| chemeng_havlena_odeh_oil_in_place | Oil initially in place from the material balance (Havlena-Odeh straight line) | F, Et, v_unit | N |
| chemeng_havlena_odeh_gas_cap | Oil in place and gas-cap size from the material balance (Havlena-Odeh, gas cap) | F, Eo, Eg, v_unit | N, m |
| chemeng_gas_pz_material_balance | Gas initially in place from the p/z plot | p, z, Gp, p_unit, gp_unit | G, pz_initial |

| id | operation | inputs | outputs |
|---|---|---|---|
| chemeng_horner_buildup | Pressure build-up test: Horner analysis for permeability and extrapolated pressure | dt, pws, t_unit, p_unit, tp, q, mu, h | m, k, p_star |
| chemeng_line_source_drawdown | Transient drawdown around a well (line-source solution) | q, mu, k, h, phi, ct, r, t, pi | dp, p |
| chemeng_gas_well_backpressure | Gas-well deliverability from a back-pressure test (Rawlins-Schellhardt) | q, pwf, q_unit, p_unit, pr | n, log10_C, AOF |
| chemeng_fenske_minimum_stages | Minimum number of stages at total reflux (Fenske) | alpha, xD, xB | Nmin, Nmin_trays |
| chemeng_pump_head | Pump total head and power | p1, p2, rho, dz, v1, v2, Q, eta | H, dp_equivalent, P_hydraulic, P_shaft |
| chemeng_npsh_available | Net positive suction head available | p_s, p_v, rho, z_s, h_f, NPSHr | NPSHa, pressure_head, vapour_head, margin |
| chemeng_mass_transfer_coefficient | Mass-transfer coefficient from a stated Sherwood correlation | geometry, u, L, D, rho, mu | Re, Sc, Sh, kc |
| chemeng_multicomponent_underwood | Minimum reflux of a multicomponent column (Underwood) | alpha, z, xD, q, lk, hk | theta, Rmin |
| chemeng_cstr_volume_second_order | CSTR volume for a conversion (second order) | v0, k, CA0, X | V, tau |
| chemeng_equimolar_counterdiffusion | Equimolar counterdiffusion flux | D, P, T, L, yA1, yA2 | NA, NB |
| chemeng_modified_raoult_bubble_pressure | Bubble-point pressure by the modified Raoult's law (activity coefficients) | x, gamma, Psat, p_unit | P, y1, y |

### Blending: feed flows that make a product of given composition — `chemeng_blending_flows`

The mass flow of each feed stream needed to make a product of stated flow and composition, from the component balances (as many feeds as components).

$$\sum_i F_i\, w_{ij} = P\, p_j \quad (j = 1..n)$$

| input | meaning | condition |
|---|---|---|
| `W` | mass fractions of the feed streams: one row per feed stream, one column per component (rows sum to 1) | |
| `p` | mass fractions of the product, one per component (sum to 1) | |
| `P` | product mass flow | |

| output | meaning |
|---|---|
| `F1` | flow of the first feed stream |
| `F` | flow of every feed stream, in kg/s, in the order of the rows of W |

- **Method:** component balances sum_i F_i $w_i j = P$ p_j for every component j, solved by Gaussian elimination with partial pivoting
- **Assumes:** as many feed streams as components (2 to 6); each composition sums to 1; the product lies within the reach of the feeds (no negative flow)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_blending_flows', W=[[0.9, 0.1], [0.2, 0.8]], p=[0.5,
0.5], P=100.0)
# -> F1 = 42.8571 kg/s, F = [42.8571428571, 57.1428571429]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_blending_flows', W=[[0.8, 0.1, 0.1], [0.1, 0.7, 0.2],
[0.05, 0.15, 0.8]], p=[0.4, 0.3, 0.3], P=10.0)
# -> F1 = 4.45783 kg/s, F = [4.4578313253, 3.13253012048, 2.40963855422]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Blending feed flows: W = [[0.9, 0.1], [0.2, 0.8]], p = [0.5, 0.5], P = 100.0 kg/s | SOLVED | F1 = 42.8571 kg/s, F = [42.8571428571, 57.1428571429] |
| Blending feed flows: W = [[0.8, 0.1, 0.1], [0.1, 0.7, 0.2], [0.05, 0.15, 0.8]], p = [0.4, 0.3, 0.3], P = 10.0 kg/s | SOLVED | F1 = 4.45783 kg/s, F = [4.4578313253, 3.13253012048, 2.40963855422] |
| Blending feed flows: p = [0.5, 0.5], P = 100.0 kg/s | NEEDS_INPUT | chemeng_blending_flows needs W (mass fractions of the feed streams: one row per feed stream, one column per component (r |
| Blending feed flows: W = [[0.9, 0.1], [0.2, 0.8]], p = [0.5, 0.5], P = 100.0 kg/s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Blending: W = [[0.9, 0.1], [0.2, 0.8]], p = [0.95, 0.05], P = 100 kg/s | INVALID_INPUT | Blending: feed flows that make a product of given composition: this product cannot be made from these feeds (a feed flow |
| Blending: W = [[0.9, 0.1], [0.2, 0.8]], p = [0.5, 0.5], P = 360000 kg/h | SOLVED | F1 = 42.8571 kg/s, F = [42.8571428571, 57.1428571429] |

## Two-product separator balance (binary distillation column) —

`chemeng_binary_separator_balance`

Distillate and bottoms flows of a binary separator from the feed flow and the three compositions, with the recoveries of the light component in the distillate and of the heavy component in the bottoms.

$$D = F\,\frac{z_F - x_B}{x_D - x_B}, \quad B = F - D$$

| input | meaning | condition |
|---|---|---|
| `F` | feed molar flow | |

| input | meaning | condition |
|---|---|---|
| `zF` | mole fraction of the light component in the feed | |
| `xD` | mole fraction of the light component in the distillate (top product) | |
| `xB` | mole fraction of the light component in the bottoms | |

| output | meaning |
|---|---|
| `D` | distillate flow |
| `B` | bottoms flow |
| `recovery_D` | fraction of the light component recovered in the distillate |
| `recovery_B` | fraction of the heavy component recovered in the bottoms |

- **Method:** total balance $F = D + B$ and light-component balance F $zF = D$ xD + B xB
- **Assumes:** $xB < zF < xD$; steady state, no reaction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_binary_separator_balance', F=100.0, zF=0.4, xD=0.95,
xB=0.05)
# -> D = 38.8889 mol/s, B = 61.1111 mol/s, recovery_D = 0.923611, recovery_B = 0.967593   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_binary_separator_balance', F=2.5, zF=0.3, xD=0.9,
xB=0.02)
# -> D = 0.795455 mol/s, B = 1.70455 mol/s, recovery_D = 0.954545, recovery_B = 0.954545   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Two-product separator balance: F = 100.0 mol/s, zF = 0.4, xD = 0.95, xB = 0.05 | SOLVED | D = 38.8889 mol/s, B = 61.1111 mol/s, recovery_D = 0.923611, recovery_B = 0.967593 |
| Two-product separator balance: F = 2.5 mol/s, zF = 0.3, xD = 0.9, xB = 0.02 | SOLVED | D = 0.795455 mol/s, B = 1.70455 mol/s, recovery_D = 0.954545, recovery_B = 0.954545 |
| Two-product separator balance: zF = 0.4, xD = 0.95, xB = 0.05 | NEEDS_INPUT | chemeng_binary_separator_balance needs F (feed molar flow). Example: solve('chemeng_binary_separator_balance', F='100 mo |
| Two-product separator balance: F = 100.0 mol/s, zF = 0.4, xD = 0.95, xB = 0.05, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Binary distillation balance: F = 360 kmol/h, zF = 40%, xD = 95%, xB = 5% | SOLVED | D = 38.8889 mol/s, B = 61.1111 mol/s, recovery_D = 0.923611, recovery_B = 0.967593 |
| Two-product separator balance: F = 100 mol/s, zF = 0.4, xD = 0.3, xB = 0.05 | INVALID_INPUT | Two-product separator balance (binary distillation column): the compositions must satisfy xB < zF < xD (light component |

| question | outcome | result |
|---|---|---|
| Two-product separator balance: F = 100, zF = 0.4, xD = 0.95, xB = 0.05 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Reactor with recycle and purge — `chemeng_recycle_purge`

Steady-state flows of a reactor A -> B with complete product separation, recycle of unreacted A and inert, and a purge that removes a fraction of the recycle stream to stop the inert accumulating.

$$A_{in} = \frac{F_0 z_A}{1-(1-f)(1-X)}, \quad I_{in} = \frac{F_0(1-z_A)}{f}$$

| input | meaning | condition |
|---|---|---|
| `F0` | fresh feed flow (reactant A plus inert) | |
| `zA` | mole fraction of A in the fresh feed (the rest is inert) | |
| `X` | single-pass conversion of A in the reactor | |
| `f` | purge fraction: the fraction of the separator overhead (unreacted A + inert) that is purged | |

| output | meaning |
|---|---|
| `R` | recycle flow |
| `P` | purge flow |
| `reactor_feed` | combined reactor feed flow |
| `X_overall` | overall conversion of A (fresh feed to product) |
| `product` | production rate of B |
| `y_inert` | mole fraction of inert in the recycle and purge |

- **Method:** balances on A and inert around the mixer, reactor, separator and splitter: $A_in = F_0$ zA/(1 - (1 - f)$(1-X)$), inert_in = F0 $(1-zA)/f$
- **Assumes:** $0 < zA \le 1$; $0 < X < 1$; $0 < f < 1$; A -> B one to one; the separator removes all B
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_recycle_purge', F0=100.0, zA=0.98, X=0.25, f=0.1)
# -> R = 221.538 mol/s, P = 24.6154 mol/s, reactor_feed = 321.538 mol/s, X_overall = 0.769231,
product = 75.3846 mol/s, y_inert = 0.08125   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_recycle_purge', F0=50.0, zA=0.9, X=0.6, f=0.05)
# -> R = 122.581 mol/s, P = 6.45161 mol/s, reactor_feed = 172.581 mol/s, X_overall = 0.967742,
product = 43.5484 mol/s, y_inert = 0.775   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Recycle and purge balance: F0 = 100.0 mol/s, zA = 0.98, X = 0.25, f = 0.1 | SOLVED | R = 221.538 mol/s, P = 24.6154 mol/s, reactor_feed = 321.538 mol/s, X_overall = 0.769231, product = 75.3846 mol/s, y_inert = 0.08125 |

| question | outcome | result |
|---|---|---|
| Recycle and purge balance: F0 = 50.0 mol/s, zA = 0.9, X = 0.6, f = 0.05 | SOLVED | R = 122.581 mol/s, P = 6.45161 mol/s, reactor_feed = 172.581 mol/s, X_overall = 0.967742, product = 43.5484 mol/s, y_inert = 0.775 |
| Recycle and purge balance: zA = 0.98, X = 0.25, f = 0.1 | NEEDS_INPUT | chemeng_recycle_purge needs F0 (fresh feed flow (reactant A plus inert)). Example: solve('chemeng_recycle_purge', F0='10 |
| Recycle and purge balance: F0 = 100.0 mol/s, zA = 0.98, X = 0.25, f = 0.1, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Recycle and purge balance: F0 = 100 mol/s, zA = 98%, X = 25%, f = 0 | INVALID_INPUT | Reactor with recycle and purge: the purge fraction f must be a fraction between 0 and 1. |

### Unsteady well-mixed tank (concentration change) — `chemeng_unsteady_mixing_tank`

Concentration in a constant-volume well-mixed tank at time t after the inlet concentration changes (inflow equals outflow).

$$C(t) = C_{in} + (C_0 - C_{in})\, e^{-qt/V}$$

| input | meaning | condition |
|---|---|---|
| `V` | liquid volume of the tank | |
| `q` | volumetric flow in and out | |
| `C_in` | inlet concentration | |
| `C0` | concentration in the tank at $t = 0$ | |
| `t` | time | |

| output | meaning |
|---|---|
| `C` | concentration in the tank (and outlet) at time t |

- **Method:** solute balance V $dC/dt = q$ (C_in - C), integrated: $C = C_in + (C_0 - C_in)$ exp(-q t/V)
- **Assumes:** $V > 0$, $q > 0$; perfect mixing; constant volume (inflow = outflow)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_unsteady_mixing_tank', V=10.0, q=0.01, C_in=500.0,
C0=0.0, t=1000.0)
# -> C = 316.06 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_unsteady_mixing_tank', V=2.0, q=0.001, C_in=0.0,
C0=800.0, t=3600.0)
# -> C = 132.239 mol/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Unsteady mixing tank: V = 10.0 m^3, q = 0.01 m^3/s, C_in = | SOLVED | C = 316.06 mol/m^3 |

| question | outcome | result |
|---|---|---|
| 500.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s | | |
| Unsteady mixing tank: V = 2.0 m^3, q = 0.001 m^3/s, C_in = 0.0 mol/m^3, C0 = 800.0 mol/m^3, t = 3600.0 s | SOLVED | C = 132.239 mol/m^3 |
| Unsteady mixing tank: q = 0.01 m^3/s, C_in = 500.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s | NEEDS_INPUT | chemeng_unsteady_mixing_tank needs V (liquid volume of the tank). Example: solve('chemeng_unsteady_mixing_tank', V='10 m |
| Unsteady mixing tank: V = 10.0 m^3, q = 0.01 m^3/s, C_in = 500.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Unsteady mixing tank: V = 10 m^3, q = 36 m^3/h, C_in = 0.5 mol/L, C0 = 0 mol/L, t = 1000 s | SOLVED | C = 316.06 mol/m^3 |

### Filling or draining well-mixed tank (variable volume) — `chemeng_variable_volume_tank`

Volume and concentration of a well-mixed tank whose inflow and outflow differ, at time t.

$$C = C_{in} - (C_{in} - C_0)\left(\frac{V_0}{V}\right)^{q_{in}/(q_{in}-q_{out})},\ V = V_0 + (q_{in} - q_{out})t$$

| input | meaning | condition |
|---|---|---|
| `V0` | liquid volume at $t = 0$ | |
| `q_in` | volumetric inflow | |
| `q_out` | volumetric outflow | |
| `C_in` | inlet concentration | |
| `C0` | concentration in the tank at $t = 0$ | |
| `t` | time | |

| output | meaning |
|---|---|
| `V` | liquid volume at time t |
| `C` | concentration at time t |

- **Method:** total balance dV/dt = q_in - q_out and solute balance d(VC)/dt = q_in C_in - q_out C, integrated: $C = C_in - (C_in - C_0)$(V0/V)^(q_in/(q_in - q_out))
- **Assumes:** $V_0 > 0$, $q_in > 0$; the tank does not empty before t; perfect mixing
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_variable_volume_tank', V0=1.0, q_in=0.002, q_out=0.001,
C_in=1000.0, C0=0.0, t=1000.0)
# -> V = 2 m^3, C = 750 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_variable_volume_tank', V0=5.0, q_in=0.001, q_out=0.003,
C_in=0.0, C0=500.0, t=1000.0)
# -> V = 3 m^3, C = 387.298 mol/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Variable-volume tank: V0 = 1.0 m^3, q_in = 0.002 m^3/s, q_out = 0.001 m^3/s, C_in = 1000.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s | SOLVED | V = 2 m^3, C = 750 mol/m^3 |
| Variable-volume tank: V0 = 5.0 m^3, q_in = 0.001 m^3/s, q_out = 0.003 m^3/s, C_in = 0.0 mol/m^3, C0 = 500.0 mol/m^3, t = 1000.0 s | SOLVED | V = 3 m^3, C = 387.298 mol/m^3 |
| Variable-volume tank: q_in = 0.002 m^3/s, q_out = 0.001 m^3/s, C_in = 1000.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s | NEEDS_INPUT | chemeng_variable_volume_tank needs V0 (liquid volume at t = 0). Example: solve('chemeng_variable_volume_tank', V0='1 m^3 |
| Variable-volume tank: V0 = 1.0 m^3, q_in = 0.002 m^3/s, q_out = 0.001 m^3/s, C_in = 1000.0 mol/m^3, C0 = 0.0 mol/m^3, t = 1000.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Variable-volume tank: V0 = 1 m^3, q_in = 0.001 m^3/s, q_out = 0.003 m^3/s, C_in = 0 mol/m^3, C0 = 500 mol/m^3, t = 1000 s | INVALID_INPUT | Filling or draining well-mixed tank (variable volume): the tank would be empty before time t. |

## Combustion with excess air: air requirement and flue-gas composition —

`chemeng_combustion_flue_gas`

Theoretical oxygen, air supplied, flue-gas moles and wet and dry compositions, and the air-fuel mass ratio for complete combustion of a fuel with excess dry air (21 % O2, 79 % N2).

$$n_{O_2,th} = x + \tfrac{y}{4} + v - \tfrac{z}{2}, \quad n_{air} = \frac{(1+e)\,n_{O_2,th}}{0.21}$$

| input | meaning | condition |
|---|---|---|
| `fuel` | molecular formula of the fuel (C, H, O, N and S), e.g. CH4, C3H8, C2H5OH | |
| `excess_air` | excess air as a fraction of the theoretical air (20% = 0.2) | |

| output | meaning |
|---|---|
| `O2_theoretical` | theoretical O2, mol per mol fuel |
| `air` | air supplied, mol per mol fuel |
| `n_CO2` | CO2 in the flue gas, mol per mol fuel |
| `n_H2O` | H2O, mol per mol fuel |
| `n_SO2` | SO2, mol per mol fuel |
| `n_O2` | O2, mol per mol fuel |

| output | meaning |
|---|---|
| `n_N2` | N2, mol per mol fuel |
| `y_CO2_dry` | CO2 mole fraction, dry basis |
| `y_O2_dry` | O2 mole fraction, dry basis |
| `y_H2O_wet` | H2O mole fraction, wet basis |
| `AFR_mass` | air-fuel ratio, kg air per kg fuel |

- **Method:** O2 theoretical = C + H/4 + S - O/2 per mol fuel; air = (1 + e) $O_2/0.21$; products by element balance
- **Assumes:** complete combustion to CO2, H2O and SO2; fuel N leaves as N2; dry air 21 % $O_2/79$ % N2; 0 <= excess air <= 500 %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_combustion_flue_gas', fuel='CH4', excess_air=0.2)
# -> O2_theoretical = 2, air = 11.4286, n_CO2 = 1, n_H2O = 2, n_SO2 = 0, n_O2 = 0.4, n_N2 =
9.02857, y_CO2_dry = 0.0958904, y_O2_dry = 0.0383562, y_H2O_wet = 0.16092, AFR_mass = 20.5528
(SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_combustion_flue_gas', fuel='C3H8', excess_air=0.0)
# -> O2_theoretical = 5, air = 23.8095, n_CO2 = 3, n_H2O = 4, n_SO2 = 0, n_O2 = 0, n_N2 = 18.8095,
y_CO2_dry = 0.137555, y_O2_dry = 0, y_H2O_wet = 0.154982, AFR_mass = 15.5778   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_combustion_flue_gas', fuel='C2H5OH', excess_air=0.5)
# -> O2_theoretical = 3, air = 21.4286, n_CO2 = 2, n_H2O = 3, n_SO2 = 0, n_O2 = 1.5, n_N2 =
16.9286, y_CO2_dry = 0.0979021, y_O2_dry = 0.0734266, y_H2O_wet = 0.128049, AFR_mass = 13.4196
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Combustion flue gas: fuel = CH4, excess_air = 0.2 | SOLVED | O2_theoretical = 2, air = 11.4286, n_CO2 = 1, n_H2O = 2, n_SO2 = 0, n_O2 = 0.4, n_N2 = 9.02857, y_CO2_dry = 0.0958904, y_O2_dry = 0.0383562, y_H2O_wet = 0.16092, AFR_mass = 20.5528 |
| Combustion flue gas: fuel = C3H8, excess_air = 0.0 | SOLVED | O2_theoretical = 5, air = 23.8095, n_CO2 = 3, n_H2O = 4, n_SO2 = 0, n_O2 = 0, n_N2 = 18.8095, y_CO2_dry = 0.137555, y_O2_dry = 0, y_H2O_wet = 0.154982, AFR_mass = 15.5778 |
| Combustion flue gas: fuel = C2H5OH, excess_air = 0.5 | SOLVED | O2_theoretical = 3, air = 21.4286, n_CO2 = 2, n_H2O = 3, n_SO2 = 0, n_O2 = 1.5, n_N2 = 16.9286, y_CO2_dry = 0.0979021, y_O2_dry = 0.0734266, y_H2O_wet = 0.128049, AFR_mass = 13.4196 |
| Combustion flue gas: excess_air = 0.2 | NEEDS_INPUT | chemeng_combustion_flue_gas needs fuel (molecular formula of the fuel (C, H, O, N and S), e.g. CH4, C3H8, C2H5OH). Examp |
| Combustion flue gas: fuel = CH4, excess_air = 0.2, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Combustion flue gas: fuel = CH4, excess_air = 20% | SOLVED | O2_theoretical = 2, air = 11.4286, n_CO2 = 1, n_H2O = 2, n_SO2 = 0, n_O2 = 0.4, n_N2 = 9.02857, y_CO2_dry = 0.0958904, y_O2_dry = 0.0383562, y_H2O_wet = 0.16092, AFR_mass = 20.5528 |

| question | outcome | result |
|---|---|---|
| Combustion flue gas: fuel = CH4, excess_air = 20 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Combustion flue gas: fuel = CH4Cl, excess_air = 0.2 | INVALID_INPUT | Combustion with excess air: air requirement and flue-gas composition: element Cl is not supported (fuels of C, H, O, N a |

### Sensible heat with a polynomial heat capacity — `chemeng_sensible_heat_cp_polynomial`

Heat needed to take n moles from T1 to T2 when $cp = a + bT + cT^2 + dT^3$ (J/(mol K), T in K), and the mean heat capacity over the range.

$$Q = n \int_{T_1}^{T_2} (a + bT + cT^2 + dT^3)\, dT$$

| input | meaning | condition |
|---|---|---|
| `n` | amount of substance | |
| `a` | cp coefficient a, J/(mol K) | |
| `b` | cp coefficient b, J/(mol K^2) | |
| `c` | cp coefficient c, J/(mol K^3) (default 0) | |
| `d` | cp coefficient d, J/(mol K^4) (default 0) | |
| `T1` | initial temperature | |
| `T2` | final temperature | |

| output | meaning |
|---|---|
| `Q` | heat added (negative when cooled) |
| `cp_mean` | mean heat capacity between T1 and T2 |

- **Method:** Q = n * integral of cp dT, integrated term by term
- **Assumes:** no phase change between T1 and T2; T in kelvin inside the polynomial
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_sensible_heat_cp_polynomial', n=10.0, a=29.0, b=0.002,
c=0.0, d=0.0, T1=300.0, T2=500.0)
# -> Q = 59600 J, cp_mean = 29.8 J/(mol*K)   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_sensible_heat_cp_polynomial', n=1.0, a=19.89, b=0.05024,
c=1.269e-05, d=-1.101e-08, T1=298.15, T2=1000.0)
# -> Q = 38233.9 J, cp_mean = 54.4759 J/(mol*K)   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Sensible heat with polynomial cp: n = 10.0 mol, a = 29.0, b = | SOLVED | Q = 59600 J, cp_mean = 29.8 J/(mol*K) |

| question | outcome | result |
|---|---|---|
| 0.002, c = 0.0, d = 0.0, T1 = 300.0 K, T2 = 500.0 K | | |
| Sensible heat with polynomial cp: n = 1.0 mol, a = 19.89, b = 0.05024, c = 1.269e-05, d = -1.101e-08, T1 = 298.15 K, T2 = 1000.0 K | SOLVED | Q = 38233.9 J, cp_mean = 54.4759 J/(mol*K) |
| Sensible heat with polynomial cp: a = 29.0, b = 0.002, c = 0.0, d = 0.0, T1 = 300.0 K, T2 = 500.0 K | NEEDS_INPUT | chemeng_sensible_heat_cp_polynomial needs n (amount of substance). Example: solve('chemeng_sensible_heat_cp_polynomial', |
| Sensible heat with polynomial cp: n = 10.0 mol, a = 29.0, b = 0.002, c = 0.0, d = 0.0, T1 = 300.0 K, T2 = 500.0 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Sensible heat: n = 10 mol, a = 29, b = 0.002, T1 = 26.85 degC, T2 = 226.85 degC | SOLVED | Q = 59600 J, cp_mean = 29.8 J/(mol*K) |

## Heat of reaction at another temperature (Kirchhoff) —

`chemeng_reaction_enthalpy_at_temperature`

The heat of reaction at T from its value at T0 and the polynomial heat capacities of the species (Kirchhoff's law).

$$\Delta H_r(T) = \Delta H_r(T_0) + \int_{T_0}^{T} \sum_i \nu_i c_{p,i}\, dT$$

| input | meaning | condition |
|---|---|---|
| `nu` | stoichiometric coefficients, negative for reactants and positive for products | |
| `cp` | heat-capacity coefficients, one row (a, b, c, d) per species in the order of nu; $cp = a + bT + cT^2 + dT^3$ in J/(mol K) | |
| `dH0` | heat of reaction at T0, per mole of extent | |
| `T0` | temperature of the known heat of reaction | |
| `T` | temperature wanted | |

| output | meaning |
|---|---|
| `dH` | heat of reaction at T, per mole of extent |

- **Method:** dH(T) = dH(T0) + integral from T0 to T of sum_i nu_i cp_i dT
- **Assumes:** no phase change of any species between T0 and T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_reaction_enthalpy_at_temperature', nu=[-1.0, -3.0, 2.0],
cp=[[27.27, 0.004927, 0.0, 0.0], [27.01, 0.003509, 0.0, 0.0], [24.62, 0.03753, -7.1e-06, 0.0]],
dH0=-92220.0, T0=298.15, T=700.0)
# -> dH = -105497 J/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_reaction_enthalpy_at_temperature', nu=[-1.0, -0.5, 1.0],
cp=[[29.0, 0.0], [29.4, 0.004], [30.0, 0.01]], dH0=-241826.0, T0=298.15, T=1000.0)
# -> dH = -247797 J/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Heat of reaction at temperature: nu = [-1.0, -3.0, 2.0], cp = [[27.27, 0.004927, 0.0, 0.0], [27.01, 0.003509, 0.0, 0.0], [24.62, 0.03753, -7.1e-06, 0.0]], dH0 = -92220.0 J/mol, T0 = 298.15 K, T = 700.0 K | SOLVED | dH = -105497 J/mol |
| Heat of reaction at temperature: nu = [-1.0, -0.5, 1.0], cp = [[29.0, 0.0], [29.4, 0.004], [30.0, 0.01]], dH0 = -241826.0 J/mol, T0 = 298.15 K, T = 1000.0 K | SOLVED | dH = -247797 J/mol |
| Heat of reaction at temperature: cp = [[27.27, 0.004927, 0.0, 0.0], [27.01, 0.003509, 0.0, 0.0], [24.62, 0.03753, -7.1e-06, 0.0]], dH0 = -92220.0 J/mol, T0 = 298.15 K, T = 700.0 K | NEEDS_INPUT | chemeng_reaction_enthalpy_at_temperature needs nu (stoichiometric coefficients, negative for reactants and positive for |
| Heat of reaction at temperature: nu = [-1.0, -3.0, 2.0], cp = [[27.27, 0.004927, 0.0, 0.0], [27.01, 0.003509, 0.0, 0.0], [24.62, 0.03753, -7.1e-06, 0.0]], dH0 = -92220.0 J/mol, T0 = 298.15 K, T = 700.0 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Adiabatic reactor outlet temperature — `chemeng_adiabatic_reactor_temperature`

Outlet temperature of an adiabatic reactor from the feed, the conversion of the limiting reactant, the heat of reaction at a reference temperature and polynomial heat capacities.

$$\sum_i n_{i0} \int_{T_{in}}^{T_{ref}} c_{p,i} dT + \xi \, \Delta H_r(T_{ref}) + \sum_i n_i \int_{T_{ref}}^{T_{out}} c_{p,i} dT = 0$$

| input | meaning | condition |
|---|---|---|
| `n0` | feed amounts of each species, mol (or mol/s) on one basis | |
| `nu` | stoichiometric coefficients, negative for reactants | |
| `cp` | heat-capacity coefficients, one row (a, b, c, d) per species; $cp = a + bT + cT^2 + dT^3$ in J/(mol K) | |
| `dH` | heat of reaction at T_ref, per mole of extent | |
| `T_ref` | reference temperature of dH | |
| `T_in` | feed temperature | |
| `key` | number of the limiting reactant in the species list (1 = first) | |
| `X` | conversion of the limiting reactant | |

| output | meaning |
|---|---|
| `T_out` | adiabatic outlet temperature |

- **Method:** energy balance along the path: cool the feed to T_ref, react at T_ref, heat the products to T_out; T_out by bisection
- **Assumes:** adiabatic, no shaft work, no phase change; outlet between 200 and 3200 K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_adiabatic_reactor_temperature', n0=[1.0, 2.0, 0.0], nu=
[-1.0, -1.0, 1.0], cp=[[30.0, 0.01], [29.0, 0.004], [45.0, 0.02]], dH=-60000.0, T_ref=298.15,
T_in=400.0, key=1, X=0.8)
# -> T_out = 932.348 K   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_adiabatic_reactor_temperature', n0=[2.0, 0.0], nu=[-1.0,
2.0], cp=[[40.0, 0.02], [25.0, 0.01]], dH=30000.0, T_ref=298.15, T_in=900.0, key=1, X=0.3)
# -> T_out = 717.393 K   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Adiabatic reactor temperature: n0 = [1.0, 2.0, 0.0], nu = [-1.0, -1.0, 1.0], cp = [[30.0, 0.01], [29.0, 0.004], [45.0, 0.02]], dH = -60000.0 J/mol, T_ref = 298.15 K, T_in = 400.0 K, key = 1, X = 0.8 | SOLVED | T_out = 932.348 K |
| Adiabatic reactor temperature: n0 = [2.0, 0.0], nu = [-1.0, 2.0], cp = [[40.0, 0.02], [25.0, 0.01]], dH = 30000.0 J/mol, T_ref = 298.15 K, T_in = 900.0 K, key = 1, X = 0.3 | SOLVED | T_out = 717.393 K |
| Adiabatic reactor temperature: nu = [-1.0, -1.0, 1.0], cp = [[30.0, 0.01], [29.0, 0.004], [45.0, 0.02]], dH = -60000.0 J/mol, T_ref = 298.15 K, T_in = 400.0 K, key = 1, X = 0.8 | NEEDS_INPUT | chemeng_adiabatic_reactor_temperature needs n0 (feed amounts of each species, mol (or mol/s) on one basis). Example: sol |
| Adiabatic reactor temperature: n0 = [1.0, 2.0, 0.0], nu = [-1.0, -1.0, 1.0], cp = [[30.0, 0.01], [29.0, 0.004], [45.0, 0.02]], dH = -60000.0 J/mol, T_ref = 298.15 K, T_in = 400.0 K, key = 1, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Adiabatic flame temperature (complete combustion, NASA polynomials) — `chemeng_adiabatic_flame_temperature`

Adiabatic flame temperature of a gaseous fuel burned with excess dry air at constant pressure, complete combustion without dissociation, from NASA 7-coefficient polynomials (GRI-Mech 3.0 data).

$$\sum_p n_p\, h_p(T_{ad}) = \sum_r n_r\, h_r(T_{in}), \qquad \frac{h}{RT} = a_1 + \frac{a_2 T}{2} + \frac{a_3 T^2}{3} + \frac{a_4 T^3}{4} + \frac{a_5 T^4}{5} + \frac{a_6}{T}$$

| input | meaning | condition |
|---|---|---|
| `fuel` | fuel: CH4, C2H6, C3H8, H2, CO, C2H4, C2H2, CH3OH | |
| `excess_air` | excess air as a fraction of theoretical (0 = stoichiometric) | |
| `T_in` | temperature of the fuel and air entering | |

| output | meaning |
|---|---|
| `T_ad` | adiabatic flame temperature |

- **Method:** energy balance H_products(T_ad) = H_reactants(T_in) with NASA polynomial enthalpies (including formation enthalpies), solved by bisection
- **Assumes:** complete combustion to CO2 and H2O vapour, no dissociation; 300 $K \leq T_in \leq 1000$ K; result <= 3500 K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_adiabatic_flame_temperature', fuel='CH4',
excess_air=0.0, T_in=300.0)
# -> T_ad = 2326.32 K   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_adiabatic_flame_temperature', fuel='C3H8',
excess_air=0.5, T_in=400.0)
# -> T_ad = 1906.88 K   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Adiabatic flame temperature: fuel = CH4, excess_air = 0.0, T_in = 300.0 K | SOLVED | T_ad = 2326.32 K |
| Adiabatic flame temperature: fuel = C3H8, excess_air = 0.5, T_in = 400.0 K | SOLVED | T_ad = 1906.88 K |
| Adiabatic flame temperature: excess_air = 0.0, T_in = 300.0 K | NEEDS_INPUT | chemeng_adiabatic_flame_temperature needs fuel (fuel: CH4, C2H6, C3H8, H2, CO, C2H4, C2H2, CH3OH). Example: solve('cheme |
| Adiabatic flame temperature: fuel = CH4, excess_air = 0.0, T_in = 300.0 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Adiabatic flame temperature: fuel = CH4, excess_air = 0%, T_in = 26.85 degC | SOLVED | T_ad = 2326.32 K |
| Adiabatic flame temperature: fuel = C8H18, excess_air = 0.2, T_in = 300 K | INVALID_INPUT | fuel: one of CH4, C2H6, C3H8, H2, CO, C2H4, C2H2, CH3OH; got 'c8h18'. |

## Batch heating of a stirred tank: temperature after a time — `chemeng_batch_heating_temperature`

Temperature of a well-stirred batch heated or cooled by a medium at constant temperature (condensing steam, boiling refrigerant) after time t.

$$T = T_s - (T_s - T_0)\, e^{-UA\, t/(m c_p)}$$

| input | meaning | condition |
|---|---|---|
| `m` | mass of liquid in the tank | |
| `cp` | specific heat of the liquid | |
| `UA` | overall heat-transfer coefficient times area of the coil or jacket | |
| `T_s` | temperature of the heating or cooling medium (constant, e.g. condensing steam) | |
| `T0` | initial liquid temperature | |
| `t` | heating time | |

| output | meaning |
|---|---|
| `T` | liquid temperature at time t |

- **Method:** energy balance m cp $dT/dt = UA$ (T_s - T), integrated
- **Assumes:** well stirred; constant UA and cp; medium at constant temperature
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_heating_temperature', m=1000.0, cp=4180.0,
UA=2000.0, T_s=393.15, T0=293.15, t=3600.0)
# -> T = 375.288 K   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_heating_temperature', m=500.0, cp=2000.0,
UA=500.0, T_s=273.15, T0=353.15, t=1800.0)
# -> T = 305.676 K   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Batch heating temperature: m = 1000.0 kg, cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, t = 3600.0 s | SOLVED | T = 375.288 K |
| Batch heating temperature: m = 500.0 kg, cp = 2000.0 J/(kg*K), UA = 500.0 W/K, T_s = 273.15 K, T0 = 353.15 K, t = 1800.0 s | SOLVED | T = 305.676 K |
| Batch heating temperature: cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, t = 3600.0 s | NEEDS_INPUT | chemeng_batch_heating_temperature needs m (mass of liquid in the tank). Example: solve('chemeng_batch_heating_temperatur |
| Batch heating temperature: m = 1000.0 kg, cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, t = 3600.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Batch heating temperature: m = 1 t, cp = 4.18 kJ/(kg*K), UA = 2 kW/K, | SOLVED | T = 375.288 K |

| question | outcome | result |
|---|---|---|
| T_s = 120 degC, T0 = 20 degC, t = 1 h | | |

### Batch heating of a stirred tank: time to reach a temperature — `chemeng_batch_heating_time`

Time for a well-stirred batch heated or cooled by a medium at constant temperature to reach a stated temperature.

$$t = \frac{mc_p}{UA} \ln \frac{T_s - T_0}{T_s - T}$$

| input | meaning | condition |
|---|---|---|
| `m` | mass of liquid in the tank | |
| `cp` | specific heat of the liquid | |
| `UA` | overall heat-transfer coefficient times area of the coil or jacket | |
| `T_s` | temperature of the heating or cooling medium (constant, e.g. condensing steam) | |
| `T0` | initial liquid temperature | |
| `T` | temperature to reach | |

| output | meaning |
|---|---|
| `t` | time needed |

- **Method:** t = (m cp/UA) ln($(T_s - T_0)/(T_s - T)$)
- **Assumes:** T between T0 and T_s; well stirred; constant UA and cp
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_heating_time', m=1000.0, cp=4180.0, UA=2000.0,
T_s=393.15, T0=293.15, T=353.15)
# -> t = 1915.05 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_heating_time', m=500.0, cp=2000.0, UA=500.0,
T_s=273.15, T0=353.15, T=300.0)
# -> t = 2183.52 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Batch heating time: m = 1000.0 kg, cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, T = 353.15 K | SOLVED | t = 1915.05 s |
| Batch heating time: m = 500.0 kg, cp = 2000.0 J/(kg*K), UA = 500.0 W/K, T_s = 273.15 K, T0 = 353.15 K, T = 300.0 K | SOLVED | t = 2183.52 s |

| question | outcome | result |
|---|---|---|
| Batch heating time: cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, T = 353.15 K | NEEDS_INPUT | chemeng_batch_heating_time needs m (mass of liquid in the tank). Example: solve('chemeng_batch_heating_time', m='1000 kg |
| Batch heating time: m = 1000.0 kg, cp = 4180.0 J/(kg*K), UA = 2000.0 W/K, T_s = 393.15 K, T0 = 293.15 K, T = 353.15 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Batch heating time: m = 1000 kg, cp = 4180 J/(kg*K), UA = 2000 W/K, T_s = 393.15 K, T0 = 293.15 K, T = 400 K | INVALID_INPUT | Batch heating of a stirred tank: time to reach a temperature: T must lie between T0 and the medium temperature T_s. |

### Batch cooling (or heating) with a non-isothermal coolant in a coil — `chemeng_batch_coil_cooling`

Temperature of a well-stirred batch after time t when a coolant (or heating liquid) flows once through a coil and leaves warmer (Kern's batch cooling).

$$T = t_{in} + (T_0 - t_{in}) \exp\left[-\frac{W c_{p,c}\left(1 - e^{-UA/(W c_{p,c})}\right) t}{M c_p}\right]$$

| input | meaning | condition |
|---|---|---|
| `M` | mass of liquid in the tank | |
| `cp` | specific heat of the batch liquid | |
| `W` | coolant mass flow | |
| `cp_c` | specific heat of the coolant | |
| `t_in` | coolant inlet temperature | |
| `UA` | overall heat-transfer coefficient times coil area | |
| `T0` | initial batch temperature | |
| `t` | time | |

| output | meaning |
|---|---|
| `T` | batch temperature at time t |

- **Method:** coolant outlet approaches the batch temperature: T - t_out = (T - t_in) exp(-UA/(W cp_c)); tank balance M cp $dT/dt = -W$ cp_c (t_out - t_in), integrated
- **Assumes:** well-stirred batch; constant UA, cp and coolant flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_coil_cooling', M=5000.0, cp=2500.0, W=2.0,
cp_c=4180.0, t_in=288.15, UA=3000.0, T0=393.15, t=7200.0)
# -> T = 312.732 K   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_coil_cooling', M=1000.0, cp=4180.0, W=0.5,
cp_c=4180.0, t_in=363.15, UA=1000.0, T0=293.15, t=3600.0)
# -> T = 327.845 K   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Batch coil cooling: M = 5000.0 kg, cp = 2500.0 J/(kg*K), W = 2.0 kg/s, cp_c = 4180.0 J/(kg*K), t_in = 288.15 K, UA = 3000.0 W/K, T0 = 393.15 K, t = 7200.0 s | SOLVED | T = 312.732 K |
| Batch coil cooling: M = 1000.0 kg, cp = 4180.0 J/(kg*K), W = 0.5 kg/s, cp_c = 4180.0 J/(kg*K), t_in = 363.15 K, UA = 1000.0 W/K, T0 = 293.15 K, t = 3600.0 s | SOLVED | T = 327.845 K |
| Batch coil cooling: cp = 2500.0 J/(kg*K), W = 2.0 kg/s, cp_c = 4180.0 J/(kg*K), t_in = 288.15 K, UA = 3000.0 W/K, T0 = 393.15 K, t = 7200.0 s | NEEDS_INPUT | chemeng_batch_coil_cooling needs M (mass of liquid in the tank). Example: solve('chemeng_batch_coil_cooling', M='5000 kg |
| Batch coil cooling: M = 5000.0 kg, cp = 2500.0 J/(kg*K), W = 2.0 kg/s, cp_c = 4180.0 J/(kg*K), t_in = 288.15 K, UA = 3000.0 W/K, T0 = 393.15 K, t = 7200.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Batch reactor time for a conversion (first order) — `chemeng_batch_time_first_order`

Reaction time an isothermal batch reactor needs to reach conversion X for a first-order reaction.

$$t = \frac{1}{k} \ln \frac{1}{1 - X}$$

| input | meaning | condition |
|---|---|---|
| `k` | first-order rate constant | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `t` | reaction time |

- **Method:** design equation t = C_A0 * integral of $dX/(-r_A) = ln(1/(1 - X))$/k
- **Assumes:** $0 \leq X < 1$; isothermal, constant volume, $-r_A = k$ C_A
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_time_first_order', k=0.001, X=0.9)
# -> t = 2302.59 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_time_first_order', k=0.05, X=0.5)
# -> t = 13.8629 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order batch reactor time: k = 0.001 1/s, X = 0.9 | SOLVED | t = 2302.59 s |
| First-order batch reactor time: k = 0.05 1/s, X = 0.5 | SOLVED | t = 13.8629 s |
| First-order batch reactor time: X = 0.9 | NEEDS_INPUT | chemeng_batch_time_first_order needs k (first-order rate constant). Example: solve('chemeng_batch_time_first_order', k=' |
| First-order batch reactor time: k = 0.001 1/s, X = 0.9, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| First-order batch reactor time: k = 0.06 1/min, X = 90% | SOLVED | t = 2302.59 s |
| First-order batch reactor time: k = 0.001 1/s, X = 1 | INVALID_INPUT | Batch reactor time for a conversion (first order): the conversion X must be a fraction between 0 and 1. |

## Batch reactor time for a conversion (second order) — `chemeng_batch_time_second_order`

Reaction time an isothermal batch reactor needs to reach conversion X when -r_A = k $C_A^2$.

$$t = \frac{X}{k C_{A0}(1 - X)}$$

| input | meaning | condition |
|---|---|---|
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `t` | reaction time |

- **Method:** t = C_A0 * integral of dX/(k $C_A 0^2\ (1 - X)^2$) $= X/(k$ C_A0 $(1 - X)$)
- **Assumes:** $0 \leq X < 1$; isothermal, constant volume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_time_second_order', k=0.0001, CA0=1000.0, X=0.8)
# -> t = 40 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_time_second_order', k=5e-06, CA0=2000.0, X=0.5)
# -> t = 100 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order batch reactor time: k = 0.0001 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8 | SOLVED | t = 40 s |
| Second-order batch reactor time: k = 5e-06 m^3/(mol*s), CA0 = 2000.0 mol/m^3, X = 0.5 | SOLVED | t = 100 s |
| Second-order batch reactor time: CA0 = 1000.0 mol/m^3, X = 0.8 | NEEDS_INPUT | chemeng_batch_time_second_order needs k (second-order rate constant (-r_A = k C_A^2)). Example: solve('chemeng_batch_tim |
| Second-order batch reactor time: k = 0.0001 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Second-order batch reactor time: k = 0.1 L/(mol*s), CA0 = 1 mol/L, X = 0.8 | SOLVED | t = 40 s |

### Batch reactor conversion after a time (first order) — `chemeng_batch_conversion_first_order`

Conversion reached in an isothermal batch reactor after time t for a first-order reaction.

$$X = 1 - e^{-kt}$$

| input | meaning | condition |
|---|---|---|
| `k` | first-order rate constant | |
| `t` | reaction time | |

| output | meaning |
|---|---|
| `X` | conversion of A |

- **Method:** $X = 1 - exp$(-k t)
- **Assumes:** isothermal, constant volume, $-r_A = k$ C_A
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_conversion_first_order', k=0.001, t=1000.0)
# -> X = 0.632121   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_conversion_first_order', k=0.02, t=60.0)
# -> X = 0.698806   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order batch reactor conversion: k = 0.001 1/s, t = 1000.0 s | SOLVED | X = 0.632121 |
| First-order batch reactor conversion: k = 0.02 1/s, t = | SOLVED | X = 0.698806 |

| question | outcome | result |
|---|---|---|
| 60.0 s | | |
| First-order batch reactor conversion: t = 1000.0 s | NEEDS_INPUT | chemeng_batch_conversion_first_order needs k (first-order rate constant). Example: solve('chemeng_batch_conversion_first |
| First-order batch reactor conversion: k = 0.001 1/s, t = 1000.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| First-order batch reactor conversion: k = 0.06 1/min, t = 1000 s | SOLVED | X = 0.632121 |

### Batch reactor conversion after a time (second order) — `chemeng_batch_conversion_second_order`

Conversion and concentration reached in an isothermal batch reactor after time t when -r_A = k $C_A^2$.

$$X = \frac{kC_{A0}t}{1 + kC_{A0}t}$$

| input | meaning | condition |
|---|---|---|
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |
| `t` | reaction time | |

| output | meaning |
|---|---|
| `X` | conversion of A |
| `CA` | concentration of A at time t |

- **Method:** $1/C_A = 1/C_A0 + k$ t
- **Assumes:** isothermal, constant volume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_batch_conversion_second_order', k=0.0001, CA0=1000.0,
t=40.0)
# -> X = 0.8, CA = 200 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_batch_conversion_second_order', k=5e-06, CA0=2000.0,
t=600.0)
# -> X = 0.857143, CA = 285.714 mol/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order batch reactor conversion: k = 0.0001 m^3/(mol*s), CA0 = 1000.0 mol/m^3, t = 40.0 s | SOLVED | X = 0.8, CA = 200 mol/m^3 |

| question | outcome | result |
|---|---|---|
| Second-order batch reactor conversion: k = 5e-06 m^3/(mol*s), CA0 = 2000.0 mol/m^3, t = 600.0 s | SOLVED | X = 0.857143, CA = 285.714 mol/m^3 |
| Second-order batch reactor conversion: CA0 = 1000.0 mol/m^3, t = 40.0 s | NEEDS_INPUT | chemeng_batch_conversion_second_order needs k (second-order rate constant (-r_A = k C_A^2)). Example: solve('chemeng_bat |
| Second-order batch reactor conversion: k = 0.0001 m^3/(mol*s), CA0 = 1000.0 mol/m^3, t = 40.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### CSTR volume for a conversion (first order) — `chemeng_cstr_volume_first_order`

Volume and space time of a CSTR that reaches conversion X for a first-order liquid-phase reaction.

$$V = \frac{v_0\, X}{k(1 - X)}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `k` | first-order rate constant | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |
| `tau` | space time $V/v_0$ |

- **Method:** design equation $V = F_A0$ X/(-r_A) at the exit concentration: $V = v_0$ X/(k $(1 - X)$)
- **Assumes:** $0 \le X < 1$; steady state, perfect mixing, constant density, $-r_A = k$ C_A
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstr_volume_first_order', v0=0.01, k=0.001, X=0.9)
# -> V = 90 m^3, tau = 9000 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstr_volume_first_order', v0=0.002, k=0.05, X=0.5)
# -> V = 0.04 m^3, tau = 20 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order CSTR volume: v0 = 0.01 m^3/s, k = 0.001 1/s, X = 0.9 | SOLVED | V = 90 m^3, tau = 9000 s |

| question | outcome | result |
|---|---|---|
| First-order CSTR volume: v0 = 0.002 m^3/s, k = 0.05 1/s, X = 0.5 | SOLVED | V = 0.04 m^3, tau = 20 s |
| First-order CSTR volume: k = 0.001 1/s, X = 0.9 | NEEDS_INPUT | chemeng_cstr_volume_first_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstr_volume_first_order', v0='0 |
| First-order CSTR volume: v0 = 0.01 m^3/s, k = 0.001 1/s, X = 0.9, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| First-order CSTR volume: v0 = 36 m^3/h, k = 0.06 1/min, X = 90% | SOLVED | V = 90 m^3, tau = 9000 s |

### CSTR conversion for a volume (first order) — `chemeng_cstr_conversion_first_order`

Steady-state conversion of a CSTR of volume V for a first-order liquid-phase reaction.

$$X = \frac{k\tau}{1 + k\tau},\ \tau = V/v_0$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | reactor volume | |
| `k` | first-order rate constant | |

| output | meaning |
|---|---|
| `X` | conversion of A |

- **Method:** $X = k$ tau/(1 + k tau), $\tau = V/v_0$ (the Damkohler number k tau)
- **Assumes:** steady state, perfect mixing, constant density, $-r_A = k$ C_A
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstr_conversion_first_order', v0=0.01, V=20.0, k=0.001)
# -> X = 0.666667   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstr_conversion_first_order', v0=0.002, V=0.1, k=0.05)
# -> X = 0.714286   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order CSTR conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, k = 0.001 1/s | SOLVED | X = 0.666667 |
| First-order CSTR conversion: v0 = 0.002 m^3/s, V = 0.1 m^3, k = 0.05 1/s | SOLVED | X = 0.714286 |

| question | outcome | result |
|---|---|---|
| First-order CSTR conversion: V = 20.0 m^3, k = 0.001 1/s | NEEDS_INPUT | chemeng_cstr_conversion_first_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstr_conversion_first_order |
| First-order CSTR conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, k = 0.001 1/s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### CSTR conversion for a volume (second order) — `chemeng_cstr_conversion_second_order`

Steady-state conversion and exit concentration of a CSTR of volume V when -r_A = k $C_A^{2.}$

$$\frac{C_A}{C_{A0}} = \frac{2}{1+\sqrt{1+4\,k\tau C_{A0}}}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | reactor volume | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |

| output | meaning |
|---|---|
| `X` | conversion of A |
| `CA` | exit concentration of A |

- **Method:** balance $C_A0 - C_A = k$ tau $C_A^2$ solved for C_A (positive root, cancellation-free form)
- **Assumes:** steady state, perfect mixing, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstr_conversion_second_order', v0=0.01, V=2.0, k=1e-05,
CA0=1000.0)
# -> X = 0.5, CA = 500 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstr_conversion_second_order', v0=0.001, V=1.0, k=1e-06,
CA0=500.0)
# -> X = 0.267949, CA = 366.025 mol/m^3   (SOLVED)
```

#### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Second-order CSTR conversion: v0 = 0.01 m^3/s, V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3 | SOLVED | X = 0.5, CA = 500 mol/m^3 |
| Second-order CSTR conversion: v0 = 0.001 m^3/s, V = 1.0 m^3, k = 1e-06 m^3/(mol*s), CA0 = 500.0 mol/m^3 | SOLVED | X = 0.267949, CA = 366.025 mol/m^3 |

| question | outcome | result |
|---|---|---|
| Second-order CSTR conversion: V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3 | NEEDS_INPUT | chemeng_cstr_conversion_second_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstr_conversion_second_ord |
| Second-order CSTR conversion: v0 = 0.01 m^3/s, V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### PFR volume for a conversion (first order, liquid) — `chemeng_pfr_volume_first_order`

Volume of an isothermal plug-flow reactor that reaches conversion X for a first-order constant-density reaction.

$$V = \frac{v_0}{k} \ln \frac{1}{1 - X}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `k` | first-order rate constant | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |

- **Method:** V = F_A0 * integral of $dX/(-r_A) = (v_0/k)$ ln$(1/(1 - X))$
- **Assumes:** $0 \le X < 1$; plug flow, isothermal, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_volume_first_order', v0=0.01, k=0.001, X=0.9)
# -> V = 23.0259 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_volume_first_order', v0=0.002, k=0.05, X=0.5)
# -> V = 0.0277259 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order PFR volume: v0 = 0.01 m^3/s, k = 0.001 1/s, X = 0.9 | SOLVED | V = 23.0259 m^3 |
| First-order PFR volume: v0 = 0.002 m^3/s, k = 0.05 1/s, X = 0.5 | SOLVED | V = 0.0277259 m^3 |
| First-order PFR volume: k = 0.001 1/s, X = 0.9 | NEEDS_INPUT | chemeng_pfr_volume_first_order needs v0 (volumetric feed flow). Example: solve('chemeng_pfr_volume_first_order', v0='0.0 |

| question | outcome | result |
|---|---|---|
| First-order PFR volume: v0 = 0.01 m^3/s, k = 0.001 1/s, X = 0.9, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| First-order PFR volume: v0 = 600 L/min, k = 0.06 1/min, X = 90% | SOLVED | V = 23.0259 m^3 |

### PFR volume for a conversion (second order, liquid) — `chemeng_pfr_volume_second_order`

Volume of an isothermal plug-flow reactor that reaches conversion X when -r_A = k $C_A^2$ (constant density).

$$V = \frac{v_0}{kC_{A0}} \frac{X}{1-X}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |

- **Method:** V = (v0/(k C_A0)) $X/(1-X)$
- **Assumes:** $0 \le X < 1$; plug flow, isothermal, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_volume_second_order', v0=0.01, k=1e-05, CA0=1000.0,
X=0.8)
# -> V = 4 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_volume_second_order', v0=0.001, k=1e-06, CA0=500.0,
X=0.5)
# -> V = 2 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order PFR volume: v0 = 0.01 m^3/s, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8 | SOLVED | V = 4 m^3 |
| Second-order PFR volume: v0 = 0.001 m^3/s, k = 1e-06 m^3/(mol*s), CA0 = 500.0 mol/m^3, X = 0.5 | SOLVED | V = 2 m^3 |

| question | outcome | result |
|---|---|---|
| Second-order PFR volume: k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8 | NEEDS_INPUT | chemeng_pfr_volume_second_order needs v0 (volumetric feed flow). Example: solve('chemeng_pfr_volume_second_order', v0='0 |
| Second-order PFR volume: v0 = 0.01 m^3/s, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### PFR conversion for a volume (first order, liquid) — `chemeng_pfr_conversion_first_order`

Conversion at the exit of an isothermal plug-flow reactor of volume V for a first-order constant-density reaction.

$$X = 1 - e^{-k\tau}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | reactor volume | |
| `k` | first-order rate constant | |

| output | meaning |
|---|---|
| `X` | conversion of A |

- **Method:** $X = 1 - exp$(-k tau), $\tau = V/v_0$
- **Assumes:** plug flow, isothermal, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_conversion_first_order', v0=0.01, V=20.0, k=0.001)
# -> X = 0.864665   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_conversion_first_order', v0=0.002, V=0.1, k=0.05)
# -> X = 0.917915   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order PFR conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, k = 0.001 1/s | SOLVED | X = 0.864665 |
| First-order PFR conversion: v0 = 0.002 m^3/s, V = 0.1 m^3, k = 0.05 1/s | SOLVED | X = 0.917915 |
| First-order PFR conversion: V = 20.0 m^3, k = 0.001 1/s | NEEDS_INPUT | chemeng_pfr_conversion_first_order needs v0 (volumetric feed flow). Example: solve('chemeng_pfr_conversion_first_order', |
| First-order PFR conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

| question | outcome | result |
|---|---|---|
| k = 0.001 1/s, zz = 1 | | |

### PFR conversion for a volume (second order, liquid) — `chemeng_pfr_conversion_second_order`

Exit conversion and concentration of an isothermal plug-flow reactor of volume V when -r_A = k $C_A^2$ (constant density).

$$X = \frac{k\tau C_{A0}}{1 + k\tau C_{A0}}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | reactor volume | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |

| output | meaning |
|---|---|
| `X` | conversion of A |
| `CA` | exit concentration of A |

- **Method:** $1/C_A = 1/C_A0 + k$ tau
- **Assumes:** plug flow, isothermal, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_conversion_second_order', v0=0.01, V=2.0, k=1e-05,
CA0=1000.0)
# -> X = 0.666667, CA = 333.333 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_conversion_second_order', v0=0.001, V=1.0, k=1e-06,
CA0=500.0)
# -> X = 0.333333, CA = 333.333 mol/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order PFR conversion: v0 = 0.01 m^3/s, V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3 | SOLVED | X = 0.666667, CA = 333.333 mol/m^3 |
| Second-order PFR conversion: v0 = 0.001 m^3/s, V = 1.0 m^3, k = 1e-06 m^3/(mol*s), CA0 = 500.0 mol/m^3 | SOLVED | X = 0.333333, CA = 333.333 mol/m^3 |

| question | outcome | result |
|---|---|---|
| Second-order PFR conversion: V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3 | NEEDS_INPUT | chemeng_pfr_conversion_second_order needs v0 (volumetric feed flow). Example: solve('chemeng_pfr_conversion_second_order |
| Second-order PFR conversion: v0 = 0.01 m^3/s, V = 2.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Gas-phase PFR volume with volume change (first order) — `chemeng_pfr_gas_volume_first_order`

Volume of an isothermal, isobaric gas-phase plug-flow reactor for conversion X of a first-order reaction whose mole number changes (factor eps).

$$V = \frac{F_{A0}}{kC_{A0}}\left[(1+\varepsilon)\ln\frac{1}{1-X} - \varepsilon X\right]$$

| input | meaning | condition |
|---|---|---|
| `FA0` | molar feed flow of A | |
| `CA0` | concentration of A at the inlet | |
| `k` | first-order rate constant | |
| `eps` | volume-change factor eps = y_A0 * (change in moles per mole of A); e.g. A -> 2B with pure A: eps = 1 | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |

- **Method:** V = F_A0 * integral of (1 + eps X)/(k C_A0 $(1-X)$) dX = (F_A0/(k C_A0)) [(1 + eps) ln( $1/(1-X)$) - eps X]
- **Assumes:** isothermal, isobaric ideal gas; eps > -1; $0 \le X < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_gas_volume_first_order', FA0=10.0, CA0=40.0, k=0.05,
eps=1.0, X=0.8)
# -> V = 12.0944 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_gas_volume_first_order', FA0=2.0, CA0=20.0, k=0.01,
eps=-0.5, X=0.6)
# -> V = 7.58145 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order gas-phase PFR volume: FA0 = 10.0 mol/s, CA0 = 40.0 mol/m^3, k = 0.05 1/s, eps = 1.0, X = 0.8 | SOLVED | V = 12.0944 m^3 |
| First-order gas-phase PFR volume: FA0 = 2.0 mol/s, CA0 = 20.0 mol/m^3, k = 0.01 1/s, eps = -0.5, X = 0.6 | SOLVED | V = 7.58145 m^3 |
| First-order gas-phase PFR volume: CA0 = 40.0 mol/m^3, k = 0.05 1/s, eps = 1.0, X = 0.8 | NEEDS_INPUT | chemeng_pfr_gas_volume_first_order needs FA0 (molar feed flow of A). Example: solve('chemeng_pfr_gas_volume_first_order' |
| First-order gas-phase PFR volume: FA0 = 10.0 mol/s, CA0 = 40.0 mol/m^3, k = 0.05 1/s, eps = 1.0, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Gas-phase PFR volume with volume change (second order) —

`chemeng_pfr_gas_volume_second_order`

Volume of an isothermal, isobaric gas-phase plug-flow reactor for conversion X when -r_A = k $C_A^2$ and the mole number changes (factor eps).

$$V = \frac{F_{A0}}{kC_{A0}^2}\left[2\varepsilon(1+\varepsilon)\ln(1-X) + \varepsilon^2 X + \frac{(1+\varepsilon)^2 X}{1-X}\right]$$

| input | meaning | condition |
|---|---|---|
| `FA0` | molar feed flow of A | |
| `CA0` | concentration of A at the inlet | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `eps` | volume-change factor eps = y_A0 * (change in moles per mole of A); e.g. A -> 2B with pure A: eps = 1 | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |

- **Method:** V = (F_A0/(k C_A0^2)) [2 eps (1 + eps) ln(1 - X) + eps^2 X + (1 + eps)^2 $X/(1-X)$]
- **Assumes:** isothermal, isobaric ideal gas; eps > -1; $0 \le X < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pfr_gas_volume_second_order', FA0=10.0, CA0=40.0,
k=0.001, eps=1.0, X=0.8)
# -> V = 64.7641 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pfr_gas_volume_second_order', FA0=2.0, CA0=20.0,
k=0.0005, eps=-0.5, X=0.6)
# -> V = 9.83145 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order gas-phase PFR volume: FA0 = 10.0 mol/s, CA0 = 40.0 mol/m^3, k = 0.001 m^3/(mol*s), eps = 1.0, X = 0.8 | SOLVED | V = 64.7641 m^3 |
| Second-order gas-phase PFR volume: FA0 = 2.0 mol/s, CA0 = 20.0 mol/m^3, k = 0.0005 m^3/(mol*s), eps = -0.5, X = 0.6 | SOLVED | V = 9.83145 m^3 |
| Second-order gas-phase PFR volume: CA0 = 40.0 mol/m^3, k = 0.001 m^3/(mol*s), eps = 1.0, X = 0.8 | NEEDS_INPUT | chemeng_pfr_gas_volume_second_order needs FA0 (molar feed flow of A). Example: solve('chemeng_pfr_gas_volume_second_orde |
| Second-order gas-phase PFR volume: FA0 = 10.0 mol/s, CA0 = 40.0 mol/m^3, k = 0.001 m^3/(mol*s), eps = 1.0, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Equal CSTRs in series: conversion (first order) — `chemeng_cstrs_in_series_first_order`

Conversion leaving N equal CSTRs in series, each of volume V, for a first-order liquid-phase reaction.

$$X = 1 - (1 + k\tau_i)^{-N}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | volume of EACH tank | |
| `k` | first-order rate constant | |
| `N` | number of equal tanks in series | |

| output | meaning |
|---|---|
| `X` | overall conversion of A |

- **Method:** each tank divides the concentration by (1 + k tau_i): X = 1 - (1 + k V/v0)^(-N)
- **Assumes:** equal tanks; steady state, perfect mixing in each tank, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstrs_in_series_first_order', v0=0.01, V=5.0, k=0.001,
N=3)
# -> X = 0.703704   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstrs_in_series_first_order', v0=0.002, V=0.05, k=0.05,
N=2)
# -> X = 0.802469   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order CSTRs in series: v0 = 0.01 m^3/s, V = 5.0 m^3, k = 0.001 1/s, N = 3 | SOLVED | X = 0.703704 |
| First-order CSTRs in series: v0 = 0.002 m^3/s, V = 0.05 m^3, k = 0.05 1/s, N = 2 | SOLVED | X = 0.802469 |
| First-order CSTRs in series: V = 5.0 m^3, k = 0.001 1/s, N = 3 | NEEDS_INPUT | chemeng_cstrs_in_series_first_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstrs_in_series_first_order |
| First-order CSTRs in series: v0 = 0.01 m^3/s, V = 5.0 m^3, k = 0.001 1/s, N = 3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| First-order CSTRs in series: v0 = 0.01 m^3/s, V = 5 m^3, k = 0.001 1/s, N = 2.5 | INVALID_INPUT | N: expected a whole number written exactly, got '2.5'. |

## Equal CSTRs in series: conversion (second order) — `chemeng_cstrs_in_series_second_order`

Conversion leaving N equal CSTRs in series, each of volume V, when -r_A = k $C_A^{2.}$

$$C_i = \frac{2C_{i-1}}{1+\sqrt{1+4k\tau_i C_{i-1}}}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `V` | volume of EACH tank | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |
| `N` | number of equal tanks in series | |

| output | meaning |
|---|---|
| `X` | overall conversion of A |
| `CA` | exit concentration of A |

- **Method:** tank by tank: $C_i = 2$ C_($i-1$)/(1 + $sqrt$(1 + 4 k tau_i C_(i-1)))
- **Assumes:** equal tanks; steady state, perfect mixing, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstrs_in_series_second_order', v0=0.01, V=1.0, k=1e-05,
CA0=1000.0, N=3)
# -> X = 0.674359, CA = 325.641 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstrs_in_series_second_order', v0=0.001, V=0.5, k=1e-06, CA0=500.0, N=4)
# -> X = 0.461462, CA = 269.269 mol/m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order CSTRs in series: v0 = 0.01 m^3/s, V = 1.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, N = 3 | SOLVED | X = 0.674359, CA = 325.641 mol/m^3 |
| Second-order CSTRs in series: v0 = 0.001 m^3/s, V = 0.5 m^3, k = 1e-06 m^3/(mol*s), CA0 = 500.0 mol/m^3, N = 4 | SOLVED | X = 0.461462, CA = 269.269 mol/m^3 |
| Second-order CSTRs in series: V = 1.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, N = 3 | NEEDS_INPUT | chemeng_cstrs_in_series_second_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstrs_in_series_second_ord |
| Second-order CSTRs in series: v0 = 0.01 m^3/s, V = 1.0 m^3, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, N = 3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Plug-flow reactor with recycle: conversion (first order) — `chemeng_recycle_reactor_first_order`

Overall conversion of a plug-flow reactor with recycle ratio R (recycled flow / product flow) for a first-order constant-density reaction.

$$\frac{k\tau}{R+1} = \ln\frac{1+R(1-X)}{(R+1)(1-X)}$$

| input | meaning | condition |
|---|---|---|
| `v0` | fresh volumetric feed flow | |
| `V` | reactor volume | |
| `k` | first-order rate constant | |
| `R` | recycle ratio: recycled flow divided by the flow leaving the system | |

| output | meaning |
|---|---|
| `X` | overall conversion of A |

- **Method:** k $\tau/(R+1) = ln$[(1 + R(1 - X))/($(R+1)(1-X)$)], $\tau = V/v_0$, solved for X
- **Assumes:** $R \geq 0$ (R = 0 is a plain PFR; large R approaches a CSTR); isothermal, constant density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_recycle_reactor_first_order', v0=0.01, V=20.0, k=0.001, R=2.0)
# -> X = 0.7398   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_recycle_reactor_first_order', v0=0.002, V=0.1, k=0.05,
R=0.0)
# -> X = 0.917915   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Recycle reactor conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, k = 0.001 1/s, R = 2.0 | SOLVED | X = 0.7398 |
| Recycle reactor conversion: v0 = 0.002 m^3/s, V = 0.1 m^3, k = 0.05 1/s, R = 0.0 | SOLVED | X = 0.917915 |
| Recycle reactor conversion: V = 20.0 m^3, k = 0.001 1/s, R = 2.0 | NEEDS_INPUT | chemeng_recycle_reactor_first_order needs v0 (fresh volumetric feed flow). Example: solve('chemeng_recycle_reactor_first |
| Recycle reactor conversion: v0 = 0.01 m^3/s, V = 20.0 m^3, k = 0.001 1/s, R = 2.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Reversible first-order reaction A <-> B in a batch reactor —

`chemeng_reversible_first_order_batch`

Concentrations and conversion after time t, and the equilibrium conversion, for A <-> B with first-order forward and reverse steps.

$$C_A = C_{Ae} + (C_{A0} - C_{Ae})e^{-(k_1+k_2)t},\ C_{Ae} = \frac{k_2(C_{A0} + C_{B0})}{k_1 + k_2}$$

| input | meaning | condition |
|---|---|---|
| `k1` | forward rate constant (A -> B) | |
| `k2` | reverse rate constant (B -> A) | |
| `CA0` | feed (initial) concentration of A | |
| `CB0` | initial concentration of B | |
| `t` | reaction time | |

| output | meaning |
|---|---|
| `CA` | concentration of A at t |
| `CB` | concentration of B at t |
| `X` | conversion of A at t |
| `X_eq` | equilibrium conversion of A |

- **Method:** $C_A = C_Ae + (C_A0 - C_Ae)$ exp(-$(k_1 + k_2)$ t), $C_Ae = k_2\ (C_A0 + C_B0)/(k_1 + k_2)$
- **Assumes:** isothermal, constant volume
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_reversible_first_order_batch', k1=0.002, k2=0.001,
CA0=1000.0, CB0=0.0, t=300.0)
# -> CA = 604.38 mol/m^3, CB = 395.62 mol/m^3, X = 0.39562, X_eq = 0.666667   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_reversible_first_order_batch', k1=0.01, k2=0.04,
CA0=500.0, CB0=100.0, t=30.0)
# -> CA = 484.463 mol/m^3, CB = 115.537 mol/m^3, X = 0.0310748, X_eq = 0.04   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Reversible first-order batch reaction: k1 = 0.002 1/s, k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3, CB0 = 0.0 mol/m^3, t = 300.0 s | SOLVED | CA = 604.38 mol/m^3, CB = 395.62 mol/m^3, X = 0.39562, X_eq = 0.666667 |
| Reversible first-order batch reaction: k1 = 0.01 1/s, k2 = 0.04 1/s, CA0 = 500.0 mol/m^3, CB0 = 100.0 mol/m^3, t = 30.0 s | SOLVED | CA = 484.463 mol/m^3, CB = 115.537 mol/m^3, X = 0.0310748, X_eq = 0.04 |
| Reversible first-order batch reaction: k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3, CB0 = 0.0 mol/m^3, t = 300.0 s | NEEDS_INPUT | chemeng_reversible_first_order_batch needs k1 (forward rate constant (A -> B)). Example: solve('chemeng_reversible_first |
| Reversible first-order batch reaction: k1 = 0.002 1/s, k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3, CB0 = 0.0 mol/m^3, t = 300.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Series reactions A -> B -> C: maximum of the intermediate (batch or PFR) — `chemeng_series_reaction_maximum`

Time (or plug-flow space time) at which the intermediate B of first-order series reactions A -> B -> C peaks, starting from pure A, and the concentrations there.

$$t_{max} = \frac{\ln(k_2/k_1)}{k_2 - k_1}, \quad C_{B,max} = C_{A0}\left(\frac{k_1}{k_2}\right)^{k_2/(k_2-k_1)}$$

| input | meaning | condition |
|---|---|---|
| `k1` | rate constant of A -> B | |
| `k2` | rate constant of B -> C | |
| `CA0` | feed (initial) concentration of A | |

| output | meaning |
|---|---|
| `t_max` | time (or space time) of the maximum of B |
| `CB_max` | maximum concentration of B |
| `CA` | concentration of A at the maximum |
| `CC` | concentration of C at the maximum |

- **Method:** t_max = ln(k2/k1)/(k2 - k1), C_B,max = C_A0 (k1/k2)^(k2/(k2 - k1)) (k1 = k2: $t = 1/k$, C_B = C_A0/e)
- **Assumes:** starting from pure A (no B or C); isothermal, constant volume, first-order steps
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_series_reaction_maximum', k1=0.002, k2=0.001,
CA0=1000.0)
# -> t_max = 693.147 s, CB_max = 500 mol/m^3, CA = 250 mol/m^3, CC = 250 mol/m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_series_reaction_maximum', k1=0.01, k2=0.05, CA0=200.0)
# -> t_max = 40.2359 s, CB_max = 26.7496 mol/m^3, CA = 133.748 mol/m^3, CC = 39.5023 mol/m^3
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Series reaction maximum: k1 = 0.002 1/s, k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3 | SOLVED | t_max = 693.147 s, CB_max = 500 mol/m^3, CA = 250 mol/m^3, CC = 250 mol/m^3 |
| Series reaction maximum: k1 = 0.01 1/s, k2 = 0.05 1/s, CA0 = 200.0 mol/m^3 | SOLVED | t_max = 40.2359 s, CB_max = 26.7496 mol/m^3, CA = 133.748 mol/m^3, CC = 39.5023 mol/m^3 |
| Series reaction maximum: k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3 | NEEDS_INPUT | chemeng_series_reaction_maximum needs k1 (rate constant of A -> B). Example: solve('chemeng_series_reaction_maximum', k1 |
| Series reaction maximum: k1 = 0.002 1/s, k2 = 0.001 1/s, CA0 = 1000.0 mol/m^3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Residence-time distribution: mean, variance and tanks-in-series number from tracer data — `chemeng_rtd_moments`

Mean residence time, variance and the equivalent number of tanks in series from a pulse-tracer response C(t) (trapezoidal rule, uneven spacing allowed).

$$\bar{t} = \frac{\int t\,C\,dt}{\int C\,dt}, \quad \sigma^2 = \frac{\int (t-\bar{t})^2 C\,dt}{\int C\,dt}$$

| input | meaning | condition |
|---|---|---|
| `t` | sample times, in the unit t_unit | |
| `C` | tracer concentrations at those times (any unit) | |
| `t_unit` | unit of the times: s, min or h | |

| output | meaning |
|---|---|
| `t_mean` | mean residence time |
| `variance` | variance of the RTD |

| output | meaning |
|---|---|
| `N_tanks` | equivalent number of equal tanks in series, t_mean^2/variance |

- **Method:** E(t) = C/integral C dt; t_mean = integral t E dt; variance = integral (t - t_mean)^2 E dt; trapezoidal rule
- **Assumes:** pulse tracer, closed vessel; times increasing, concentrations >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_rtd_moments', t=[0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0,
8.0, 9.0, 10.0, 12.0, 14.0], C=[0.0, 1.0, 5.0, 8.0, 10.0, 8.0, 6.0, 4.0, 3.0, 2.2, 1.5, 0.6, 0.0],
t_unit='min')
# -> t_mean = 307.641 s, variance = 21424.3 s^2, N_tanks = 4.41753   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_rtd_moments', t=[0.0, 5.0, 10.0, 20.0, 30.0, 45.0,
60.0], C=[0.0, 4.0, 6.0, 5.0, 3.0, 1.0, 0.2], t_unit='s')
# -> t_mean = 20.355 s, variance = 144.992 s^2, N_tanks = 2.85758   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| RTD moments: t = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 12.0, 14.0], C = [0.0, 1.0, 5.0, 8.0, 10.0, 8.0, 6.0, 4.0, 3.0, 2.2, 1.5, 0.6, 0.0], t_unit = min | SOLVED | t_mean = 307.641 s, variance = 21424.3 s^2, N_tanks = 4.41753 |
| RTD moments: t = [0.0, 5.0, 10.0, 20.0, 30.0, 45.0, 60.0], C = [0.0, 4.0, 6.0, 5.0, 3.0, 1.0, 0.2], t_unit = s | SOLVED | t_mean = 20.355 s, variance = 144.992 s^2, N_tanks = 2.85758 |
| RTD moments: C = [0.0, 1.0, 5.0, 8.0, 10.0, 8.0, 6.0, 4.0, 3.0, 2.2, 1.5, 0.6, 0.0], t_unit = min | NEEDS_INPUT | chemeng_rtd_moments needs t (sample times, in the unit t_unit). Example: solve('chemeng_rtd_moments', t=[0.0, 1.0, 2.0, |
| RTD moments: t = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 12.0, 14.0], C = [0.0, 1.0, 5.0, 8.0, 10.0, 8.0, 6.0, 4.0, 3.0, 2.2, 1.5, 0.6, 0.0], t_unit = min, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| RTD moments: t = [0, 1, 2], C = [0, 1, 0] | NEEDS_INPUT | chemeng_rtd_moments needs t_unit (unit of the times: s, min or h). Example: solve('chemeng_rtd_moments', t=[0.0, 1.0, 2. |

### Vapour pressure from the Antoine equation — `chemeng_antoine_vapour_pressure`

Vapour pressure of a pure liquid at temperature T from Antoine coefficients written in a stated convention (logarithm and units).

$$\log_{10} P^{sat} = A - \frac{B}{T + C}$$

| input | meaning | condition |
|---|---|---|
| `A` | Antoine coefficient A | |

| input | meaning | condition |
|---|---|---|
| `B` | Antoine coefficient B | |
| `C` | Antoine coefficient C | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `T` | temperature | |

| output | meaning |
|---|---|
| `Psat` | vapour pressure |

- **Method:** log $P = A - B/(T + C)$ in the stated units and logarithm; P returned in Pa
- **Assumes:** the convention (units, base) is stated; T within the range the coefficients were fitted for
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_antoine_vapour_pressure', A=8.07131, B=1730.63,
C=233.426, units='mmHg_degC', base='log10', T=373.15)
# -> Psat = 101337 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_antoine_vapour_pressure', A=4.72583, B=1660.652,
C=-1.461, units='bar_K', base='log10', T=350.0)
# -> Psat = 91457.9 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Antoine vapour pressure: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, T = 373.15 K | SOLVED | Psat = 101337 Pa |
| Antoine vapour pressure: A = 4.72583, B = 1660.652, C = -1.461, units = bar_K, base = log10, T = 350.0 K | SOLVED | Psat = 91457.9 Pa |
| Antoine vapour pressure: B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, T = 373.15 K | NEEDS_INPUT | chemeng_antoine_vapour_pressure needs A (Antoine coefficient A). Example: solve('chemeng_antoine_vapour_pressure', A=8.0 |
| Antoine vapour pressure: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, T = 373.15 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Antoine vapour pressure: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, T = 100 degC | SOLVED | Psat = 101337 Pa |

| question | outcome | result |
|---|---|---|
| Antoine vapour pressure: A = 8.07131, B = 1730.63, C = 233.426, T = 100 degC | NEEDS_INPUT | chemeng_antoine_vapour_pressure needs units, base (units the Antoine coefficients are written for: pressure_temperature, |

### Boiling (saturation) temperature from the Antoine equation —

`chemeng_antoine_boiling_temperature`

Temperature at which a pure liquid boils at pressure P, from Antoine coefficients written in a stated convention.

$$T^{sat} = \frac{B}{A - \log_{10} P} - C$$

| input | meaning | condition |
|---|---|---|
| `A` | Antoine coefficient A | |
| `B` | Antoine coefficient B | |
| `C` | Antoine coefficient C | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `P` | pressure | |

| output | meaning |
|---|---|
| `Tsat` | saturation temperature |

- **Method:** T = B/(A - log P) - C in the stated units; returned in K
- **Assumes:** the convention (units, base) is stated; P within the fitted range
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_antoine_boiling_temperature', A=8.07131, B=1730.63,
C=233.426, units='mmHg_degC', base='log10', P=101325.0)
# -> Tsat = 373.147 K   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_antoine_boiling_temperature', A=4.72583, B=1660.652,
C=-1.461, units='bar_K', base='log10', P=50000.0)
# -> Tsat = 331.817 K   (SOLVED)
```

#### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Antoine boiling temperature: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, P = 101325.0 Pa | SOLVED | Tsat = 373.147 K |
| Antoine boiling temperature: A = 4.72583, B = 1660.652, C = -1.461, | SOLVED | Tsat = 331.817 K |

| question | outcome | result |
|---|---|---|
| units = bar_K, base = log10, P = 50000.0 Pa | | |
| Antoine boiling temperature: B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, P = 101325.0 Pa | NEEDS_INPUT | chemeng_antoine_boiling_temperature needs A (Antoine coefficient A). Example: solve('chemeng_antoine_boiling_temperature |
| Antoine boiling temperature: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, P = 101325.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Antoine boiling temperature: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg_degC, base = log10, P = 1 atm | SOLVED | Tsat = 373.147 K |
| Antoine boiling temperature: A = 8.07131, B = 1730.63, C = 233.426, units = mmHg, base = log10, P = 1 atm | INVALID_INPUT | units: one of mmHg_degC, kPa_degC, bar_degC, mmHg_K, Pa_K, kPa_K, bar_K; got 'mmhg'. |

### Bubble-point pressure (Raoult's law) — `chemeng_raoult_bubble_pressure`

Pressure at which a liquid of composition x starts to boil at temperature T, and the composition of the first bubble, for an ideal solution (Raoult's law, Antoine vapour pressures).

$$P_{bubble} = \sum_i x_i P_i^{sat}(T)$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions | |
| `ant` | Antoine coefficients, one row (A, B, C) per component, in the stated convention | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `T` | temperature | |

| output | meaning |
|---|---|
| `P` | bubble-point pressure |
| `y` | vapour mole fractions of the first bubble |

- **Method:** P = sum x_i Psat_i(T); $y_i = x_i$ Psat_i/P
- **Assumes:** ideal liquid and ideal gas (Raoult's law); fractions sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_raoult_bubble_pressure', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', x=[0.4, 0.6], T=368.15)
# -> P = 100906 Pa, y = [0.621960532969, 0.378039467031]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_raoult_bubble_pressure', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', x=[0.25, 0.75], T=380.0)
# -> P = 122162 Pa, y = [0.441901544104, 0.558098455896]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Bubble-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.4, 0.6], T = 368.15 K | SOLVED | P = 100906 Pa, y = [0.621960532969, 0.378039467031] |
| Bubble-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.25, 0.75], T = 380.0 K | SOLVED | P = 122162 Pa, y = [0.441901544104, 0.558098455896] |
| Bubble-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, T = 368.15 K | NEEDS_INPUT | chemeng_raoult_bubble_pressure needs x (liquid mole fractions). Example: solve('chemeng_raoult_bubble_pressure', ant=[[6 |
| Bubble-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.4, 0.6], T = 368.15 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Dew-point pressure (Raoult's law) — `chemeng_raoult_dew_pressure`

Pressure at which a vapour of composition y starts to condense at temperature T, and the composition of the first drop (Raoult's law, Antoine vapour pressures).

$$\frac{1}{P_{dew}} = \sum_i \frac{y_i}{P_i^{sat}(T)}$$

| input | meaning | condition |
|---|---|---|
| `y` | vapour mole fractions | |
| `ant` | Antoine coefficients, one row (A, B, C) per component, in the stated convention | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `T` | temperature | |

| output | meaning |
|---|---|
| `P` | dew-point pressure |
| `x` | liquid mole fractions of the first drop |

- **Method:** 1/P = sum y_i/Psat_i(T); $x_i = y_i$ P/Psat_i

- **Assumes:** ideal liquid and ideal gas; fractions sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_raoult_dew_pressure', ant=[[6.90565, 1211.033, 220.79],
[6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', y=[0.4, 0.6], T=368.15)
# -> P = 83426 Pa, x = [0.212686334545, 0.787313665455]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_raoult_dew_pressure', ant=[[6.90565, 1211.033, 220.79],
[6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', y=[0.6, 0.4], T=380.0)
# -> P = 139298 Pa, x = [0.387057256563, 0.612942743437]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Dew-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.4, 0.6], T = 368.15 K | SOLVED | P = 83426 Pa, x = [0.212686334545, 0.787313665455] |
| Dew-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.6, 0.4], T = 380.0 K | SOLVED | P = 139298 Pa, x = [0.387057256563, 0.612942743437] |
| Dew-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, T = 368.15 K | NEEDS_INPUT | chemeng_raoult_dew_pressure needs y (vapour mole fractions). Example: solve('chemeng_raoult_dew_pressure', ant=[[6.90565 |
| Dew-point pressure: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.4, 0.6], T = 368.15 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Bubble-point temperature (Raoult's law) — `chemeng_raoult_bubble_temperature`

Temperature at which a liquid of composition x starts to boil at pressure P, and the first-bubble composition (Raoult's law, Antoine vapour pressures; iterative).

$$\sum_i x_i P_i^{sat}(T_{bubble}) = P$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions | |
| `ant` | Antoine coefficients, one row (A, B, C) per component, in the stated convention | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `P` | pressure | |

| output | meaning |
|---|---|
| `T` | bubble-point temperature |
| `y` | vapour mole fractions of the first bubble |

- **Method:** sum x_i Psat_i(T) = P solved by bisection between the pure-component boiling points
- **Assumes:** ideal liquid and ideal gas; fractions sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_raoult_bubble_temperature', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', x=[0.4, 0.6], P=101325.0)
# -> T = 368.292 K, y = [0.62184905372, 0.37815094628]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_raoult_bubble_temperature', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', x=[0.7, 0.3], P=50000.0)
# -> T = 338.024 K, y = [0.865597485424, 0.134402514576]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Bubble-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.4, 0.6], P = 101325.0 Pa | SOLVED | T = 368.292 K, y = [0.62184905372, 0.37815094628] |
| Bubble-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.7, 0.3], P = 50000.0 Pa | SOLVED | T = 338.024 K, y = [0.865597485424, 0.134402514576] |
| Bubble-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, P = 101325.0 Pa | NEEDS_INPUT | chemeng_raoult_bubble_temperature needs x (liquid mole fractions). Example: solve('chemeng_raoult_bubble_temperature', a |
| Bubble-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, x = [0.4, 0.6], P = 101325.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Bubble-point temperature: x = [0.4, 0.6], ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, P = 1 atm | SOLVED | T = 368.292 K, y = [0.62184905372, 0.37815094628] |
| Bubble-point temperature: x = [0.4, 0.5], ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, P = 1 atm | INVALID_INPUT | Bubble-point temperature (Raoult's law): x: the fractions must sum to 1. |

### Dew-point temperature (Raoult's law) — `chemeng_raoult_dew_temperature`

Temperature at which a vapour of composition y starts to condense at pressure P, and the first-drop composition (Raoult's law, Antoine vapour pressures; iterative).

$$\sum_i \frac{y_i P}{P_i^{sat}(T_{dew})} = 1$$

| input | meaning | condition |
|---|---|---|
| `y` | vapour mole fractions | |
| `ant` | Antoine coefficients, one row (A, B, C) per component, in the stated convention | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `P` | pressure | |

| output | meaning |
|---|---|
| `T` | dew-point temperature |
| `x` | liquid mole fractions of the first drop |

- **Method:** sum y_i P/Psat_i(T) = 1 solved by bisection between the pure-component boiling points
- **Assumes:** ideal liquid and ideal gas; fractions sum to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_raoult_dew_temperature', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', y=[0.4, 0.6], P=101325.0)
# -> T = 374.635 K, x = [0.216267034105, 0.783732965895]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_raoult_dew_temperature', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', y=[0.7, 0.3], P=50000.0)
# -> T = 343.96 K, x = [0.464057188008, 0.535942811992]   (SOLVED)
```

#### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Dew-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.4, 0.6], P = 101325.0 Pa | SOLVED | T = 374.635 K, x = [0.216267034105, 0.783732965895] |
| Dew-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.7, 0.3], P = 50000.0 Pa | SOLVED | T = 343.96 K, x = [0.464057188008, 0.535942811992] |

| question | outcome | result |
|---|---|---|
| Dew-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, P = 101325.0 Pa | NEEDS_INPUT | chemeng_raoult_dew_temperature needs y (vapour mole fractions). Example: solve('chemeng_raoult_dew_temperature', ant=[[6 |
| Dew-point temperature: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, y = [0.4, 0.6], P = 101325.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Isothermal flash with given K-values (Rachford-Rice) — `chemeng_rachford_rice_flash`

Vapour fraction and the liquid and vapour compositions of a feed flashed at conditions where the K-values are known.

$$\sum_i \frac{z_i(K_i-1)}{1+\frac{V}{F}(K_i-1)} = 0$$

| input | meaning | condition |
|---|---|---|
| z | feed mole fractions | |
| K | equilibrium ratios $K_i = y_i/x_i$ | |

| output | meaning |
|---|---|
| VF | vapour fraction V/F |
| x | liquid mole fractions |
| y | vapour mole fractions |

- **Method:** Rachford-Rice equation sum z_i (K_i - 1)/(1 + V $(K_i - 1)$) = 0 solved for V by bisection; x_i = z_i/(1 + V $(K_i - 1)$), $y_i = K_i$ x_i
- **Assumes:** two-phase: sum z $K > 1$ and sum $z/K > 1$; K-values independent of composition; some K-value differs from 1 by more than 1 % (well-conditioned)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_rachford_rice_flash', z=[0.5, 0.3, 0.2], K=[1.685,
0.742, 0.532])
# -> VF = 0.69073, x = [0.339408696966, 0.365056059037, 0.295535243996], y = [0.571903654388,
0.270871595806, 0.157224749806]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_rachford_rice_flash', z=[0.2, 0.5, 0.3], K=[4.2, 1.1,
0.15])
# -> VF = 0.335818, x = [0.096403236013, 0.483754625418, 0.419842138569], y = [0.404893591254,
0.53213008796, 0.0629763207853]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Rachford-Rice flash: z = [0.5, 0.3, 0.2], K = [1.685, 0.742, 0.532] | SOLVED | VF = 0.69073, x = [0.339408696966, 0.365056059037, 0.295535243996], y = [0.571903654388, 0.270871595806, 0.157224749806] |
| Rachford-Rice flash: z = [0.2, 0.5, 0.3], K = [4.2, 1.1, 0.15] | SOLVED | VF = 0.335818, x = [0.096403236013, 0.483754625418, 0.419842138569], y = [0.404893591254, 0.53213008796, 0.0629763207853] |
| Rachford-Rice flash: K = [1.685, 0.742, 0.532] | NEEDS_INPUT | chemeng_rachford_rice_flash needs z (feed mole fractions). Example: solve('chemeng_rachford_rice_flash', z=[0.5, 0.3, 0. |
| Rachford-Rice flash: z = [0.5, 0.3, 0.2], K = [1.685, 0.742, 0.532], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Rachford-Rice flash: z = [0.5, 0.5], K = [0.5, 0.8] | INVALID_INPUT | Isothermal flash with given K-values (Rachford-Rice): the mixture is not two-phase at these K-values (sum zK <= 1: all l |

### Isothermal flash of an ideal mixture (Raoult's law) — `chemeng_raoult_isothermal_flash`

Vapour fraction and phase compositions of a feed flashed at temperature T and pressure P, with Raoult's-law K-values from Antoine vapour pressures.

$$K_i = \frac{P_i^{sat}(T)}{P}, \quad \sum_i \frac{z_i(K_i - 1)}{1 + \frac{V}{F}(K_i - 1)} = 0$$

| input | meaning | condition |
|---|---|---|
| `z` | feed mole fractions | |
| `ant` | Antoine coefficients, one row (A, B, C) per component, in the stated convention | |
| `units` | units the Antoine coefficients are written for: pressure_temperature, e.g. mmHg_degC (P in mmHg, T in degC) or kPa_K | |
| `base` | logarithm of the Antoine equation: log10 or ln | |
| `T` | flash temperature | |
| `P` | flash pressure | |

| output | meaning |
|---|---|
| `VF` | vapour fraction V/F |
| `x` | liquid mole fractions |
| `y` | vapour mole fractions |

- **Method:** K_i = Psat_i(T)/P, then the Rachford-Rice equation solved for V/F by bisection
- **Assumes:** P between the dew- and bubble-point pressures at T (two phases); ideal liquid and gas; some K-value differs from 1 by more than 1 %
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_raoult_isothermal_flash', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', z=[0.5, 0.5], T=368.15,
P=101325.0)
# -> VF = 0.430535, x = [0.404485322459, 0.595514677541], y = [0.626336606294, 0.373663393706]
(SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_raoult_isothermal_flash', ant=[[6.90565, 1211.033,
220.79], [6.95464, 1344.8, 219.482]], units='mmHg_degC', base='log10', z=[0.3, 0.7], T=373.0,
P=101325.0)
# -> VF = 0.196311, x = [0.260617931568, 0.739382068432], y = [0.461228873413, 0.538771126587]
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Raoult isothermal flash: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, z = [0.5, 0.5], T = 368.15 K, P = 101325.0 Pa | SOLVED | VF = 0.430535, x = [0.404485322459, 0.595514677541], y = [0.626336606294, 0.373663393706] |
| Raoult isothermal flash: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, z = [0.3, 0.7], T = 373.0 K, P = 101325.0 Pa | SOLVED | VF = 0.196311, x = [0.260617931568, 0.739382068432], y = [0.461228873413, 0.538771126587] |
| Raoult isothermal flash: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, T = 368.15 K, P = 101325.0 Pa | NEEDS_INPUT | chemeng_raoult_isothermal_flash needs z (feed mole fractions). Example: solve('chemeng_raoult_isothermal_flash', ant=[[6 |
| Raoult isothermal flash: ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, z = [0.5, 0.5], T = 368.15 K, P = 101325.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Raoult isothermal flash: z = [0.5, 0.5], ant = [[6.90565, 1211.033, 220.79], [6.95464, 1344.8, 219.482]], units = mmHg_degC, base = log10, T = 368.15 K, P = 5 bar | INVALID_INPUT | Isothermal flash of an ideal mixture (Raoult's law): the mixture is not two-phase at these K-values (sum zK <= 1: all li |

## Flash with Wilson K-values (hydrocarbon mixtures) — `chemeng_wilson_k_flash`

Vapour fraction and phase compositions of a hydrocarbon feed at T and P with K-values estimated by Wilson's correlation from critical constants and acentric factors.

$$K_i = \frac{P_{c,i}}{P}\exp\left[5.37(1+\omega_i)\left(1-\frac{T_{c,i}}{T}\right)\right]$$

| input | meaning | condition |
|---|---|---|
| `z` | feed mole fractions | |
| `Tc` | critical temperatures, K | |

| input | meaning | condition |
|---|---|---|
| `Pc` | critical pressures, in the unit p_unit | |
| `omega` | acentric factors | |
| `p_unit` | unit of the critical pressures: Pa, kPa, MPa, bar, atm or psi | |
| `T` | flash temperature | |
| `P` | flash pressure | |

| output | meaning |
|---|---|
| `VF` | vapour fraction V/F |
| `x` | liquid mole fractions |
| `y` | vapour mole fractions |
| `K` | Wilson K-values |

- **Method:** $K_i = (Pc_i/P)$ exp[5.37 $(1 + omega_i)(1 - Tc_i/T)$], then the Rachford-Rice equation by bisection
- **Assumes:** two-phase at the Wilson K-values; some K-value differs from 1 by more than 1 %; Wilson's correlation suits light hydrocarbons at low to moderate pressure
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_wilson_k_flash', z=[0.3, 0.4, 0.3], Tc=[190.564, 369.83,
507.6], Pc=[45.99, 42.48, 30.25], omega=[0.011, 0.152, 0.3], p_unit='bar', T=300.0, P=2000000.0)
# -> VF = 0.362332, x = [0.0449443371671, 0.487795919806, 0.467259743027], y = [0.748872564861,
0.245487924989, 0.00563951014995], K = [16.6622229198, 0.503259488285, 0.0120693259672]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_wilson_k_flash', z=[0.5, 0.5], Tc=[425.12, 507.6], Pc=
[3.796, 3.025], omega=[0.2, 0.3], p_unit='MPa', T=380.0, P=1000000.0)
# -> VF = 0.0518041, x = [0.480912489953, 0.519087510047], y = [0.849368379403, 0.150631620597], K
= [1.7661599504, 0.290185407435]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Wilson K-value flash: z = [0.3, 0.4, 0.3], Tc = [190.564, 369.83, 507.6], Pc = [45.99, 42.48, 30.25], omega = [0.011, 0.152, 0.3], p_unit = bar, T = 300.0 K, P = 2000000.0 Pa | SOLVED | VF = 0.362332, x = [0.0449443371671, 0.487795919806, 0.467259743027], y = [0.748872564861, 0.245487924989, 0.00563951014995], K = [16.6622229198, 0.503259488285, 0.0120693259672] |
| Wilson K-value flash: z = [0.5, 0.5], Tc = [425.12, 507.6], Pc = [3.796, 3.025], omega = [0.2, 0.3], p_unit = MPa, T = 380.0 K, P = 1000000.0 Pa | SOLVED | VF = 0.0518041, x = [0.480912489953, 0.519087510047], y = [0.849368379403, 0.150631620597], K = [1.7661599504, 0.290185407435] |
| Wilson K-value flash: Tc = [190.564, 369.83, 507.6], Pc = [45.99, 42.48, 30.25], omega = [0.011, 0.152, 0.3], p_unit = bar, T = 300.0 K, P = 2000000.0 Pa | NEEDS_INPUT | chemeng_wilson_k_flash needs z (feed mole fractions). Example: solve('chemeng_wilson_k_flash', z=[0.3, 0.4, 0.3], Tc=[19 |
| Wilson K-value flash: z = [0.3, 0.4, 0.3], Tc = [190.564, 369.83, 507.6], | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: |

| question | outcome | result |
|---|---|---|
| Pc = [45.99, 42.48, 30.25], omega = [0.011, 0.152, 0.3], p_unit = bar, T = 300.0 K, P = 2000000.0 Pa, zz = 1 | | chemical_engineering_ai999x.templ |

### Minimum reflux ratio of a binary column (Underwood) — `chemeng_binary_minimum_reflux`

Minimum reflux ratio of a binary distillation with constant relative volatility, for any feed condition q (Underwood's equations).

$$\sum_i \frac{\alpha_i z_{F,i}}{\alpha_i - \theta} = 1 - q, \quad R_{min} + 1 = \sum_i \frac{\alpha_i x_{D,i}}{\alpha_i - \theta}$$

| input | meaning | condition |
|---|---|---|
| `alpha` | relative volatility of the light component (constant) | |
| `zF` | mole fraction of the light component in the feed | |
| `xD` | mole fraction of the light component in the distillate | |
| `q` | feed quality q: 1 saturated liquid, 0 saturated vapour, > 1 subcooled, < 0 superheated | |

| output | meaning |
|---|---|
| `Rmin` | minimum reflux ratio L/D |

- **Method:** Underwood: sum alpha_i $z_i/(alpha_i - \theta) = 1 - q$ for theta between 1 and alpha (bisection); R_min + 1 = sum alpha_i x_Di/(alpha_i - theta)
- **Assumes:** binary, constant alpha, constant molar overflow; $0 < zF < xD < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_binary_minimum_reflux', alpha=2.5, zF=0.5, xD=0.95,
q=1.0)
# -> Rmin = 1.1   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_binary_minimum_reflux', alpha=2.4, zF=0.4, xD=0.97,
q=0.5)
# -> Rmin = 2.26137   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Minimum reflux ratio: alpha = 2.5, zF = 0.5, xD = 0.95, q = 1.0 | SOLVED | Rmin = 1.1 |
| Minimum reflux ratio: alpha = 2.4, zF = 0.4, xD = 0.97, q = 0.5 | SOLVED | Rmin = 2.26137 |
| Minimum reflux ratio: zF = 0.5, xD = 0.95, q = 1.0 | NEEDS_INPUT | chemeng_binary_minimum_reflux needs alpha (relative volatility of the light component (constant)). Example: solve('cheme |

| question | outcome | result |
|---|---|---|
| Minimum reflux ratio: alpha = 2.5, zF = 0.5, xD = 0.95, q = 1.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Minimum reflux ratio: alpha = 0.8, zF = 0.5, xD = 0.95, q = 1 | INVALID_INPUT | Minimum reflux ratio of a binary column (Underwood): the relative volatility alpha must exceed 1. |

### McCabe-Thiele number of theoretical stages (binary) — `chemeng_mccabe_thiele_stages`

Number of theoretical stages (the partial reboiler counted as one) of a binary column at reflux ratio R, by stepping between the operating lines and a constant-alpha equilibrium curve, with the feed switched at the intersection of the operating lines.

$$y_{n+1} = \frac{R}{R+1}x_n + \frac{x_D}{R+1}, \quad x_n = \frac{y_n}{\alpha - (\alpha-1)y_n}$$

| input | meaning | condition |
|---|---|---|
| `alpha` | relative volatility of the light component (constant) | |
| `zF` | mole fraction of the light component in the feed | |
| `xD` | mole fraction of the light component in the distillate | |
| `q` | feed quality q: 1 saturated liquid, 0 saturated vapour, > 1 subcooled, < 0 superheated | |
| `xB` | mole fraction of the light component in the bottoms | |
| `R` | reflux ratio L/D | |

| output | meaning |
|---|---|
| `N` | theoretical stages including the partial reboiler |
| `Rmin` | minimum reflux ratio |

- **Method:** stepping from the top: equilibrium x from y, then y from the rectifying line above the intersection and the stripping line below it, until $x \le xB$
- **Assumes:** binary, constant alpha, constant molar overflow, total condenser; R > R_min; $0 < xB < zF < xD < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_mccabe_thiele_stages', alpha=2.5, zF=0.5, xD=0.95,
q=1.0, xB=0.05, R=2.0)
# -> N = 11, Rmin = 1.1   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_mccabe_thiele_stages', alpha=2.4, zF=0.4, xD=0.97,
q=0.5, xB=0.02, R=3.5)
# -> N = 14, Rmin = 2.26137   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| McCabe-Thiele stages: alpha = 2.5, zF = 0.5, xD = 0.95, q = 1.0, xB = 0.05, R = 2.0 | SOLVED | N = 11, Rmin = 1.1 |
| McCabe-Thiele stages: alpha = 2.4, zF = 0.4, xD = 0.97, q = 0.5, xB = 0.02, R = 3.5 | SOLVED | N = 14, Rmin = 2.26137 |
| McCabe-Thiele stages: zF = 0.5, xD = 0.95, q = 1.0, xB = 0.05, R = 2.0 | NEEDS_INPUT | chemeng_mccabe_thiele_stages needs alpha (relative volatility of the light component (constant)). Example: solve('chemen |
| McCabe-Thiele stages: alpha = 2.5, zF = 0.5, xD = 0.95, q = 1.0, xB = 0.05, R = 2.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| McCabe-Thiele stages: alpha = 2.5, zF = 0.5, xD = 0.95, q = 1, xB = 0.05, R = 1 | INVALID_INPUT | McCabe-Thiele number of theoretical stages (binary): the reflux ratio must exceed the minimum reflux ratio 1.1. |

### Absorber with N equilibrium stages (Kremser) — `chemeng_kremser_absorber`

Solute mole fraction in the gas leaving a countercurrent absorber of N ideal stages, and the fraction of solute absorbed, for dilute gas with linear equilibrium $y = m$ x.

$$\frac{y_{in} - y_{out}}{y_{in} - m x_{in}} = \frac{A^{N+1} - A}{A^{N+1} - 1}$$

| input | meaning | condition |
|---|---|---|
| `N` | number of ideal stages | |
| `A` | absorption factor A = L/(m V) | |
| `m` | slope of the equilibrium line $y = m$ x | |
| `y_in` | solute mole fraction in the entering gas | |
| `x_in` | solute mole fraction in the entering liquid (solvent) | |

| output | meaning |
|---|---|
| `y_out` | solute mole fraction in the leaving gas |
| `recovery` | fraction of the entering solute absorbed |

- **Method:** Kremser: (y_in - y_out)/(y_in - m x_in) = (A^(N+1) - A)/(A^(N+1) - 1); $N/(N+1)$ when $A = 1$
- **Assumes:** dilute, constant L and V, linear equilibrium; $y_i n > m$ x_in
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_kremser_absorber', N=5, A=1.4, m=2.0, y_in=0.05,
x_in=0.0)
# -> y_out = 0.003063, recovery = 0.93874   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_kremser_absorber', N=3, A=1.0, m=1.5, y_in=0.02,
x_in=0.001)
# -> y_out = 0.006125, recovery = 0.69375   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Kremser absorber: N = 5, A = 1.4, m = 2.0, y_in = 0.05, x_in = 0.0 | SOLVED | y_out = 0.003063, recovery = 0.93874 |
| Kremser absorber: N = 3, A = 1.0, m = 1.5, y_in = 0.02, x_in = 0.001 | SOLVED | y_out = 0.006125, recovery = 0.69375 |
| Kremser absorber: A = 1.4, m = 2.0, y_in = 0.05, x_in = 0.0 | NEEDS_INPUT | chemeng_kremser_absorber needs N (number of ideal stages). Example: solve('chemeng_kremser_absorber', N=5, A=1.4, m=2, y |
| Kremser absorber: N = 5, A = 1.4, m = 2.0, y_in = 0.05, x_in = 0.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Stripper with N equilibrium stages (Kremser) — `chemeng_kremser_stripper`

Solute mole fraction in the liquid leaving a countercurrent stripper of N ideal stages, and the fraction stripped, for dilute liquid with linear equilibrium $y = m$ x.

$$\frac{x_{in} - x_{out}}{x_{in} - y_{in}/m} = \frac{S^{N+1} - S}{S^{N+1} - 1}$$

| input | meaning | condition |
|---|---|---|
| `N` | number of ideal stages | |
| `S` | stripping factor $S = m$ V/L | |
| `m` | slope of the equilibrium line $y = m$ x | |
| `x_in` | solute mole fraction in the entering liquid | |
| `y_in` | solute mole fraction in the entering stripping gas | |

| output | meaning |
|---|---|
| `x_out` | solute mole fraction in the leaving liquid |
| `recovery` | fraction of the entering solute stripped |

- **Method:** Kremser: (x_in - x_out)/(x_in - y_in/m) = (S^(N+1) - S)/(S^(N+1) - 1); $N/(N+1)$ when $S = 1$
- **Assumes:** dilute, constant L and V, linear equilibrium; $x_i n > y_i n/m$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_kremser_stripper', N=6, S=1.5, m=3.0, x_in=0.01,
y_in=0.0)
# -> x_out = 0.000310831, recovery = 0.968917   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_kremser_stripper', N=4, S=0.8, m=2.0, x_in=0.05,
y_in=0.01)
# -> x_out = 0.0183865, recovery = 0.63227   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Kremser stripper: N = 6, S = 1.5, m = 3.0, x_in = 0.01, y_in = 0.0 | SOLVED | x_out = 0.000310831, recovery = 0.968917 |
| Kremser stripper: N = 4, S = 0.8, m = 2.0, x_in = 0.05, y_in = 0.01 | SOLVED | x_out = 0.0183865, recovery = 0.63227 |
| Kremser stripper: S = 1.5, m = 3.0, x_in = 0.01, y_in = 0.0 | NEEDS_INPUT | chemeng_kremser_stripper needs N (number of ideal stages). Example: solve('chemeng_kremser_stripper', N=6, S=1.5, m=3, x |
| Kremser stripper: N = 6, S = 1.5, m = 3.0, x_in = 0.01, y_in = 0.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Packed absorber: number of transfer units and height (Colburn) — `chemeng_absorber_transfer_units`

Number of overall gas-phase transfer units and packed height of a dilute countercurrent absorber with linear equilibrium (Colburn's equation).

$$N_{OG} = \frac{\ln\left[\left(1-\frac{1}{A}\right)\frac{y_1 - m x_2}{y_2 - m x_2} + \frac{1}{A}\right]}{1 - 1/A}$$

| input | meaning | condition |
|---|---|---|
| `y1` | solute mole fraction in the gas entering (bottom) | |
| `y2` | solute mole fraction in the gas leaving (top) | |
| `x2` | solute mole fraction in the liquid entering (top) | |
| `m` | slope of the equilibrium line $y = m$ x | |
| `A` | absorption factor A = L/(m V) | |
| `HOG` | height of an overall gas-phase transfer unit | |

| output | meaning |
|---|---|
| `NOG` | number of overall gas-phase transfer units |
| `Z` | packed height |

- **Method:** $N_OG = ln[(1 - 1/A)$(y1 - m $x_2)/(y_2 - m$ x2) + 1/A]/$(1 - 1/A)$; $Z = H_OG$ N_OG
- **Assumes:** dilute, constant L and V, linear equilibrium; $y_1 > y_2 > m$ x2; no pinch
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_absorber_transfer_units', y1=0.02, y2=0.001, x2=0.0,
m=1.2, A=1.5, HOG=0.6)
# -> NOG = 5.97729, Z = 3.58637 m   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_absorber_transfer_units', y1=0.05, y2=0.005, x2=0.0005,
m=0.8, A=2.0, HOG=1.0)
# -> NOG = 3.54695, Z = 3.54695 m   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Absorber transfer units: y1 = 0.02, y2 = 0.001, x2 = 0.0, m = 1.2, A = 1.5, HOG = 0.6 m | SOLVED | NOG = 5.97729, Z = 3.58637 m |
| Absorber transfer units: y1 = 0.05, y2 = 0.005, x2 = 0.0005, m = 0.8, A = 2.0, HOG = 1.0 m | SOLVED | NOG = 3.54695, Z = 3.54695 m |
| Absorber transfer units: y2 = 0.001, x2 = 0.0, m = 1.2, A = 1.5, HOG = 0.6 m | NEEDS_INPUT | chemeng_absorber_transfer_units needs y1 (solute mole fraction in the gas entering (bottom)). Example: solve('chemeng_ab |
| Absorber transfer units: y1 = 0.02, y2 = 0.001, x2 = 0.0, m = 1.2, A = 1.5, HOG = 0.6 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Absorber transfer units: y1 = 0.02, y2 = 0.001, x2 = 0, m = 1.2, A = 1.5, HOG = 60 cm | SOLVED | NOG = 5.97729, Z = 3.58637 m |

## Countercurrent liquid-liquid extraction (N stages, immiscible solvents) — `chemeng_countercurrent_extraction`

Raffinate and extract solute ratios and the fraction extracted for N ideal countercurrent stages with immiscible carrier and solvent and a constant distribution ratio $Y = KX$ (solute-free mass-ratio basis).

$$\frac{X_F - X_N}{X_F - Y_S/K} = \frac{E^{N+1} - E}{E^{N+1} - 1},\ E = \frac{KS}{F}$$

| input | meaning | condition |
|---|---|---|
| `F` | solute-free carrier flow in the feed | |
| `S` | solute-free solvent flow | |
| `K` | distribution ratio $Y = KX$ (mass ratios) | |
| `N` | number of ideal stages | |
| `XF` | solute ratio in the feed, kg solute per kg carrier | |
| `YS` | solute ratio in the entering solvent, kg solute per kg solvent | |

| output | meaning |
|---|---|
| `XN` | solute ratio in the final raffinate |
| `Y1` | solute ratio in the final extract |

| output | meaning |
|---|---|
| `recovery` | fraction of the solute extracted |

- **Method:** Kremser form with the extraction factor $E = K$ S/F: $(XF - XN)/(XF - YS/K) = (E^{N+1} - E)/(E^{N+1} - 1)$
- **Assumes:** immiscible carrier and solvent, constant K; $XF > YS/K$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_countercurrent_extraction', F=100.0, S=80.0, K=1.5, N=4,
XF=0.1, YS=0.0)
# -> XN = 0.013438, Y1 = 0.108203, recovery = 0.86562   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_countercurrent_extraction', F=10.0, S=5.0, K=2.0, N=3,
XF=0.2, YS=0.01)
# -> XN = 0.05375, Y1 = 0.3025, recovery = 0.73125   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Countercurrent extraction: F = 100.0 kg/s, S = 80.0 kg/s, K = 1.5, N = 4, XF = 0.1, YS = 0.0 | SOLVED | XN = 0.013438, Y1 = 0.108203, recovery = 0.86562 |
| Countercurrent extraction: F = 10.0 kg/s, S = 5.0 kg/s, K = 2.0, N = 3, XF = 0.2, YS = 0.01 | SOLVED | XN = 0.05375, Y1 = 0.3025, recovery = 0.73125 |
| Countercurrent extraction: S = 80.0 kg/s, K = 1.5, N = 4, XF = 0.1, YS = 0.0 | NEEDS_INPUT | chemeng_countercurrent_extraction needs F (solute-free carrier flow in the feed). Example: solve('chemeng_countercurrent |
| Countercurrent extraction: F = 100.0 kg/s, S = 80.0 kg/s, K = 1.5, N = 4, XF = 0.1, YS = 0.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Crosscurrent (multiple-contact) extraction with fresh solvent — `chemeng_crosscurrent_extraction`

Raffinate solute ratio after N ideal stages, each fed with the same amount of fresh solvent, for immiscible solvents and a constant distribution ratio.

$$X_N = \frac{X_F}{(1 + KS/F)^N}$$

| input | meaning | condition |
|---|---|---|
| `F` | solute-free carrier flow (or mass) in the feed | |
| `S` | solute-free solvent flow (or mass) added to EACH stage | |
| `K` | distribution ratio $Y = KX$ (mass ratios) | |
| `N` | number of stages | |
| `XF` | solute ratio in the feed, kg solute per kg carrier | |

| output | meaning |
|---|---|
| `XN` | solute ratio in the final raffinate |
| `recovery` | fraction of the solute extracted |

- **Method:** each stage divides the raffinate ratio by (1 + K S/F): X_N = X_F/(1 + K S/F)^N
- **Assumes:** immiscible carrier and solvent, constant K; fresh (solute-free) solvent to every stage
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_crosscurrent_extraction', F=100.0, S=20.0, K=1.5, N=4,
XF=0.1)
# -> XN = 0.0350128, recovery = 0.649872   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_crosscurrent_extraction', F=10.0, S=10.0, K=2.0, N=1,
XF=0.2)
# -> XN = 0.0666667, recovery = 0.666667   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Crosscurrent extraction: F = 100.0 kg/s, S = 20.0 kg/s, K = 1.5, N = 4, XF = 0.1 | SOLVED | XN = 0.0350128, recovery = 0.649872 |
| Crosscurrent extraction: F = 10.0 kg/s, S = 10.0 kg/s, K = 2.0, N = 1, XF = 0.2 | SOLVED | XN = 0.0666667, recovery = 0.666667 |
| Crosscurrent extraction: S = 20.0 kg/s, K = 1.5, N = 4, XF = 0.1 | NEEDS_INPUT | chemeng_crosscurrent_extraction needs F (solute-free carrier flow (or mass) in the feed). Example: solve('chemeng_crossc |
| Crosscurrent extraction: F = 100.0 kg/s, S = 20.0 kg/s, K = 1.5, N = 4, XF = 0.1, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Activity coefficients from the Wilson equation — `chemeng_wilson_activity_coefficients`

Activity coefficients of every component and the excess Gibbs energy of a liquid mixture from Wilson parameters Lambda_ij.

$$\ln \gamma_i = 1 - \ln\left(\sum_j x_j \Lambda_{ij}\right) - \sum_k \frac{x_k \Lambda_{ki}}{\sum_j x_j \Lambda_{kj}}$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions | |
| `Lam` | Wilson parameters Lambda_ij (dimensionless, Lambda_ii = 1) at the temperature of interest | |

| output | meaning |
|---|---|
| `gamma1` | activity coefficient of component 1 |
| `gamma` | activity coefficients of every component |

| output | meaning |
|---|---|
| `GE_RT` | excess Gibbs energy over RT |

- **Method:** ln $gamma_i = 1 - ln$(sum_j x_j Lambda_ij) - sum_k x_k Lambda_ki/(sum_j x_j Lambda_kj)
- **Assumes:** $Lambda_i i = 1$, $Lambda_i j > 0$; all mole fractions positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_wilson_activity_coefficients', x=[0.3, 0.7], Lam=[[1.0, 0.5], [1.5, 1.0]])
# -> gamma1 = 1.05781, gamma = [1.05781025716, 1.02099043889], GE_RT = 0.0314015   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_wilson_activity_coefficients', x=[0.2, 0.3, 0.5], Lam=[[1.0, 0.8, 0.4], [1.2, 1.0, 0.9], [0.6, 1.1, 1.0]])
# -> gamma1 = 1.57517, gamma = [1.57516694787, 0.885212689617, 1.13570117122], GE_RT = 0.117919   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Wilson activity coefficients: x = [0.3, 0.7], Lam = [[1.0, 0.5], [1.5, 1.0]] | SOLVED | gamma1 = 1.05781, gamma = [1.05781025716, 1.02099043889], GE_RT = 0.0314015 |
| Wilson activity coefficients: x = [0.2, 0.3, 0.5], Lam = [[1.0, 0.8, 0.4], [1.2, 1.0, 0.9], [0.6, 1.1, 1.0]] | SOLVED | gamma1 = 1.57517, gamma = [1.57516694787, 0.885212689617, 1.13570117122], GE_RT = 0.117919 |
| Wilson activity coefficients: Lam = [[1.0, 0.5], [1.5, 1.0]] | NEEDS_INPUT | chemeng_wilson_activity_coefficients needs x (liquid mole fractions). Example: solve('chemeng_wilson_activity_coefficien |
| Wilson activity coefficients: x = [0.3, 0.7], Lam = [[1.0, 0.5], [1.5, 1.0]], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Activity coefficients from the NRTL equation — `chemeng_nrtl_activity_coefficients`

Activity coefficients and excess Gibbs energy of a liquid mixture from NRTL interaction parameters tau_ij and non-randomness parameters alpha_ij.

$$\ln \gamma_i = \frac{\sum_j x_j \tau_{ji} G_{ji}}{\sum_k x_k G_{ki}} + \sum_j \frac{x_j G_{ij}}{\sum_k x_k G_{kj}} \left( \tau_{ij} - \frac{\sum_m x_m \tau_{mj} G_{mj}}{\sum_k x_k G_{kj}} \right)$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions | |
| `tau` | NRTL parameters tau_ij (dimensionless, tau_ii = 0) at the temperature of interest | |
| `alpha` | non-randomness parameters alpha_ij (symmetric, alpha_ii = 0) | |

| output | meaning |
|---|---|
| `gamma1` | activity coefficient of component 1 |
| `gamma` | activity coefficients of every component |

| output | meaning |
|---|---|
| `GE_RT` | excess Gibbs energy over RT |

- **Method:** $G_ij = exp$(-alpha_ij tau_ij); ln gamma_i = sum_j x_j tau_ji G_ji/sum_k x_k G_ki + sum_j (x_j G_ij/sum_k x_k G_kj)(tau_ij - sum_m x_m tau_mj G_mj/sum_k x_k G_kj)
- **Assumes:** $tau_i i = 0$, $alpha_i i = 0$; all mole fractions positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_nrtl_activity_coefficients', x=[0.3, 0.7], tau=[[0.0, 0.8], [1.2, 0.0]], alpha=[[0.0, 0.3], [0.3, 0.0]])
# -> gamma1 = 2.25346, gamma = [2.25346353086, 1.18777953647], GE_RT = 0.3642   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_nrtl_activity_coefficients', x=[0.25, 0.25, 0.5], tau=[[0.0, 1.5, 0.4], [0.2, 0.0, 2.0], [0.9, -0.3, 0.0]], alpha=[[0.0, 0.3, 0.2], [0.3, 0.0, 0.47], [0.2, 0.47, 0.0]])
# -> gamma1 = 1.822, gamma = [1.82200497499, 1.44773823655, 1.13894017298], GE_RT = 0.307534   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| NRTL activity coefficients: x = [0.3, 0.7], tau = [[0.0, 0.8], [1.2, 0.0]], alpha = [[0.0, 0.3], [0.3, 0.0]] | SOLVED | gamma1 = 2.25346, gamma = [2.25346353086, 1.18777953647], GE_RT = 0.3642 |
| NRTL activity coefficients: x = [0.25, 0.25, 0.5], tau = [[0.0, 1.5, 0.4], [0.2, 0.0, 2.0], [0.9, -0.3, 0.0]], alpha = [[0.0, 0.3, 0.2], [0.3, 0.0, 0.47], [0.2, 0.47, 0.0]] | SOLVED | gamma1 = 1.822, gamma = [1.82200497499, 1.44773823655, 1.13894017298], GE_RT = 0.307534 |
| NRTL activity coefficients: tau = [[0.0, 0.8], [1.2, 0.0]], alpha = [[0.0, 0.3], [0.3, 0.0]] | NEEDS_INPUT | chemeng_nrtl_activity_coefficients needs x (liquid mole fractions). Example: solve('chemeng_nrtl_activity_coefficients', |
| NRTL activity coefficients: x = [0.3, 0.7], tau = [[0.0, 0.8], [1.2, 0.0]], alpha = [[0.0, 0.3], [0.3, 0.0]], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Activity coefficients from the UNIQUAC equation — `chemeng_uniquac_activity_coefficients`

Activity coefficients of a liquid mixture from UNIQUAC volume and area parameters r, q and interaction parameters tau_ij (coordination number 10).

$$\ln\gamma_i = \ln\frac{\Phi_i}{x_i} + \frac{z}{2}q_i\ln\frac{\theta_i}{\Phi_i} + l_i - \frac{\Phi_i}{x_i}\sum_j x_j l_j + q_i\Big[1 - \ln\sum_j \theta_j\tau_{ji} - \sum_j \frac{\theta_j\tau_{ij}}{\sum_k \theta_k\tau_{kj}}\Big]$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions | |
| `r` | UNIQUAC volume parameters r_i | |
| `q` | UNIQUAC area parameters q_i | |

| input | meaning | condition |
|---|---|---|
| `tau` | UNIQUAC parameters $tau_i j = exp$(-a_ij/T) at the temperature of interest $(tau_i i = 1)$ | |

| output | meaning |
|---|---|
| `gamma1` | activity coefficient of component 1 |
| `gamma` | activity coefficients of every component |

- **Method:** combinatorial part ln(Phi_i/x_i) + 5 q_i ln(theta_i/Phi_i) + l_i - (Phi_i/x_i) sum x_j l_j plus residual part q_i [$1 - ln$(sum_j theta_j tau_ji) - sum_j theta_j tau_ij/sum_k theta_k tau_kj]
- **Assumes:** $tau_i i = 1$, $tau_i j > 0$; all mole fractions positive; coordination number $z = 10$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_uniquac_activity_coefficients', x=[0.3, 0.7], r=[2.1055, 0.92], q=[1.972, 1.4], tau=[[1.0, 0.9], [1.1, 1.0]])
# -> gamma1 = 1.29869, gamma = [1.29868568479, 1.10358464339]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_uniquac_activity_coefficients', x=[0.2, 0.3, 0.5], r=[2.5735, 3.1878, 0.92], q=[2.336, 2.4, 1.4], tau=[[1.0, 0.7, 1.3], [1.2, 1.0, 0.6], [0.5, 1.6, 1.0]])
# -> gamma1 = 1.1955, gamma = [1.19549838541, 1.20648140089, 1.47615584685]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| UNIQUAC activity coefficients: x = [0.3, 0.7], r = [2.1055, 0.92], q = [1.972, 1.4], tau = [[1.0, 0.9], [1.1, 1.0]] | SOLVED | gamma1 = 1.29869, gamma = [1.29868568479, 1.10358464339] |
| UNIQUAC activity coefficients: x = [0.2, 0.3, 0.5], r = [2.5735, 3.1878, 0.92], q = [2.336, 2.4, 1.4], tau = [[1.0, 0.7, 1.3], [1.2, 1.0, 0.6], [0.5, 1.6, 1.0]] | SOLVED | gamma1 = 1.1955, gamma = [1.19549838541, 1.20648140089, 1.47615584685] |
| UNIQUAC activity coefficients: r = [2.1055, 0.92], q = [1.972, 1.4], tau = [[1.0, 0.9], [1.1, 1.0]] | NEEDS_INPUT | chemeng_uniquac_activity_coefficients needs x (liquid mole fractions). Example: solve('chemeng_uniquac_activity_coeffici |
| UNIQUAC activity coefficients: x = [0.3, 0.7], r = [2.1055, 0.92], q = [1.972, 1.4], tau = [[1.0, 0.9], [1.1, 1.0]], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Peng-Robinson equation of state: compressibility and fugacity coefficient — `chemeng_peng_robinson`

Compressibility factors (vapour and liquid roots) and fugacity coefficients of a pure substance at T and P from the Peng-Robinson equation of state.

$$Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)Z - (AB - B^2 - B^3) = 0$$

| input | meaning | condition |
|---|---|---|
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |
| `omega` | acentric factor | |
| `T` | temperature | |
| `P` | pressure | |

| output | meaning |
|---|---|
| `Z_vapour` | compressibility factor of the vapour-like (largest) root |
| `Z_liquid` | compressibility factor of the liquid-like (smallest) root |
| `phi_vapour` | fugacity coefficient on the vapour root |
| `phi_liquid` | fugacity coefficient on the liquid root |
| `n_roots` | number of real roots above the covolume (1 or 3) |

- **Method:** Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)$Z - (AB - B^2 - B^3) = 0$ with $a = 0.45724$ Double exponent: use braces to clarify alpha(T), $b = 0.07780$ R Tc/Pc, $\kappa = 0.37464 + 1.54226\ \omega - 0.26992\ \omega^2$
- **Assumes:** pure substance; not too close to the critical point or a spinodal (a double root is refused); one real root: both roots reported equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_peng_robinson', Tc=507.6, Pc=3025000.0, omega=0.2975,
T=400.0, P=1000000.0)
# -> Z_vapour = 0.644021, Z_liquid = 0.0469282, phi_vapour = 0.746232, phi_liquid = 0.421597,
n_roots = 3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_peng_robinson', Tc=190.564, Pc=4599000.0, omega=0.011,
T=300.0, P=5000000.0)
# -> Z_vapour = 0.90176, Z_liquid = 0.90176, phi_vapour = 0.901312, phi_liquid = 0.901312, n_roots
= 1   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Peng-Robinson: Tc = 507.6 K, Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa | SOLVED | Z_vapour = 0.644021, Z_liquid = 0.0469282, phi_vapour = 0.746232, phi_liquid = 0.421597, n_roots = 3 |
| Peng-Robinson: Tc = 190.564 K, Pc = 4599000.0 Pa, omega = 0.011, T = 300.0 K, P = 5000000.0 Pa | SOLVED | Z_vapour = 0.90176, Z_liquid = 0.90176, phi_vapour = 0.901312, phi_liquid = 0.901312, n_roots = 1 |
| Peng-Robinson: Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa | NEEDS_INPUT | chemeng_peng_robinson needs Tc (critical temperature). Example: solve('chemeng_peng_robinson', Tc='507.6 K', Pc='3.025e+ |
| Peng-Robinson: Tc = 507.6 K, Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Soave-Redlich-Kwong equation of state: compressibility and fugacity coefficient — `chemeng_srk`

Compressibility factors (vapour and liquid roots) and fugacity coefficients of a pure substance at T and P from the Soave-Redlich-Kwong equation of state.

$$Z^3 - Z^2 + (A - B - B^2)Z - AB = 0$$

| input | meaning | condition |
|---|---|---|
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |
| `omega` | acentric factor | |
| `T` | temperature | |
| `P` | pressure | |

| output | meaning |
|---|---|
| `Z_vapour` | compressibility factor of the vapour-like (largest) root |
| `Z_liquid` | compressibility factor of the liquid-like (smallest) root |
| `phi_vapour` | fugacity coefficient on the vapour root |
| `phi_liquid` | fugacity coefficient on the liquid root |
| `n_roots` | number of real roots above the covolume (1 or 3) |

- **Method:** Z^3 - Z^2 + (A - B - B^2)$Z - AB = 0$ with $a = 0.42748$ Double exponent: use braces to clarify alpha(T), $b = 0.08664$ R Tc/Pc, $m = 0.480 + 1.574\,\omega - 0.176\,\omega^2$
- **Assumes:** pure substance; not too close to the critical point or a spinodal; one real root: both roots reported equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_srk', Tc=507.6, Pc=3025000.0, omega=0.2975, T=400.0,
P=1000000.0)
# -> Z_vapour = 0.65891, Z_liquid = 0.0532184, phi_vapour = 0.757696, phi_liquid = 0.430644,
n_roots = 3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_srk', Tc=190.564, Pc=4599000.0, omega=0.011, T=300.0,
P=5000000.0)
# -> Z_vapour = 0.923848, Z_liquid = 0.923848, phi_vapour = 0.923117, phi_liquid = 0.923117,
n_roots = 1   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Soave-Redlich-Kwong: Tc = 507.6 K, Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa | SOLVED | Z_vapour = 0.65891, Z_liquid = 0.0532184, phi_vapour = 0.757696, phi_liquid = 0.430644, n_roots = 3 |
| Soave-Redlich-Kwong: Tc = 190.564 K, Pc = 4599000.0 Pa, omega = | SOLVED | Z_vapour = 0.923848, Z_liquid = 0.923848, phi_vapour = 0.923117, phi_liquid = 0.923117, n_roots = 1 |

| question | outcome | result |
|---|---|---|
| 0.011, T = 300.0 K, P = 5000000.0 Pa | | |
| Soave-Redlich-Kwong: Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa | NEEDS_INPUT | chemeng_srk needs Tc (critical temperature). Example: solve('chemeng_srk', Tc='507.6 K', Pc='3.025e+06 Pa', omega=0.2975 |
| Soave-Redlich-Kwong: Tc = 507.6 K, Pc = 3025000.0 Pa, omega = 0.2975, T = 400.0 K, P = 1000000.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Vapour pressure by the Lee-Kesler correlation — `chemeng_lee_kesler_vapour_pressure`

Vapour pressure of a pure substance from its critical constants and acentric factor (Lee-Kesler corresponding-states correlation).

$$\ln \frac{P^{sat}}{P_c} = f^{(0)}(T_r) + \omega f^{(1)}(T_r)$$

| input | meaning | condition |
|---|---|---|
| `T` | temperature | |
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |
| `omega` | acentric factor | |

| output | meaning |
|---|---|
| `Psat` | vapour pressure |

- **Method:** ln(Psat/Pc) = f0(Tr) + omega f1(Tr) with the Lee-Kesler functions
- **Assumes:** $T < Tc$; non-polar or slightly polar substances
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_lee_kesler_vapour_pressure', T=400.0, Tc=568.7,
Pc=2490000.0, omega=0.399)
# -> Psat = 104829 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_lee_kesler_vapour_pressure', T=300.0, Tc=425.12,
Pc=3796000.0, omega=0.2)
# -> Psat = 258310 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Lee-Kesler vapour pressure: T = 400.0 K, Tc = 568.7 K, Pc = 2490000.0 Pa, omega = 0.399 | SOLVED | Psat = 104829 Pa |

| question | outcome | result |
|---|---|---|
| Lee-Kesler vapour pressure: T = 300.0 K, Tc = 425.12 K, Pc = 3796000.0 Pa, omega = 0.2 | SOLVED | Psat = 258310 Pa |
| Lee-Kesler vapour pressure: Tc = 568.7 K, Pc = 2490000.0 Pa, omega = 0.399 | NEEDS_INPUT | chemeng_lee_kesler_vapour_pressure needs T (temperature). Example: solve('chemeng_lee_kesler_vapour_pressure', T='400 K' |
| Lee-Kesler vapour pressure: T = 400.0 K, Tc = 568.7 K, Pc = 2490000.0 Pa, omega = 0.399, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Heat of vaporisation at another temperature (Watson) — `chemeng_watson_heat_of_vaporization`

Heat of vaporisation at T from a known value at T_ref and the critical temperature (Watson's correlation, exponent 0.38).

$$\Delta H_v(T) = \Delta H_v(T_{ref}) \left( \frac{1 - T_r}{1 - T_{r,ref}} \right)^{0.38}$$

| input | meaning | condition |
|---|---|---|
| `T` | temperature wanted | |
| `T_ref` | temperature of the known value | |
| `Hvap_ref` | heat of vaporisation at T_ref | |
| `Tc` | critical temperature | |

| output | meaning |
|---|---|
| `Hvap` | heat of vaporisation at T |

- **Method:** dHvap(T) = dHvap(T_ref) [(1 - T/Tc)/(1 - T_ref/Tc)]^0.38
- **Assumes:** T, T_ref < Tc
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_watson_heat_of_vaporization', T=298.15, T_ref=373.15,
Hvap_ref=40656.0, Tc=647.1)
# -> Hvap = 44571.7 J/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_watson_heat_of_vaporization', T=400.0, T_ref=353.24,
Hvap_ref=30720.0, Tc=562.05)
# -> Hvap = 27898.6 J/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Watson heat of vaporization: T = 298.15 K, T_ref = 373.15 K, Hvap_ref = 40656.0 J/mol, Tc = 647.1 K | SOLVED | Hvap = 44571.7 J/mol |
| Watson heat of vaporization: T = 400.0 K, T_ref = 353.24 K, Hvap_ref = 30720.0 J/mol, Tc = 562.05 K | SOLVED | Hvap = 27898.6 J/mol |
| Watson heat of vaporization: T_ref = 373.15 K, Hvap_ref = 40656.0 J/mol, Tc = 647.1 K | NEEDS_INPUT | chemeng_watson_heat_of_vaporization needs T (temperature wanted). Example: solve('chemeng_watson_heat_of_vaporization', |
| Watson heat of vaporization: T = 298.15 K, T_ref = 373.15 K, Hvap_ref = 40656.0 J/mol, Tc = 647.1 K, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Heat of vaporisation at the normal boiling point (Riedel) —

`chemeng_riedel_heat_of_vaporization`

Heat of vaporisation at the normal boiling point from the boiling point and critical constants (Riedel's equation, Pc in bar).

$$\Delta H_{vb} = 1.093\, R\, T_b\, \frac{\ln P_c - 1.013}{0.930 - T_{br}}$$

| input | meaning | condition |
|---|---|---|
| `Tb` | normal boiling point | |
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |

| output | meaning |
|---|---|
| `Hvap_b` | heat of vaporisation at the normal boiling point |

- **Method:** dHvb = 1.093 R Tb (ln Pc[bar] - 1.013)/(0.930 - Tbr)
- **Assumes:** Tb well below Tc
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_riedel_heat_of_vaporization', Tb=294.0, Tc=466.0,
Pc=5550000.0)
# -> Hvap_b = 26828.6 J/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_riedel_heat_of_vaporization', Tb=353.24, Tc=562.05,
Pc=4895000.0)
# -> Hvap_b = 30639.1 J/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Riedel heat of vaporization: Tb = 294.0 K, Tc = 466.0 K, Pc = 5550000.0 Pa | SOLVED | Hvap_b = 26828.6 J/mol |
| Riedel heat of vaporization: Tb = 353.24 K, Tc = 562.05 K, Pc = 4895000.0 Pa | SOLVED | Hvap_b = 30639.1 J/mol |
| Riedel heat of vaporization: Tc = 466.0 K, Pc = 5550000.0 Pa | NEEDS_INPUT | chemeng_riedel_heat_of_vaporization needs Tb (normal boiling point). Example: solve('chemeng_riedel_heat_of_vaporization |
| Riedel heat of vaporization: Tb = 294.0 K, Tc = 466.0 K, Pc = 5550000.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Heat of vaporisation from the acentric factor (Pitzer) — `chemeng_pitzer_heat_of_vaporization`

Heat of vaporisation at T from the critical temperature and acentric factor (Pitzer's corresponding-states correlation as fitted by Carruth and Kobayashi).

$$\frac{\Delta H_v}{RT_c} = 7.08(1-T_r)^{0.354} + 10.95\,\omega(1-T_r)^{0.456}$$

| input | meaning | condition |
|---|---|---|
| `T` | temperature | |
| `Tc` | critical temperature | |
| `omega` | acentric factor | |

| output | meaning |
|---|---|
| `Hvap` | heat of vaporisation |

- **Method:** dHv/(R Tc) = 7.08 $(1-Tr)^{0.354}$ + 10.95 omega $(1-Tr)^{0.456}$
- **Assumes:** $0.6 < Tr < 1$ (fit range)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pitzer_heat_of_vaporization', T=452.0, Tc=645.6,
omega=0.35)
# -> Hvap = 36691 J/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pitzer_heat_of_vaporization', T=300.0, Tc=425.12,
omega=0.2)
# -> Hvap = 20662.5 J/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Pitzer heat of vaporization: T = 452.0 K, Tc = 645.6 K, omega = 0.35 | SOLVED | Hvap = 36691 J/mol |

| question | outcome | result |
|---|---|---|
| Pitzer heat of vaporization: T = 300.0 K, Tc = 425.12 K, omega = 0.2 | SOLVED | Hvap = 20662.5 J/mol |
| Pitzer heat of vaporization: Tc = 645.6 K, omega = 0.35 | NEEDS_INPUT | chemeng_pitzer_heat_of_vaporization needs T (temperature). Example: solve('chemeng_pitzer_heat_of_vaporization', T='452 |
| Pitzer heat of vaporization: T = 452.0 K, Tc = 645.6 K, omega = 0.35, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Saturated liquid molar volume (Rackett) — `chemeng_rackett_liquid_volume`

Molar volume of a saturated liquid from the critical constants and the critical compressibility factor (Rackett equation).

$$V_s = \frac{RT_c}{P_c} Z_c^{1+(1-T_r)^{2/7}}$$

| input | meaning | condition |
|---|---|---|
| `T` | temperature | |
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |
| `Zc` | critical compressibility factor | |

| output | meaning |
|---|---|
| `Vm` | saturated liquid molar volume |

- **Method:** Vs = (R Tc/Pc) Zc^[$1 + (1 - Tr)^{2/7}$]
- **Assumes:** $T < Tc$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_rackett_liquid_volume', T=272.03889, Tc=369.83,
Pc=4248000.0, Zc=0.2763)
# -> Vm = 8.29923e-05 m^3/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_rackett_liquid_volume', T=298.15, Tc=562.05,
Pc=4895000.0, Zc=0.268)
# -> Vm = 8.85568e-05 m^3/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Rackett liquid volume: T = 272.03889 K, Tc = 369.83 K, Pc = 4248000.0 Pa, Zc = 0.2763 | SOLVED | Vm = 8.29923e-05 m^3/mol |

| question | outcome | result |
|---|---|---|
| Rackett liquid volume: T = 298.15 K, Tc = 562.05 K, Pc = 4895000.0 Pa, Zc = 0.268 | SOLVED | Vm = 8.85568e-05 m^3/mol |
| Rackett liquid volume: Tc = 369.83 K, Pc = 4248000.0 Pa, Zc = 0.2763 | NEEDS_INPUT | chemeng_rackett_liquid_volume needs T (temperature). Example: solve('chemeng_rackett_liquid_volume', T='272.039 K', Tc=' |
| Rackett liquid volume: T = 272.03889 K, Tc = 369.83 K, Pc = 4248000.0 Pa, Zc = 0.2763, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Gas compressibility from the virial equation (Pitzer-Abbott correlation) — `chemeng_virial_compressibility`

Second virial coefficient from the Abbott (Pitzer) correlation and the compressibility factor $Z = 1 + BP/(RT)$ of a gas at moderate pressure.

$$\frac{BP_c}{RT_c} = 0.083 - \frac{0.422}{T_r^{1.6}} + \omega\left(0.139 - \frac{0.172}{T_r^{4.2}}\right), \quad Z = 1 + \frac{BP}{RT}$$

| input | meaning | condition |
|---|---|---|
| `T` | temperature | |
| `P` | pressure | |
| `Tc` | critical temperature | |
| `Pc` | critical pressure | |
| `omega` | acentric factor | |

| output | meaning |
|---|---|
| `B` | second virial coefficient |
| `Z` | compressibility factor |

- **Method:** B Pc/(R Tc) = B0 + omega B1, $B_0 = 0.083 - 0.422/Tr^{1.6}$, $B_1 = 0.139 - 0.172/Tr^{4.2}$; $Z = 1 + B$ P/(R T)
- **Assumes:** moderate pressure (truncated virial equation); non-polar or slightly polar gas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_virial_compressibility', T=510.0, P=2500000.0, Tc=425.1,
Pc=3796000.0, omega=0.2)
# -> B = -0.000205361 m^3/mol, Z = 0.878925   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_virial_compressibility', T=300.0, P=1000000.0, Tc=304.2,
Pc=7383000.0, omega=0.224)
# -> B = -0.000122712 m^3/mol, Z = 0.950804   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Virial compressibility: T = 510.0 K, P = 2500000.0 Pa, Tc = 425.1 K, Pc = 3796000.0 Pa, omega = 0.2 | SOLVED | B = -0.000205361 m^3/mol, Z = 0.878925 |
| Virial compressibility: T = 300.0 K, P = 1000000.0 Pa, Tc = 304.2 K, Pc = 7383000.0 Pa, omega = 0.224 | SOLVED | B = -0.000122712 m^3/mol, Z = 0.950804 |
| Virial compressibility: P = 2500000.0 Pa, Tc = 425.1 K, Pc = 3796000.0 Pa, omega = 0.2 | NEEDS_INPUT | chemeng_virial_compressibility needs T (temperature). Example: solve('chemeng_virial_compressibility', T='510 K', P='2.5 |
| Virial compressibility: T = 510.0 K, P = 2500000.0 Pa, Tc = 425.1 K, Pc = 3796000.0 Pa, omega = 0.2, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Mass fractions to mole fractions and mean molar mass — `chemeng_mass_to_mole_fractions`

Mole fractions of a mixture and its mean molar mass from the mass fractions and the molar masses of the components.

$$z_i = \frac{w_i/M_i}{\sum_j w_j/M_j}, \quad \bar{M} = \sum_i z_i M_i$$

| input | meaning | condition |
|---|---|---|
| `w` | mass fractions (sum to 1) | |
| `MW` | molar masses of the components, g/mol | |

| output | meaning |
|---|---|
| `z` | mole fractions |
| `MW_mean` | mean molar mass of the mixture |

- **Method:** z_i = (w_i/M_i)/sum_j (w_j/M_j); M = sum z_i M_i
- **Assumes:** fractions sum to 1; positive molar masses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_mass_to_mole_fractions', w=[0.5, 0.5], MW=[32.042, 18.015])
# -> z = [0.359889725713, 0.640110274287], MW_mean = 0.0230632 kg/mol   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_mass_to_mole_fractions', w=[0.2, 0.3, 0.5], MW=[16.043, 30.07, 44.097])
# -> z = [0.369029335988, 0.2953277837, 0.335642880312], MW_mean = 0.0296017 kg/mol   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Mass to mole fractions: w = [0.5, 0.5], MW = [32.042, 18.015] | SOLVED | z = [0.359889725713, 0.640110274287], MW_mean = 0.0230632 kg/mol |
| Mass to mole fractions: w = [0.2, 0.3, 0.5], MW = [16.043, 30.07, 44.097] | SOLVED | z = [0.369029335988, 0.2953277837, 0.335642880312], MW_mean = 0.0296017 kg/mol |
| Mass to mole fractions: MW = [32.042, 18.015] | NEEDS_INPUT | chemeng_mass_to_mole_fractions needs w (mass fractions (sum to 1)). Example: solve('chemeng_mass_to_mole_fractions', w=[ |
| Mass to mole fractions: w = [0.5, 0.5], MW = [32.042, 18.015], zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Diffusion of A through stagnant B (Stefan film) — `chemeng_stefan_diffusion_flux`

Steady molar flux of A diffusing through a stagnant film of non-diffusing B in an ideal gas (evaporation, absorption through an inert).

$$N_A = \frac{cD_{AB}}{L}\ln\frac{1-y_{A2}}{1-y_{A1}}, \quad c = \frac{P}{RT}$$

| input | meaning | condition |
|---|---|---|
| `D` | diffusivity of A in B | |
| `P` | total pressure | |
| `T` | temperature | |
| `L` | film thickness (diffusion path length) | |
| `yA1` | mole fraction of A at the start of the path | |
| `yA2` | mole fraction of A at the end of the path | |

| output | meaning |
|---|---|
| `NA` | molar flux of A |

- **Method:** N_A = (c D/L) ln[$(1-yA2)/(1-yA1)$], c = P/(R T)
- **Assumes:** steady state, ideal gas, constant D; B stagnant (no net flux of B)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_stefan_diffusion_flux', D=2.6e-05, P=101325.0, T=298.15,
L=0.1, yA1=0.0313, yA2=0.0)
# -> NA = 0.00033795 mol/(m^2*s)   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_stefan_diffusion_flux', D=1e-05, P=200000.0, T=350.0,
L=0.005, yA1=0.6, yA2=0.2)
# -> NA = 0.0952759 mol/(m^2*s)   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Stefan diffusion flux: D = 2.6e-05 m^2/s, P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.0313, yA2 = 0.0 | SOLVED | NA = 0.00033795 mol/(m^2*s) |
| Stefan diffusion flux: D = 1e-05 m^2/s, P = 200000.0 Pa, T = 350.0 K, L = 0.005 m, yA1 = 0.6, yA2 = 0.2 | SOLVED | NA = 0.0952759 mol/(m^2*s) |
| Stefan diffusion flux: P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.0313, yA2 = 0.0 | NEEDS_INPUT | chemeng_stefan_diffusion_flux needs D (diffusivity of A in B). Example: solve('chemeng_stefan_diffusion_flux', D='2.6e-0 |
| Stefan diffusion flux: D = 2.6e-05 m^2/s, P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.0313, yA2 = 0.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Stefan diffusion flux: D = 0.26 cm^2/s, P = 1 atm, T = 25 degC, L = 10 cm, yA1 = 3.13%, yA2 = 0 | SOLVED | NA = 0.00033795 mol/(m^2*s) |

### Packed-bed pressure drop (Ergun equation) — `chemeng_ergun_pressure_drop`

Pressure drop of a fluid flowing through a packed bed of spheres from the particle size, voidage, superficial velocity and fluid properties (Ergun equation).

$$\frac{\Delta P}{L} = \frac{150\,\mu(1-\varepsilon)^2 v_s}{\varepsilon^3 d_p^2} + \frac{1.75\,(1-\varepsilon)\rho v_s^2}{\varepsilon^3 d_p}$$

| input | meaning | condition |
|---|---|---|
| `dp` | particle diameter | |
| `eps` | bed voidage (porosity) | |
| `vs` | superficial velocity | |
| `rho` | fluid density | |
| `mu` | fluid viscosity | |
| `L` | bed depth | |

| output | meaning |
|---|---|
| `dP` | pressure drop across the bed |

- **Method:** $dP/L = 150$ mu (1 - eps)^2 vs/(eps^3 dp^2) + 1.75 (1 - eps) rho vs^2/(eps^3 dp)
- **Assumes:** 0 < eps < 1; incompressible flow (or small pressure drop for a gas); spherical or equivalent particles
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_ergun_pressure_drop', dp=0.0008, eps=0.4, vs=0.05,
rho=1000.0, mu=0.001, L=1.0)
# -> dP = 117188 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_ergun_pressure_drop', dp=0.005, eps=0.38, vs=1.2,
rho=1.2, mu=1.8e-05, L=2.0)
# -> dP = 15483.1 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Ergun pressure drop: dp = 0.0008 m, eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m | SOLVED | dP = 117188 Pa |
| Ergun pressure drop: dp = 0.005 m, eps = 0.38, vs = 1.2 m/s, rho = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, L = 2.0 m | SOLVED | dP = 15483.1 Pa |
| Ergun pressure drop: eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m | NEEDS_INPUT | chemeng_ergun_pressure_drop needs dp (particle diameter). Example: solve('chemeng_ergun_pressure_drop', dp='0.0008 m', e |
| Ergun pressure drop: dp = 0.0008 m, eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Ergun pressure drop: dp = 0.8 mm, eps = 0.4, vs = 5 cm/s, rho = 1000 kg/m^3, mu = 1 cP, L = 1 m | SOLVED | dP = 117188 Pa |

## Packed-bed pressure drop (Carman correlation) — `chemeng_carman_pressure_drop`

Pressure drop through a packed bed from Carman's correlation of the bed friction factor (180/Re' + $2.87/Re$ '^0.1 form).

$$f_p = \frac{\left(180 + 2.871\left(\frac{Re}{1-\varepsilon}\right)^{0.9}\right)(1-\varepsilon)^2}{\varepsilon^3 Re}, \quad \Delta P = \frac{f_p\, \rho v_s^2 L}{d_p}$$

| input | meaning | condition |
|---|---|---|
| `dp` | particle diameter | |
| `eps` | bed voidage (porosity) | |
| `vs` | superficial velocity | |
| `rho` | fluid density | |
| `mu` | fluid viscosity | |
| `L` | bed depth | |

| output | meaning |
|---|---|
| `dP` | pressure drop across the bed |

- **Method:** f_p = (180 + 2.871 (Re/(1 - eps))^0.9)(1 - eps)^2/(eps^3 Re); $dP = f_p$ rho $vs^2$ L/dp, $Re = dp$ rho vs/mu
- **Assumes:** 0 < eps < 1; incompressible flow

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_carman_pressure_drop', dp=0.0008, eps=0.4, vs=0.05,
rho=1000.0, mu=0.001, L=1.0)
# -> dP = 134368 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_carman_pressure_drop', dp=0.005, eps=0.38, vs=1.2,
rho=1.2, mu=1.8e-05, L=2.0)
# -> dP = 13920.1 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Carman pressure drop: dp = 0.0008 m, eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m | SOLVED | dP = 134368 Pa |
| Carman pressure drop: dp = 0.005 m, eps = 0.38, vs = 1.2 m/s, rho = 1.2 kg/m^3, mu = 1.8e-05 Pa*s, L = 2.0 m | SOLVED | dP = 13920.1 Pa |
| Carman pressure drop: eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m | NEEDS_INPUT | chemeng_carman_pressure_drop needs dp (particle diameter). Example: solve('chemeng_carman_pressure_drop', dp='0.0008 m', |
| Carman pressure drop: dp = 0.0008 m, eps = 0.4, vs = 0.05 m/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, L = 1.0 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Constant-pressure cake filtration: filtrate volume after a time —

`chemeng_cake_filtration_volume`

Volume of filtrate collected after time t in constant-pressure cake filtration (Ruth equation with cake and medium resistances).

$$t = \frac{\mu \alpha c}{2A^2 \Delta P} V^2 + \frac{\mu R_m}{A \Delta P} V$$

| input | meaning | condition |
|---|---|---|
| `A` | filter area | |
| `dP` | pressure difference across cake and medium | |
| `mu` | filtrate viscosity | |
| `alpha` | specific cake resistance | |
| `c` | mass of dry cake deposited per volume of filtrate | |
| `Rm` | filter medium resistance | |
| `t` | filtration time | |

| output | meaning |
|---|---|
| `V` | filtrate volume collected |

- **Method:** $dV/dt = A^2$ dP/(mu (alpha c V + A Rm)), integrated: $t = mu$ alpha c V^2/(2 $A^2$ dP) + mu Rm V/(A dP), solved for V
- **Assumes:** constant pressure, incompressible cake; $Rm > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cake_filtration_volume', A=1.0, dP=100000.0, mu=0.001,
alpha=100000000000.0, c=20.0, Rm=10000000000.0, t=600.0)
# -> V = 0.24 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cake_filtration_volume', A=10.0, dP=300000.0, mu=0.0012,
alpha=50000000000.0, c=50.0, Rm=50000000000.0, t=3600.0)
# -> V = 8.28764 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Cake filtration volume: A = 1.0 m^2, dP = 100000.0 Pa, mu = 0.001 Pa*s, alpha = 100000000000.0 m/kg, c = 20.0 kg/m^3, Rm = 10000000000.0 1/m, t = 600.0 s | SOLVED | V = 0.24 m^3 |
| Cake filtration volume: A = 10.0 m^2, dP = 300000.0 Pa, mu = 0.0012 Pa*s, alpha = 50000000000.0 m/kg, c = 50.0 kg/m^3, Rm = 50000000000.0 1/m, t = 3600.0 s | SOLVED | V = 8.28764 m^3 |
| Cake filtration volume: dP = 100000.0 Pa, mu = 0.001 Pa*s, alpha = 100000000000.0 m/kg, c = 20.0 kg/m^3, Rm = 10000000000.0 1/m, t = 600.0 s | NEEDS_INPUT | chemeng_cake_filtration_volume needs A (filter area). Example: solve('chemeng_cake_filtration_volume', A='1 m^2', dP='10 |
| Cake filtration volume: A = 1.0 m^2, dP = 100000.0 Pa, mu = 0.001 Pa*s, alpha = 100000000000.0 m/kg, c = 20.0 kg/m^3, Rm = 10000000000.0 1/m, t = 600.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Specific cake and medium resistance from constant-pressure filtration data — `chemeng_filtration_constants_fit`

Specific cake resistance and filter-medium resistance from a constant-pressure filtration test (times and filtrate volumes), by the straight line of t/V against V.

$$\frac{t}{V} = \frac{\mu \alpha c}{2A^2 \Delta P} V + \frac{\mu R_m}{A \Delta P}$$

| input | meaning | condition |
|---|---|---|
| `t` | times at which the filtrate volumes were read, in t_unit | |

| input | meaning | condition |
|---|---|---|
| `V` | cumulative filtrate volumes, in v_unit | |
| `t_unit` | unit of the times | |
| `v_unit` | unit of the volumes | |
| `A` | filter area | |
| `dP` | constant pressure difference | |
| `mu` | filtrate viscosity | |
| `c` | dry cake mass per volume of filtrate | |

| output | meaning |
|---|---|
| `alpha` | specific cake resistance |
| `Rm` | filter medium resistance |
| `slope` | slope of t/V against V, $s/m^6$ |
| `intercept` | intercept of t/V against V, $s/m^3$ |

- **Method:** least squares of t/V = (mu alpha c/(2 $A^2$ dP)) V + mu Rm/(A dP)
- **Assumes:** constant pressure, incompressible cake; a positive slope and intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_filtration_constants_fit', t=[17.3, 41.3, 72.0, 108.3,
152.1, 201.7], V=[0.5, 1.0, 1.5, 2.0, 2.5, 3.0], t_unit='s', v_unit='L', A=0.0439, dP=338000.0,
mu=0.000895, c=23.47)
# -> alpha = 8.07822e+11 m/kg, Rm = 4.67973e+11 1/m, slope = 1.3025e+07, intercept = 28226.9
(SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_filtration_constants_fit', t=[2.0, 5.0, 9.5, 15.0,
21.5], V=[0.1, 0.2, 0.3, 0.4, 0.5], t_unit='min', v_unit='m3', A=2.0, dP=150000.0, mu=0.001,
c=30.0)
# -> alpha = 1.404e+11 m/kg, Rm = 2.499e+11 1/m, slope = 3510, intercept = 833   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Filtration constants fit: t = [17.3, 41.3, 72.0, 108.3, 152.1, 201.7], V = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0], t_unit = s, v_unit = L, A = 0.0439 m^2, dP = 338000.0 Pa, mu = 0.000895 Pa*s, c = 23.47 kg/m^3 | SOLVED | alpha = 8.07822e+11 m/kg, Rm = 4.67973e+11 1/m, slope = 1.3025e+07, intercept = 28226.9 |
| Filtration constants fit: t = [2.0, 5.0, 9.5, 15.0, 21.5], V = [0.1, 0.2, 0.3, 0.4, 0.5], t_unit = min, v_unit = m3, A = 2.0 m^2, dP = 150000.0 Pa, mu = 0.001 Pa*s, c = 30.0 kg/m^3 | SOLVED | alpha = 1.404e+11 m/kg, Rm = 2.499e+11 1/m, slope = 3510, intercept = 833 |
| Filtration constants fit: V = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0], t_unit = s, v_unit = L, A = 0.0439 m^2, dP = 338000.0 Pa, mu = 0.000895 Pa*s, c = 23.47 kg/m^3 | NEEDS_INPUT | chemeng_filtration_constants_fit needs t (times at which the filtrate volumes were read, in t_unit). Example: solve('che |

| question | outcome | result |
|---|---|---|
| Filtration constants fit: t = [17.3, 41.3, 72.0, 108.3, 152.1, 201.7], V = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0], t_unit = s, v_unit = L, A = 0.0439 m^2, dP = 338000.0 Pa, mu = 0.000895 Pa*s, c = 23.47 kg/m^3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### First-order-plus-dead-time process: response to a step — `chemeng_fopdt_step_response`

Deviation of the output of a first-order process with dead time at time t after a step of size M in its input.

$$y(t) = KM\left(1 - e^{-(t-\theta)/\tau}\right) u(t-\theta)$$

| input | meaning | condition |
|---|---|---|
| `K` | process gain (output change per unit input change) | |
| `tau` | process time constant | |
| `theta` | dead time | |
| `M` | size of the input step (deviation) | |
| `t` | time after the step | |

| output | meaning |
|---|---|
| `y` | output deviation at time t (in the output's units) |

- **Method:** y(t) = K M (1 - exp(-(t - theta)/tau)) for $t > \theta$, 0 before
- **Assumes:** the process starts at steady state; linear first-order dynamics with dead time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_fopdt_step_response', K=2.0, tau=300.0, theta=60.0, M=5.0, t=360.0)
# -> y = 6.32121   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_fopdt_step_response', K=-0.8, tau=120.0, theta=30.0, M=10.0, t=600.0)
# -> y = -7.93079   (SOLVED)
```

#### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| FOPDT step response: K = 2.0, tau = 300.0 s, theta = 60.0 s, M = 5.0, t = 360.0 s | SOLVED | y = 6.32121 |
| FOPDT step response: K = -0.8, tau = 120.0 s, theta = 30.0 s, M = 10.0, t = 600.0 s | SOLVED | y = -7.93079 |
| FOPDT step response: tau = 300.0 s, theta = 60.0 s, M = 5.0, t = 360.0 s | NEEDS_INPUT | chemeng_fopdt_step_response needs K (process gain (output change per unit input change)). Example: solve('chemeng_fopdt_ |

| question | outcome | result |
|---|---|---|
| FOPDT step response: K = 2.0, tau = 300.0 s, theta = 60.0 s, M = 5.0, t = 360.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| FOPDT step response: K = 2, tau = 5 min, theta = 1 min, M = 5, t = 6 min | SOLVED | y = 6.32121 |

### Proportional control: steady-state offset after a set-point step — `chemeng_p_control_offset`

Steady-state output (as a fraction of a set-point step) and the offset of a P-only feedback loop around a process of first-order lags in series, after checking the loop is stable.

$$\text{offset} = \frac{1}{1 + K_c K}$$

| input | meaning | condition |
|---|---|---|
| `Kc` | controller gain | |
| `K` | process gain | |
| `taus` | time constants of the first-order lags in series, in t_unit | |
| `t_unit` | unit of the time constants | |

| output | meaning |
|---|---|
| `y_ss` | final output change per unit set-point change |
| `offset` | steady-state offset per unit set-point change |

- **Method:** final-value theorem: $y_s s = Kc$ K/(1 + Kc K), offset = 1/(1 + Kc K); stability of prod(tau_i s + 1) + Kc K by the Routh array
- **Assumes:** Kc $K > 0$; the closed loop is stable (checked by the Routh array)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_p_control_offset', Kc=2.0, K=1.5, taus=[5.0, 2.0],
t_unit='min')
# -> y_ss = 0.75, offset = 0.25   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_p_control_offset', Kc=0.5, K=2.0, taus=[10.0, 5.0, 1.0],
t_unit='s')
# -> y_ss = 0.5, offset = 0.5   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| P-only control offset: Kc = 2.0, K = 1.5, taus = [5.0, 2.0], t_unit = min | SOLVED | y_ss = 0.75, offset = 0.25 |
| P-only control offset: Kc = 0.5, K = 2.0, taus = [10.0, 5.0, 1.0], | SOLVED | y_ss = 0.5, offset = 0.5 |

| question | outcome | result |
|---|---|---|
| t_unit = s | | |
| P-only control offset: K = 1.5, taus = [5.0, 2.0], t_unit = min | NEEDS_INPUT | chemeng_p_control_offset needs Kc (controller gain). Example: solve('chemeng_p_control_offset', Kc=2, K=1.5, taus=[5.0, |
| P-only control offset: Kc = 2.0, K = 1.5, taus = [5.0, 2.0], t_unit = min, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| P-only control offset: Kc = 20, K = 2, taus = [10, 10, 10], t_unit = s | INVALID_INPUT | Proportional control: steady-state offset after a set-point step: the closed loop is unstable at this controller gain: t |

### Frequency response of first-order lags in series (amplitude ratio and phase) — `chemeng_lag_frequency_response`

Amplitude ratio and phase angle of a process made of one to three first-order lags in series, at angular frequency w.

$$AR = \frac{K}{\prod_i \sqrt{1+\omega^2\tau_i^2}}, \quad \varphi = -\sum_i \arctan(\omega\tau_i)$$

| input | meaning | condition |
|---|---|---|
| `K` | process gain | |
| `taus` | time constants of the lags, in t_unit | |
| `t_unit` | unit of the time constants | |
| `w` | angular frequency of the input sinusoid | |

| output | meaning |
|---|---|
| `AR` | amplitude ratio (output amplitude per input amplitude) |
| `phase_deg` | phase angle in degrees (negative: lag) |

- **Method:** AR = K/prod sqrt(1 + w^2 tau_i^2); phase = -sum atan(w tau_i)
- **Assumes:** $K > 0$; linear lags without dead time
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_lag_frequency_response', K=2.0, taus=[10.0], t_unit='s',
w=0.1)
# -> AR = 1.41421, phase_deg = -45   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_lag_frequency_response', K=1.0, taus=[5.0, 2.0, 1.0],
t_unit='min', w=0.01)
# -> AR = 0.173594, phase_deg = -152.723   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Lag frequency response: K = 2.0, taus = [10.0], t_unit = s, w = 0.1 rad/s | SOLVED | AR = 1.41421, phase_deg = -45 |
| Lag frequency response: K = 1.0, taus = [5.0, 2.0, 1.0], t_unit = min, w = 0.01 rad/s | SOLVED | AR = 0.173594, phase_deg = -152.723 |
| Lag frequency response: taus = [10.0], t_unit = s, w = 0.1 rad/s | NEEDS_INPUT | chemeng_lag_frequency_response needs K (process gain). Example: solve('chemeng_lag_frequency_response', K=2, taus=[10.0] |
| Lag frequency response: K = 2.0, taus = [10.0], t_unit = s, w = 0.1 rad/s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Discrete PID controller output for an error history — `chemeng_pid_controller_output`

Output of a digital position-form PID controller at each sample for a stated history of errors (rectangular integration, derivative on the error, loop at rest before the first sample).

$$u_k = \bar{u} + K_c \left[ e_k + \frac{\Delta t}{\tau_I} \sum_{j=1}^{k} e_j + \frac{\tau_D}{\Delta t} (e_k - e_{k-1}) \right]$$

| input | meaning | condition |
|---|---|---|
| `Kc` | controller gain | |
| `tauI` | integral (reset) time | |
| `tauD` | derivative time (0 for PI) | |
| `dt` | sampling interval | |
| `e` | error samples e_1 ... e_n (set point minus measurement) | |
| `bias` | controller output bias (output when the error has always been zero) | |

| output | meaning |
|---|---|
| `u_last` | controller output at the last sample |
| `u` | controller output at every sample |

- **Method:** u_k = bias + Kc [e_k + (dt/tauI) sum_(j<=k) e_j + (tauD/dt)(e_k - e_(k-1))]
- **Assumes:** position form, $e_0 = 0$ before the first sample; rectangular integration including the current sample
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pid_controller_output', Kc=2.0, tauI=10.0, tauD=1.0,
dt=1.0, e=[1.0, 0.8, 0.5, 0.3], bias=50.0)
# -> u_last = 50.72, u = [54.2, 51.56, 50.86, 50.72]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pid_controller_output', Kc=0.5, tauI=60.0, tauD=0.0,
dt=2.0, e=[2.0, 2.0, 1.5, 1.0, 0.5], bias=40.0)
# -> u_last = 40.3667, u = [41.0333333333, 41.0666666667, 40.8416666667, 40.6083333333,
40.3666666667]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| PID controller output: Kc = 2.0, tauI = 10.0 s, tauD = 1.0 s, dt = 1.0 s, e = [1.0, 0.8, 0.5, 0.3], bias = 50.0 | SOLVED | u_last = 50.72, u = [54.2, 51.56, 50.86, 50.72] |
| PID controller output: Kc = 0.5, tauI = 60.0 s, tauD = 0.0 s, dt = 2.0 s, e = [2.0, 2.0, 1.5, 1.0, 0.5], bias = 40.0 | SOLVED | u_last = 40.3667, u = [41.0333333333, 41.0666666667, 40.8416666667, 40.6083333333, 40.3666666667] |
| PID controller output: tauI = 10.0 s, tauD = 1.0 s, dt = 1.0 s, e = [1.0, 0.8, 0.5, 0.3], bias = 50.0 | NEEDS_INPUT | chemeng_pid_controller_output needs Kc (controller gain). Example: solve('chemeng_pid_controller_output', Kc=2, tauI='10 |
| PID controller output: Kc = 2.0, tauI = 10.0 s, tauD = 1.0 s, dt = 1.0 s, e = [1.0, 0.8, 0.5, 0.3], bias = 50.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Ziegler-Nichols closed-loop tuning of lags in series — `chemeng_ziegler_nichols_tuning`

Ultimate gain and period of a process of first-order lags in series, and the Ziegler-Nichols closed-loop settings of a P, PI or PID controller.

$$\sum_i \arctan(\omega_u \tau_i) = \pi, \quad K_u = \frac{1}{|G(j\omega_u)|}, \quad P_u = \frac{2\pi}{\omega_u}$$

| input | meaning | condition |
|---|---|---|
| `K` | process gain | |
| `taus` | time constants of the lags (three or more), in t_unit | |
| `t_unit` | unit of the time constants | |
| `controller` | controller type: P, PI or PID | |

| output | meaning |
|---|---|
| `Ku` | ultimate gain |
| `Pu` | ultimate period |
| `Kc` | controller gain |
| `tauI` | integral time (PI, PID) |
| `tauD` | derivative time (PID) |

- **Method:** phase crossover sum atan(w tau_i) = pi (bisection); $Ku = 1$/|G(j wu)|, $Pu = 2$ pi/wu; P: 0.5 Ku; PI: 0.45 Ku, $Pu/1.2$; PID: 0.6 Ku, $Pu/2$, $Pu/8$
- **Assumes:** three or more lags (a finite ultimate gain); $K > 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_ziegler_nichols_tuning', K=2.0, taus=[1.0, 1.0, 1.0],
t_unit='min', controller='PID')
# -> Ku = 4, Pu = 217.656 s, Kc = 2.4, tauI = 108.828 s, tauD = 27.207 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_ziegler_nichols_tuning', K=1.5, taus=[10.0, 5.0, 2.0],
t_unit='s', controller='PI')
# -> Ku = 8.4, Pu = 15.239 s, Kc = 3.78, tauI = 12.6991 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Ziegler-Nichols tuning: K = 2.0, taus = [1.0, 1.0, 1.0], t_unit = min, controller = PID | SOLVED | Ku = 4, Pu = 217.656 s, Kc = 2.4, tauI = 108.828 s, tauD = 27.207 s |
| Ziegler-Nichols tuning: K = 1.5, taus = [10.0, 5.0, 2.0], t_unit = s, controller = PI | SOLVED | Ku = 8.4, Pu = 15.239 s, Kc = 3.78, tauI = 12.6991 s |
| Ziegler-Nichols tuning: taus = [1.0, 1.0, 1.0], t_unit = min, controller = PID | NEEDS_INPUT | chemeng_ziegler_nichols_tuning needs K (process gain). Example: solve('chemeng_ziegler_nichols_tuning', K=2, taus=[1.0, |
| Ziegler-Nichols tuning: K = 2.0, taus = [1.0, 1.0, 1.0], t_unit = min, controller = PID, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Ziegler-Nichols tuning: K = 2, taus = [1, 1], t_unit = min, controller = PID | INVALID_INPUT | taus: needs at least 3 entries, got 2. |

## Closed-loop stability of a P-controlled process (closed-loop poles) —

`chemeng_closed_loop_stability`

Whether a feedback loop with a proportional controller Kc around a process G(s) = num(s)/den(s) is stable: the largest real part of the closed-loop poles and how many lie in the right half-plane.

$$1 + K_c G(s) = 0 \iff \mathrm{den}(s) + K_c\,\mathrm{num}(s) = 0$$

| input | meaning | condition |
|---|---|---|
| `num` | numerator coefficients of G(s), highest power of s first | |
| `den` | denominator coefficients of G(s), highest power of s first | |
| `Kc` | controller gain | |

| output | meaning |
|---|---|
| `max_real` | largest real part of the closed-loop poles, 1/(time unit of the coefficients) |
| `n_rhp` | number of closed-loop poles in the right half-plane |
| `stable` | stable or unstable (text) |

- **Method:** roots of den(s) + Kc num(s) = 0 (30-digit polynomial root finding)
- **Assumes:** proper process; no pole on the imaginary axis or nearly repeated poles (refused)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_closed_loop_stability', num=[1.0], den=[1.0, 3.0, 3.0,
1.0], Kc=10.0)
# -> max_real = 0.0772173, n_rhp = 2, stable = unstable   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_closed_loop_stability', num=[2.0], den=[1.0, 3.0, 3.0,
1.0], Kc=2.0)
# -> max_real = -0.206299, n_rhp = 0, stable = stable   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Closed-loop stability: num = [1.0], den = [1.0, 3.0, 3.0, 1.0], Kc = 10.0 | SOLVED | max_real = 0.0772173, n_rhp = 2, stable = unstable |
| Closed-loop stability: num = [2.0], den = [1.0, 3.0, 3.0, 1.0], Kc = 2.0 | SOLVED | max_real = -0.206299, n_rhp = 0, stable = stable |
| Closed-loop stability: den = [1.0, 3.0, 3.0, 1.0], Kc = 10.0 | NEEDS_INPUT | chemeng_closed_loop_stability needs num (numerator coefficients of G(s), highest power of s first). Example: solve('chem |
| Closed-loop stability: num = [1.0], den = [1.0, 3.0, 3.0, 1.0], Kc = 10.0, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Gain and phase margins of a PI loop around lags in series — `chemeng_pi_loop_margins`

Gain margin, phase margin and the two crossover frequencies of a loop with a PI controller and a process of three to five first-order lags.

$$GM = \frac{1}{|L(j\omega_{pc})|}, \quad PM = 180^\circ + \arg L(j\omega_{gc})$$

| input | meaning | condition |
|---|---|---|
| `Kc` | controller gain | |
| `tauI` | integral time | |
| `K` | process gain | |
| `taus` | time constants of the lags, in t_unit (the integral time is converted to this unit too) | |
| `t_unit` | unit of the time constants | |

| output | meaning |
|---|---|
| `GM` | gain margin (ratio) |
| `PM_deg` | phase margin, degrees |

| output | meaning |
|---|---|
| `w_pc` | phase-crossover frequency (phase -180 degrees) |
| `w_gc` | gain-crossover frequency (\|L\| = 1) |

- **Method:** L(jw) = Kc (1 + 1/(j w tauI)) K/prod($1 + j$ w tau_i); |L| = 1 and arg $L = -180$ degrees by bisection; $GM = 1$/|L(j w_pc)|, PM = 180 + arg L(j w_gc)
- **Assumes:** one phase crossover (refused otherwise); Kc $K > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pi_loop_margins', Kc=1.0, tauI=5.0, K=2.0, taus=[1.0,
1.0, 1.0], t_unit='min')
# -> GM = 3.22094, PM_deg = 51.0531, w_pc = 0.02625 rad/s, w_gc = 0.0131317 rad/s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pi_loop_margins', Kc=0.5, tauI=20.0, K=1.5, taus=[10.0,
5.0, 2.0], t_unit='s')
# -> GM = 12.9321, PM_deg = 90.0229, w_pc = 0.365703 rad/s, w_gc = 0.0446205 rad/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| PI loop margins: Kc = 1.0, tauI = 5.0 s, K = 2.0, taus = [1.0, 1.0, 1.0], t_unit = min | SOLVED | GM = 3.22094, PM_deg = 51.0531, w_pc = 0.02625 rad/s, w_gc = 0.0131317 rad/s |
| PI loop margins: Kc = 0.5, tauI = 20.0 s, K = 1.5, taus = [10.0, 5.0, 2.0], t_unit = s | SOLVED | GM = 12.9321, PM_deg = 90.0229, w_pc = 0.365703 rad/s, w_gc = 0.0446205 rad/s |
| PI loop margins: tauI = 5.0 s, K = 2.0, taus = [1.0, 1.0, 1.0], t_unit = min | NEEDS_INPUT | chemeng_pi_loop_margins needs Kc (controller gain). Example: solve('chemeng_pi_loop_margins', Kc=1, tauI='5 s', K=2, tau |
| PI loop margins: Kc = 1.0, tauI = 5.0 s, K = 2.0, taus = [1.0, 1.0, 1.0], t_unit = min, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Process gain and time constant from step-test data (least squares) — `chemeng_first_order_step_fit`

Gain and time constant of a first-order process (dead time known) fitted by least squares to the recorded response of a step test.

$$\min_{K,\tau} \sum_k \left[ y_k - KM\left(1 - e^{-(t_k-\theta)/\tau}\right)\right]^2$$

| input | meaning | condition |
|---|---|---|
| `t` | sample times from the moment of the step, in t_unit | |
| `y` | output deviations at those times | |
| `t_unit` | unit of the times | |

| input | meaning | condition |
|---|---|---|
| `M` | size of the input step | |
| `theta` | dead time (known) | |

| output | meaning |
|---|---|
| `K` | process gain |
| `tau` | time constant |

- **Method:** least squares of $y = K$ M (1 - exp(-(t - theta)/tau)): K solved linearly for each tau, tau from the zero of the derivative of the residual sum (bisection)
- **Assumes:** step from steady state; first-order response after the dead time; a unique interior least-squares minimum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_first_order_step_fit', t=[0, 1, 2, 3, 4, 5, 6, 8, 10,
12, 15, 20], y=[0.0, 0.0, 1.7, 3.1, 4.3, 5.2, 6.0, 7.1, 7.9, 8.4, 8.8, 9.1], t_unit='min', M=5.0,
theta=60.0)
# -> K = 1.86707, tau = 292.778 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_first_order_step_fit', t=[0, 10, 20, 30, 40, 60, 80,
100, 150, 200], y=[0.0, -0.9, -1.75, -2.4, -2.95, -3.7, -4.2, -4.5, -4.85, -4.95], t_unit='s',
M=2.0, theta=0.0)
# -> K = -2.52645, tau = 46.0905 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| First-order step test fit: t = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 20.0], y = [0.0, 0.0, 1.7, 3.1, 4.3, 5.2, 6.0, 7.1, 7.9, 8.4, 8.8, 9.1], t_unit = min, M = 5.0, theta = 60.0 s | SOLVED | K = 1.86707, tau = 292.778 s |
| First-order step test fit: t = [0.0, 10.0, 20.0, 30.0, 40.0, 60.0, 80.0, 100.0, 150.0, 200.0], y = [0.0, -0.9, -1.75, -2.4, -2.95, -3.7, -4.2, -4.5, -4.85, -4.95], t_unit = s, M = 2.0, theta = 0.0 s | SOLVED | K = -2.52645, tau = 46.0905 s |
| First-order step test fit: y = [0.0, 0.0, 1.7, 3.1, 4.3, 5.2, 6.0, 7.1, 7.9, 8.4, 8.8, 9.1], t_unit = min, M = 5.0, theta = 60.0 s | NEEDS_INPUT | chemeng_first_order_step_fit needs t (sample times from the moment of the step, in t_unit). Example: solve('chemeng_firs |
| First-order step test fit: t = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 20.0], y = [0.0, 0.0, 1.7, 3.1, 4.3, 5.2, 6.0, 7.1, 7.9, 8.4, 8.8, 9.1], t_unit = min, M = 5.0, theta = 60.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Steady radial flow of oil to a well (Darcy) and productivity index — `chemeng_radial_darcy_flow`

Steady-state reservoir flow rate into a vertical well from a circular drainage area, with an optional skin factor, and the productivity index.

$$q = \frac{2\pi k h\,(p_e - p_w)}{\mu\left(\ln\frac{r_e}{r_w} + S\right)}$$

| input | meaning | condition |
|---|---|---|
| `k` | permeability (md is converted here) | |
| `h` | net pay thickness | |
| `mu` | fluid viscosity (cP is converted here) | |
| `pe` | pressure at the drainage radius | |
| `pw` | flowing bottom-hole pressure | |
| `re` | drainage radius | |
| `rw` | wellbore radius | |
| `S` | skin factor (default 0: undamaged well) | |

| output | meaning |
|---|---|
| `q` | flow rate at reservoir conditions |
| `PI` | productivity index q/(pe - pw) |

- **Method:** $q = 2$ pi k h (pe - pw)/(mu (ln(re/rw) + S))
- **Assumes:** steady state, single-phase incompressible flow; $re > rw$, $pe > pw$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_radial_darcy_flow', k=9.869233e-14, h=9.144, mu=0.0015,
pe=20684271.879505083, pw=17236893.232920904, re=304.8, rw=0.09144000000000001)
# -> q = 0.00160651 m^3/s, PI = 4.6601e-10 m^3/(s*Pa)   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_radial_darcy_flow', k=5e-14, h=15.0, mu=0.002,
pe=25000000.0, pw=18000000.0, re=300.0, rw=0.1, S=3.0)
# -> q = 0.00149853 m^3/s, PI = 2.14076e-10 m^3/(s*Pa)   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Radial Darcy flow: k = 9.869233e-14 m^2, h = 9.144 m, mu = 0.0015 Pa*s, pe = 20684271.879505083 Pa, pw = 17236893.232920904 Pa, re = 304.8 m, rw = 0.09144000000000001 m | SOLVED | q = 0.00160651 m^3/s, PI = 4.6601e-10 m^3/(s*Pa) |
| Radial Darcy flow: k = 5e-14 m^2, h = 15.0 m, mu = 0.002 Pa*s, pe = 25000000.0 Pa, pw = 18000000.0 Pa, re = 300.0 m, rw = 0.1 m, S = 3.0 | SOLVED | q = 0.00149853 m^3/s, PI = 2.14076e-10 m^3/(s*Pa) |

| question | outcome | result |
|---|---|---|
| Radial Darcy flow: h = 9.144 m, mu = 0.0015 Pa*s, pe = 20684271.879505083 Pa, pw = 17236893.232920904 Pa, re = 304.8 m, rw = 0.09144000000000001 m | NEEDS_INPUT | chemeng_radial_darcy_flow needs k (permeability (md is converted here)). Example: solve('chemeng_radial_darcy_flow', k=' |
| Radial Darcy flow: k = 9.869233e-14 m^2, h = 9.144 m, mu = 0.0015 Pa*s, pe = 20684271.879505083 Pa, pw = 17236893.232920904 Pa, re = 304.8 m, rw = 0.09144000000000001 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Radial Darcy flow: k = 100 md, h = 30 ft, mu = 1.5 cP, pe = 3000 psi, pw = 2500 psi, re = 1000 ft, rw = 0.3 ft | SOLVED | q = 0.00160651 m^3/s, PI = 4.6601e-10 m^3/(s*Pa) |
| Radial Darcy flow: k = 100 md, h = 30 ft, mu = 1.5 cP, pe = 2000 psi, pw = 2500 psi, re = 1000 ft, rw = 0.3 ft | INVALID_INPUT | Steady radial flow of oil to a well (Darcy) and productivity index: the boundary pressure must exceed the flowing well p |

### Exponential decline curve: rate and cumulative production — `chemeng_exponential_decline`

Production rate and cumulative production at time t of a well in exponential (constant-percentage) decline (Arps b = 0).

$$q = q_i e^{-D_i t}, \quad N_p = \frac{q_i - q}{D_i}$$

| input | meaning | condition |
|---|---|---|
| `qi` | initial rate (bbl/day and STB/day are converted here) | |
| `Di` | initial nominal decline rate, e.g. 0.3 $1/yr$ | |
| `t` | time since the start of the decline | |

| output | meaning |
|---|---|
| `q` | rate at time t |
| `Np` | cumulative production to time t |

- **Method:** $q = qi$ exp(-Di t); $Np = (qi - q)/Di$
- **Assumes:** boundary-dominated flow, constant operating conditions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_exponential_decline', qi=0.0018401307283333335,
Di=9.506426344208685e-09, t=63115200.0)
# -> q = 0.00100989 m^3/s, Np = 87335.2 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_exponential_decline', qi=0.01, Di=3.1688087814028953e-
09, t=157788000.0)
# -> q = 0.00606531 m^3/s, Np = 1.24169e+06 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Exponential decline: qi = 0.0018401307283333335 m^3/s, Di = 9.506426344208685e-09 1/s, t = 63115200.0 s | SOLVED | q = 0.00100989 m^3/s, Np = 87335.2 m^3 |
| Exponential decline: qi = 0.01 m^3/s, Di = 3.1688087814028953e-09 1/s, t = 157788000.0 s | SOLVED | q = 0.00606531 m^3/s, Np = 1.24169e+06 m^3 |
| Exponential decline: Di = 9.506426344208685e-09 1/s, t = 63115200.0 s | NEEDS_INPUT | chemeng_exponential_decline needs qi (initial rate (bbl/day and STB/day are converted here)). Example: solve('chemeng_ex |
| Exponential decline: qi = 0.0018401307283333335 m^3/s, Di = 9.506426344208685e-09 1/s, t = 63115200.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Exponential decline: qi = 1000 bbl/day, Di = 0.3 1/yr, t = 2 yr | SOLVED | q = 0.00100989 m^3/s, Np = 87335.2 m^3 |
| Exponential decline: qi = 1000 bbl/day, Di = 0.3, t = 2 yr | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Hyperbolic decline curve: rate and cumulative production — `chemeng_hyperbolic_decline`

Production rate and cumulative production at time t of a well in hyperbolic decline with Arps exponent $0 < b < 1$.

$$q = \frac{q_i}{(1 + bD_i t)^{1/b}}, \quad N_p = \frac{q_i^b}{(1-b)D_i}\left(q_i^{1-b} - q^{1-b}\right)$$

| input | meaning | condition |
|---|---|---|
| `qi` | initial rate (bbl/day and STB/day are converted here) | |
| `Di` | initial nominal decline rate, e.g. 0.3 $1/yr$ | |
| `b` | Arps decline exponent $(0 < b < 1)$ | |
| `t` | time since the start of the decline | |

| output | meaning |
|---|---|
| `q` | rate at time t |
| `Np` | cumulative production to time t |

- **Method:** $q = qi$ (1 + b Di t)^(-1/b); $Np = qi^b$/($(1-b)$ Di) (qi^(1-b) - q^(1-b))
- **Assumes:** $0 < b < 1$; boundary-dominated flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_hyperbolic_decline', qi=0.0018401307283333335,
Di=1.5844043907014474e-08, b=0.5, t=94672800.0)
# -> q = 0.000600859 m^3/s, Np = 99548.8 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_hyperbolic_decline', qi=0.01, Di=3.168808781402895e-08,
b=0.8, t=315576000.0)
# -> q = 0.0006415 m^3/s, Np = 666891 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Hyperbolic decline: qi = 0.0018401307283333335 m^3/s, Di = 1.5844043907014474e-08 1/s, b = 0.5, t = 94672800.0 s | SOLVED | q = 0.000600859 m^3/s, Np = 99548.8 m^3 |
| Hyperbolic decline: qi = 0.01 m^3/s, Di = 3.168808781402895e-08 1/s, b = 0.8, t = 315576000.0 s | SOLVED | q = 0.0006415 m^3/s, Np = 666891 m^3 |
| Hyperbolic decline: Di = 1.5844043907014474e-08 1/s, b = 0.5, t = 94672800.0 s | NEEDS_INPUT | chemeng_hyperbolic_decline needs qi (initial rate (bbl/day and STB/day are converted here)). Example: solve('chemeng_hyp |
| Hyperbolic decline: qi = 0.0018401307283333335 m^3/s, Di = 1.5844043907014474e-08 1/s, b = 0.5, t = 94672800.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Hyperbolic decline: qi = 1000 bbl/day, Di = 0.5 1/yr, b = 1, t = 3 yr | INVALID_INPUT | Hyperbolic decline curve: rate and cumulative production: the hyperbolic exponent b must lie strictly between 0 and 1 (b |

## Harmonic decline curve: rate and cumulative production — `chemeng_harmonic_decline`

Production rate and cumulative production at time t of a well in harmonic decline (Arps b = 1).

$$q = \frac{q_i}{1 + D_i t}, \quad N_p = \frac{q_i}{D_i} \ln \frac{q_i}{q}$$

| input | meaning | condition |
|---|---|---|
| `qi` | initial rate (bbl/day and STB/day are converted here) | |
| `Di` | initial nominal decline rate, e.g. 0.3 $1/yr$ | |
| `t` | time since the start of the decline | |

| output | meaning |
|---|---|
| `q` | rate at time t |
| `Np` | cumulative production to time t |

- **Method:** q = qi/(1 + Di t); $Np = (qi/Di)$ ln(qi/q)
- **Assumes:** boundary-dominated flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_harmonic_decline', qi=0.0018401307283333335,
Di=1.2675235125611581e-08, t=94672800.0)
# -> q = 0.000836423 m^3/s, Np = 114465 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_harmonic_decline', qi=0.01, Di=3.168808781402895e-08,
t=315576000.0)
# -> q = 0.000909091 m^3/s, Np = 756718 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Harmonic decline: qi = 0.0018401307283333335 m^3/s, Di = 1.2675235125611581e-08 1/s, t = 94672800.0 s | SOLVED | q = 0.000836423 m^3/s, Np = 114465 m^3 |
| Harmonic decline: qi = 0.01 m^3/s, Di = 3.168808781402895e-08 1/s, t = 315576000.0 s | SOLVED | q = 0.000909091 m^3/s, Np = 756718 m^3 |
| Harmonic decline: Di = 1.2675235125611581e-08 1/s, t = 94672800.0 s | NEEDS_INPUT | chemeng_harmonic_decline needs qi (initial rate (bbl/day and STB/day are converted here)). Example: solve('chemeng_harmo |
| Harmonic decline: qi = 0.0018401307283333335 m^3/s, Di = 1.2675235125611581e-08 1/s, t = 94672800.0 s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Oil initially in place from the material balance (Havlena-Odeh straight line) — `chemeng_havlena_odeh_oil_in_place`

Stock-tank oil initially in place from the underground withdrawal F and the total expansion term Et of successive pressure surveys (no water influx), by the straight line $F = N\,Et$ through the origin.

$$F = N\,E_t, \quad N = \frac{\sum F E_t}{\sum E_t^2}$$

| input | meaning | condition |
|---|---|---|
| `F` | underground withdrawals at each survey, in v_unit at reservoir conditions | |
| `Et` | total expansion per unit of oil in place at each survey (reservoir volume per stock-tank volume) | |
| `v_unit` | unit of F: m3 or bbl | |

| output | meaning |
|---|---|
| `N` | oil initially in place (stock-tank volume) |

- **Method:** least squares through the origin: N = sum F Et / sum $Et^2$
- **Assumes:** volumetric reservoir (no water influx); F and Et on one consistent basis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_havlena_odeh_oil_in_place', F=[580700.0, 1600000.0,
2800000.0, 4100000.0], Et=[0.0101, 0.0278, 0.0485, 0.0712], v_unit='bbl')
# -> N = 9.1612e+06 m^3   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_havlena_odeh_oil_in_place', F=[120000.0, 260000.0,
410000.0], Et=[0.004, 0.0085, 0.0137], v_unit='m3')
# -> N = 3.01044e+07 m^3   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Havlena-Odeh oil in place: F = [580700.0, 1600000.0, 2800000.0, 4100000.0], Et = [0.0101, 0.0278, 0.0485, 0.0712], v_unit = bbl | SOLVED | N = 9.1612e+06 m^3 |
| Havlena-Odeh oil in place: F = [120000.0, 260000.0, 410000.0], Et = [0.004, 0.0085, 0.0137], v_unit = m3 | SOLVED | N = 3.01044e+07 m^3 |
| Havlena-Odeh oil in place: Et = [0.0101, 0.0278, 0.0485, 0.0712], v_unit = bbl | NEEDS_INPUT | chemeng_havlena_odeh_oil_in_place needs F (underground withdrawals at each survey, in v_unit at reservoir conditions). E |
| Havlena-Odeh oil in place: F = [580700.0, 1600000.0, 2800000.0, 4100000.0], Et = [0.0101, 0.0278, 0.0485, 0.0712], v_unit = bbl, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Oil in place and gas-cap size from the material balance (Havlena-Odeh, gas cap) — `chemeng_havlena_odeh_gas_cap`

Oil initially in place and the gas-cap ratio m from the straight line $F/Eo = N + N$ m Eg/Eo of successive surveys (no water influx).

$$\frac{F}{E_o} = N + Nm\,\frac{E_g}{E_o}$$

| input | meaning | condition |
|---|---|---|
| `F` | underground withdrawals, in v_unit at reservoir conditions | |
| `Eo` | oil and dissolved-gas expansion per unit of oil in place | |
| `Eg` | gas-cap expansion term per unit of oil in place | |
| `v_unit` | unit of F: m3 or bbl | |

| output | meaning |
|---|---|
| `N` | oil initially in place (stock-tank volume) |
| `m` | ratio of initial gas-cap to oil-zone reservoir volume |

- **Method:** least squares of F/Eo against Eg/Eo: intercept N, slope N m
- **Assumes:** no water influx; consistent units
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_havlena_odeh_gas_cap', F=[5000000.0, 9300000.0,
14300000.0, 19500000.0], Eo=[0.01, 0.018, 0.027, 0.036], Eg=[0.025, 0.05, 0.08, 0.11],
v_unit='bbl')
# -> N = 5.05983e+07 m^3, m = 0.226683   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_havlena_odeh_gas_cap', F=[200000.0, 390000.0, 620000.0],
Eo=[0.005, 0.0095, 0.015], Eg=[0.008, 0.016, 0.026], v_unit='m3')
# -> N = 2.36173e+07 m^3, m = 0.434882   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Havlena-Odeh gas cap: F = [5000000.0, 9300000.0, 14300000.0, 19500000.0], Eo = [0.01, 0.018, 0.027, 0.036], Eg = [0.025, 0.05, 0.08, 0.11], v_unit = bbl | SOLVED | N = 5.05983e+07 m^3, m = 0.226683 |
| Havlena-Odeh gas cap: F = [200000.0, 390000.0, 620000.0], Eo = [0.005, 0.0095, 0.015], Eg = [0.008, 0.016, 0.026], v_unit = m3 | SOLVED | N = 2.36173e+07 m^3, m = 0.434882 |
| Havlena-Odeh gas cap: Eo = [0.01, 0.018, 0.027, 0.036], Eg = [0.025, 0.05, 0.08, 0.11], v_unit = bbl | NEEDS_INPUT | chemeng_havlena_odeh_gas_cap needs F (underground withdrawals, in v_unit at reservoir conditions). Example: solve('cheme |
| Havlena-Odeh gas cap: F = [5000000.0, 9300000.0, 14300000.0, 19500000.0], Eo = [0.01, 0.018, 0.027, 0.036], Eg = [0.025, 0.05, 0.08, 0.11], v_unit = bbl, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Gas initially in place from the p/z plot — `chemeng_gas_pz_material_balance`

Original gas in place of a volumetric gas reservoir from the straight line of p/z against cumulative gas production.

$$\frac{p}{z} = \frac{p_i}{z_i}\left(1 - \frac{G_p}{G}\right)$$

| input | meaning | condition |
|---|---|---|
| `p` | average reservoir pressures, in p_unit | |
| `z` | gas deviation factors at those pressures | |
| `Gp` | cumulative gas production at standard conditions, in gp_unit | |
| `p_unit` | unit of the pressures | |
| `gp_unit` | unit of Gp: m3, MMscf or Bscf | |

| output | meaning |
|---|---|
| `G` | original gas in place at standard conditions |
| `pz_initial` | p/z extrapolated to no production |

- **Method:** least squares of p/z = (p/z)_i $(1 - Gp/G)$: G = -intercept/slope

- **Assumes:** volumetric (no water drive) gas reservoir; p/z falls with production
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_gas_pz_material_balance', p=[3200.0, 2925.0, 2525.0,
2125.0], z=[0.85, 0.84, 0.83, 0.82], Gp=[0.0, 15.0, 35.0, 55.0], p_unit='psi', gp_unit='Bscf')
# -> G = 4.99351e+09 m^3, pz_initial = 2.60849e+07 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_gas_pz_material_balance', p=[25.0, 22.1, 18.9], z=[0.9,
0.89, 0.88], Gp=[100000000.0, 300000000.0, 500000000.0], p_unit='MPa', gp_unit='m3')
# -> G = 1.86784e+09 m^3, pz_initial = 2.94209e+07 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Gas p/z material balance: p = [3200.0, 2925.0, 2525.0, 2125.0], z = [0.85, 0.84, 0.83, 0.82], Gp = [0.0, 15.0, 35.0, 55.0], p_unit = psi, gp_unit = Bscf | SOLVED | G = 4.99351e+09 m^3, pz_initial = 2.60849e+07 Pa |
| Gas p/z material balance: p = [25.0, 22.1, 18.9], z = [0.9, 0.89, 0.88], Gp = [100000000.0, 300000000.0, 500000000.0], p_unit = MPa, gp_unit = m3 | SOLVED | G = 1.86784e+09 m^3, pz_initial = 2.94209e+07 Pa |
| Gas p/z material balance: z = [0.85, 0.84, 0.83, 0.82], Gp = [0.0, 15.0, 35.0, 55.0], p_unit = psi, gp_unit = Bscf | NEEDS_INPUT | chemeng_gas_pz_material_balance needs p (average reservoir pressures, in p_unit). Example: solve('chemeng_gas_pz_materia |
| Gas p/z material balance: p = [3200.0, 2925.0, 2525.0, 2125.0], z = [0.85, 0.84, 0.83, 0.82], Gp = [0.0, 15.0, 35.0, 55.0], p_unit = psi, gp_unit = Bscf, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

## Pressure build-up test: Horner analysis for permeability and extrapolated pressure — `chemeng_horner_buildup`

Permeability and the extrapolated pressure p* from the semilog straight line of shut-in pressure against the Horner time ratio (tp + dt)/dt.

$$p_{ws} = p^* - m \log_{10} \frac{t_p + \Delta t}{\Delta t}, \quad k = \frac{q\mu \ln 10}{4\pi m h}$$

| input | meaning | condition |
|---|---|---|
| `dt` | shut-in times, in t_unit | |
| `pws` | shut-in bottom-hole pressures, in p_unit | |
| `t_unit` | unit of the shut-in times | |
| `p_unit` | unit of the pressures | |
| `tp` | producing time before shut-in | |
| `q` | flow rate before shut-in at reservoir conditions | |

| input | meaning | condition |
|---|---|---|
| `mu` | fluid viscosity | |
| `h` | net pay thickness | |

| output | meaning |
|---|---|
| `m` | semilog slope, pressure change per log cycle |
| `k` | permeability |
| `p_star` | extrapolated pressure $pat\ (tp + dt)/dt = 1$ |

- **Method:** least squares of pws against log10((tp + dt)/dt): slope -m; $k = q$ mu ln 10/(4 pi m h)
- **Assumes:** infinite-acting radial flow on the straight line; the points given lie on the semilog straight line
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_horner_buildup', dt=[2.0, 4.0, 8.0, 16.0, 24.0], pws=
[2960.0, 3015.0, 3069.0, 3122.0, 3150.0], t_unit='h', p_unit='psi', tp=1800000.0,
q=0.0005520392185, mu=0.0008, h=6.096)
# -> m = 1.23929e+06 Pa, k = 1.07115e-14 m^2, p_star = 2.3388e+07 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_horner_buildup', dt=[1.0, 3.0, 10.0, 30.0], pws=[20.1,
20.6, 21.05, 21.4], t_unit='h', p_unit='MPa', tp=720000.0, q=0.002, mu=0.001, h=10.0)
# -> m = 913809 Pa, k = 4.01033e-14 m^2, p_star = 2.2236e+07 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Horner build-up analysis: dt = [2.0, 4.0, 8.0, 16.0, 24.0], pws = [2960.0, 3015.0, 3069.0, 3122.0, 3150.0], t_unit = h, p_unit = psi, tp = 1800000.0 s, q = 0.0005520392185 m^3/s, mu = 0.0008 Pa*s, h = 6.096 m | SOLVED | m = 1.23929e+06 Pa, k = 1.07115e-14 m^2, p_star = 2.3388e+07 Pa |
| Horner build-up analysis: dt = [1.0, 3.0, 10.0, 30.0], pws = [20.1, 20.6, 21.05, 21.4], t_unit = h, p_unit = MPa, tp = 720000.0 s, q = 0.002 m^3/s, mu = 0.001 Pa*s, h = 10.0 m | SOLVED | m = 913809 Pa, k = 4.01033e-14 m^2, p_star = 2.2236e+07 Pa |
| Horner build-up analysis: pws = [2960.0, 3015.0, 3069.0, 3122.0, 3150.0], t_unit = h, p_unit = psi, tp = 1800000.0 s, q = 0.0005520392185 m^3/s, mu = 0.0008 Pa*s, h = 6.096 m | NEEDS_INPUT | chemeng_horner_buildup needs dt (shut-in times, in t_unit). Example: solve('chemeng_horner_buildup', dt=[2.0, 4.0, 8.0, |
| Horner build-up analysis: dt = [2.0, 4.0, 8.0, 16.0, 24.0], pws = [2960.0, 3015.0, 3069.0, 3122.0, 3150.0], t_unit = h, p_unit = psi, tp = 1800000.0 s, q = 0.0005520392185 m^3/s, mu = 0.0008 Pa*s, h = 6.096 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Transient drawdown around a well (line-source solution) — `chemeng_line_source_drawdown`

Pressure drop and pressure at radius r and time t after a well starts producing at a constant rate in an infinite reservoir (line-source, exponential-integral solution).

$$\Delta p = \frac{q\mu}{4\pi kh}\, E_1\!\left(\frac{\phi\mu c_t r^2}{4kt}\right)$$

| input | meaning | condition |
|---|---|---|
| `q` | flow rate at reservoir conditions | |
| `mu` | fluid viscosity | |
| `k` | permeability | |
| `h` | net pay thickness | |
| `phi` | porosity (fraction) | |
| `ct` | total compressibility (1/psi is converted here) | |
| `r` | radius from the well | |
| `t` | time since production started | |
| `pi` | initial reservoir pressure | |

| output | meaning |
|---|---|
| `dp` | pressure drop at r and t |
| `p` | pressure at r and t |

- **Method:** dp = (q mu/(4 pi k h)) E1(phi mu ct r^2/(4 k t)); E1 by series or continued fraction
- **Assumes:** infinite-acting, homogeneous, slightly compressible fluid; constant rate from $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_line_source_drawdown', q=0.001, mu=0.001, k=1e-13,
h=10.0, phi=0.2, ct=1e-09, r=0.1, t=3600.0, pi=30000000.0)
# -> dp = 1.02733e+06 Pa, p = 2.89727e+07 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_line_source_drawdown', q=0.002, mu=0.0015, k=5e-14,
h=20.0, phi=0.15, ct=2e-09, r=100.0, t=86400.0, pi=25000000.0)
# -> dp = 241753 Pa, p = 2.47582e+07 Pa   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Line-source drawdown: q = 0.001 m^3/s, mu = 0.001 Pa*s, k = 1e-13 m^2, h = 10.0 m, phi = 0.2, ct = 1e-09 1/Pa, r = 0.1 m, t = 3600.0 s, pi = 30000000.0 Pa | SOLVED | dp = 1.02733e+06 Pa, p = 2.89727e+07 Pa |
| Line-source drawdown: q = 0.002 m^3/s, mu = 0.0015 Pa*s, k = 5e-14 m^2, h = 20.0 m, phi = 0.15, ct = 2e-09 1/Pa, r = 100.0 m, t = 86400.0 s, pi = 25000000.0 Pa | SOLVED | dp = 241753 Pa, p = 2.47582e+07 Pa |
| Line-source drawdown: mu = 0.001 Pa*s, k = 1e-13 m^2, h = 10.0 m, phi = | NEEDS_INPUT | chemeng_line_source_drawdown needs q (flow rate at reservoir conditions). Example: solve('chemeng_line_source_drawdown', |

| question | outcome | result |
|---|---|---|
| 0.2, ct = 1e-09 1/Pa, r = 0.1 m, t = 3600.0 s, pi = 30000000.0 Pa | | |
| Line-source drawdown: q = 0.001 m^3/s, mu = 0.001 Pa*s, k = 1e-13 m^2, h = 10.0 m, phi = 0.2, ct = 1e-09 1/Pa, r = 0.1 m, t = 3600.0 s, pi = 30000000.0 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Line-source drawdown: q = 3.6 m^3/h, mu = 1 cP, k = 1e-13 m^2, h = 10 m, phi = 20%, ct = 1e-9 1/Pa, r = 10 cm, t = 1 h, pi = 300 bar | SOLVED | dp = 1.02733e+06 Pa, p = 2.89727e+07 Pa |

### Gas-well deliverability from a back-pressure test (Rawlins-Schellhardt) — `chemeng_gas_well_backpressure`

Exponent n, coefficient C and absolute open flow of a gas well from multi-rate test data fitted to $q = C$ (pr^2 - pwf^2)^n.

$$q = C\left(\bar{p}_r^2 - p_{wf}^2\right)^n, \quad AOF = C\,\bar{p}_r^{2n}$$

| input | meaning | condition |
|---|---|---|
| `q` | test gas rates, in q_unit | |
| `pwf` | flowing bottom-hole pressures at those rates, in p_unit | |
| `q_unit` | unit of the rates: m3_per_day or Mscf_per_day | |
| `p_unit` | unit of the flowing pressures | |
| `pr` | average reservoir (shut-in) pressure | |

| output | meaning |
|---|---|
| `n` | back-pressure exponent |
| `log10_C` | log10 of the coefficient C, with q in $m^3/s$ and pressures in Pa |
| `AOF` | absolute open flow (rate at pwf = 0) |

- **Method:** least squares of log q against log(pr^2 - pwf^2): slope n, intercept log C; AOF = C $pr^{2n}$
- **Assumes:** stabilised flow at every rate; $0.3 < n < 1.5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_gas_well_backpressure', q=[4288.0, 9265.0, 15552.0,
20177.0], pwf=[403.1, 394.0, 378.5, 362.6], q_unit='Mscf_per_day', p_unit='psi',
pr=2814439.927071325)
# -> n = 0.727757, log10_C = -8.06553, AOF = 20.9729 m^3/s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_gas_well_backpressure', q=[100000.0, 200000.0,
320000.0], pwf=[18.0, 15.5, 11.0], q_unit='m3_per_day', p_unit='MPa', pr=20000000.0)
# -> n = 0.896485, log10_C = -12.3769, AOF = 5.17147 m^3/s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Gas well back-pressure test: q = [4288.0, 9265.0, 15552.0, 20177.0], pwf = [403.1, 394.0, 378.5, 362.6], q_unit = Mscf_per_day, p_unit = psi, pr = 2814439.927071325 Pa | SOLVED | n = 0.727757, log10_C = -8.06553, AOF = 20.9729 m^3/s |
| Gas well back-pressure test: q = [100000.0, 200000.0, 320000.0], pwf = [18.0, 15.5, 11.0], q_unit = m3_per_day, p_unit = MPa, pr = 20000000.0 Pa | SOLVED | n = 0.896485, log10_C = -12.3769, AOF = 5.17147 m^3/s |
| Gas well back-pressure test: pwf = [403.1, 394.0, 378.5, 362.6], q_unit = Mscf_per_day, p_unit = psi, pr = 2814439.927071325 Pa | NEEDS_INPUT | chemeng_gas_well_backpressure needs q (test gas rates, in q_unit). Example: solve('chemeng_gas_well_backpressure', q=[42 |
| Gas well back-pressure test: q = [4288.0, 9265.0, 15552.0, 20177.0], pwf = [403.1, 394.0, 378.5, 362.6], q_unit = Mscf_per_day, p_unit = psi, pr = 2814439.927071325 Pa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

### Minimum number of stages at total reflux (Fenske) — `chemeng_fenske_minimum_stages`

The minimum number of equilibrium stages of a binary distillation at total reflux by the Fenske equation (constant relative volatility), counting the partial reboiler as a stage.

$$N_{min} = \frac{\ln\left[\frac{x_D}{1-x_D}\,\frac{1-x_B}{x_B}\right]}{\ln \alpha}$$

| input | meaning | condition |
|---|---|---|
| `alpha` | relative volatility of the light component (constant) | |
| `xD` | mole fraction of the light component in the distillate | |
| `xB` | mole fraction of the light component in the bottoms | |

| output | meaning |
|---|---|
| `Nmin` | minimum number of equilibrium stages (partial reboiler included) |
| `Nmin_trays` | minimum number of trays (Nmin - 1, the reboiler excluded) |

- **Method:** N_min = ln[(xD/(1 - xD)) ((1 - xB)/xB)]/ln(alpha)
- **Assumes:** binary, constant alpha; $0 < xB < xD < 1$; total reflux, total condenser
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_fenske_minimum_stages', alpha=2.5, xD=0.95, xB=0.05)
# -> Nmin = 6.42687, Nmin_trays = 5.42687   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_fenske_minimum_stages', alpha=1.5, xD=0.99, xB=0.01)
# -> Nmin = 22.6659, Nmin_trays = 21.6659   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Fenske minimum stages: alpha = 2.5, xD = 0.95, xB = 0.05 | SOLVED | Nmin = 6.42687, Nmin_trays = 5.42687 |
| Fenske minimum stages: alpha = 1.5, xD = 0.99, xB = 0.01 | SOLVED | Nmin = 22.6659, Nmin_trays = 21.6659 |
| Fenske minimum stages: xD = 0.95, xB = 0.05 | NEEDS_INPUT | chemeng_fenske_minimum_stages needs alpha (relative volatility of the light component (constant)). Example: solve('cheme |
| Fenske minimum stages: alpha = 2.5, xD = 0.95, xB = 0.05, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Fenske minimum stages: alpha = 0.9, xD = 0.95, xB = 0.05 | INVALID_INPUT | Minimum number of stages at total reflux (Fenske): the relative volatility alpha must exceed 1. |

## Pump total head and power — `chemeng_pump_head`

The total head a pump adds between its suction and discharge, H = (p2 - p1)/(rho g) + (z2 - z1) + (v2^2 - v1^2)/(2g), with the hydraulic power rho g Q H and the shaft power for an efficiency.

$$H = \frac{p_2 - p_1}{\rho g} + (z_2 - z_1) + \frac{v_2^2 - v_1^2}{2g}, \quad P = \rho g Q H$$

| input | meaning | condition |
|---|---|---|
| `p1` | suction pressure (absolute) | |
| `p2` | discharge pressure (absolute) | |
| `rho` | liquid density | |
| `dz` | elevation of the discharge gauge above the suction gauge (default 0) | |
| `v1` | suction velocity (default 0) | |
| `v2` | discharge velocity (default 0) | |
| `Q` | volumetric flow (optional, for the power) | |
| `eta` | pump efficiency (optional, with Q) | |

| output | meaning |
|---|---|
| `H` | total head |
| `dp_equivalent` | head as a pressure rho g H |
| `P_hydraulic` | hydraulic power rho g Q H |
| `P_shaft` | shaft power rho g Q H/eta |

- **Method:** H = (p2 - p1)/(rho $g$) + $dz$ + ($v_2^2 - v_1^2$)/(2 g); $P = \rho$ g Q H; P_shaft = P/eta
- **Assumes:** absolute pressures; incompressible liquid; $H > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_pump_head', p1=100000.0, p2=400000.0, rho=1000.0)
# -> H = 30.5915 m, dp_equivalent = 300000 Pa   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_pump_head', p1=150000.0, p2=650000.0, rho=870.0, dz=2.0,
v1=1.5, v2=3.0, Q=0.02, eta=0.7)
# -> H = 60.9485 m, dp_equivalent = 520000 Pa, P_hydraulic = 10400 W, P_shaft = 14857.1 W
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Pump total head: p1 = 100000.0 Pa, p2 = 400000.0 Pa, rho = 1000.0 kg/m^3 | SOLVED | H = 30.5915 m, dp_equivalent = 300000 Pa |
| Pump total head: p1 = 150000.0 Pa, p2 = 650000.0 Pa, rho = 870.0 kg/m^3, dz = 2.0 m, v1 = 1.5 m/s, v2 = 3.0 m/s, Q = 0.02 m^3/s, eta = 0.7 | SOLVED | H = 60.9485 m, dp_equivalent = 520000 Pa, P_hydraulic = 10400 W, P_shaft = 14857.1 W |
| Pump total head: p2 = 400000.0 Pa, rho = 1000.0 kg/m^3 | NEEDS_INPUT | chemeng_pump_head needs p1 (suction pressure (absolute)). Example: solve('chemeng_pump_head', p1='100000 Pa', p2='400000 |
| Pump total head: p1 = 100000.0 Pa, p2 = 400000.0 Pa, rho = 1000.0 kg/m^3, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Pump total head: p1 = 400000 Pa, p2 = 100000 Pa, rho = 1000 kg/m^3 | INVALID_INPUT | Pump total head and power: the pump must add head (H > 0). |

## Net positive suction head available — `chemeng_npsh_available`

The net positive suction head available at a pump inlet, NPSHa = (p_s - p_v)/(rho g) + z_s - h_f, from the pressure on the liquid surface, the vapour pressure, the static head and the suction-line loss; with the required NPSH, the margin.

$$NPSH_a = \frac{p_s - p_v}{\rho g} + z_s - h_f$$

| input | meaning | condition |
|---|---|---|
| `p_s` | absolute pressure on the suction liquid surface | |
| `p_v` | vapour pressure of the liquid at the pumping temperature | |
| `rho` | liquid density | |
| `z_s` | liquid surface above the pump inlet (negative for a suction lift) | |
| `h_f` | friction head loss in the suction line | |
| `NPSHr` | required NPSH of the pump (optional) | |

| output | meaning |
|---|---|
| `NPSHa` | NPSH available |
| `pressure_head` | p_s/(rho g) |

| output | meaning |
|---|---|
| `vapour_head` | p_v/(rho g) |
| `margin` | NPSHa - NPSHr (with NPSHr) |

- **Method:** NPSHa = (p_s - p_v)/(rho g) + z_s - h_f
- **Assumes:** absolute pressures; $h_f \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_npsh_available', p_s=101325.0, p_v=2339.0, rho=998.0,
z_s=2.0, h_f=0.5)
# -> NPSHa = 11.614 m, pressure_head = 10.353 m, vapour_head = 0.23899 m   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_npsh_available', p_s=101325.0, p_v=47400.0, rho=971.8,
z_s=-3.0, h_f=0.8, NPSHr=2.0)
# -> NPSHa = 1.85839 m, pressure_head = 10.6321 m, vapour_head = 4.97371 m, margin = -0.141614 m
(SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| NPSH available: p_s = 101325.0 Pa, p_v = 2339.0 Pa, rho = 998.0 kg/m^3, z_s = 2.0 m, h_f = 0.5 m | SOLVED | NPSHa = 11.614 m, pressure_head = 10.353 m, vapour_head = 0.23899 m |
| NPSH available: p_s = 101325.0 Pa, p_v = 47400.0 Pa, rho = 971.8 kg/m^3, z_s = -3.0 m, h_f = 0.8 m, NPSHr = 2.0 m | SOLVED | NPSHa = 1.85839 m, pressure_head = 10.6321 m, vapour_head = 4.97371 m, margin = -0.141614 m |
| NPSH available: p_v = 2339.0 Pa, rho = 998.0 kg/m^3, z_s = 2.0 m, h_f = 0.5 m | NEEDS_INPUT | chemeng_npsh_available needs p_s (absolute pressure on the suction liquid surface). Example: solve('chemeng_npsh_availab |
| NPSH available: p_s = 101325.0 Pa, p_v = 2339.0 Pa, rho = 998.0 kg/m^3, z_s = 2.0 m, h_f = 0.5 m, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| NPSH available: p_s = 101325 Pa, p_v = 2339 Pa, rho = 998 kg/m^3, z_s = 2 m, h_f = -0.5 m | INVALID_INPUT | Net positive suction head available: the suction friction head loss must be zero or positive. |

## Mass-transfer coefficient from a stated Sherwood correlation —

**`chemeng_mass_transfer_coefficient`**

The convective mass-transfer coefficient $k_c = Sh$ D/L from the Reynolds and Schmidt numbers by a stated correlation: a sphere (Ranz-Marshall), a flat plate in laminar flow (average) or turbulent flow in a pipe (Linton-Sherwood).

$$Sh = \frac{k_c L}{D}, \quad Sh = 2 + 0.6\, Re^{1/2} Sc^{1/3}\ \text{(sphere)}$$

| input | meaning | condition |
|---|---|---|
| `geometry` | correlation: sphere (Ranz-Marshall), flat_plate (laminar, average) or pipe (Linton-Sherwood, turbulent) | |
| `u` | fluid velocity | |
| `L` | characteristic length: sphere or pipe diameter, plate length | |
| `D` | diffusivity of the solute in the fluid | |
| `rho` | fluid density | |
| `mu` | fluid viscosity | |

| output | meaning |
|---|---|
| `Re` | Reynolds number rho u L/mu |
| `Sc` | Schmidt number mu/(rho D) |
| `Sh` | Sherwood number |
| `kc` | mass-transfer coefficient Sh D/L |

- **Method:** $Re = \rho$ u L/mu, Sc = mu/(rho D); sphere $Sh = 2 + 0.6\ Re^{1/2}\ Sc^{1/3}$; plate $Sh = 0.664\ Re^{1/2}\ Sc^{1/3}$; pipe $Sh = 0.023\ Re^{0.83}\ Sc^{1/3}$; $k_c = Sh$ D/L
- **Assumes:** within each correlation's stated range of Re and Sc; dilute solute
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_mass_transfer_coefficient', geometry='sphere', u=0.1,
L=0.001, D=5e-09, rho=1000.0, mu=0.001)
# -> Re = 100, Sc = 200, Sh = 37.0882, kc = 0.000185441 m/s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_mass_transfer_coefficient', geometry='pipe', u=1.0,
L=0.025, D=1.5e-09, rho=998.0, mu=0.001)
# -> Re = 24950, Sc = 668.003, Sh = 897.2, kc = 5.3832e-05 m/s   (SOLVED)
```

### Questions in words (documented forms), as executed:

| question | outcome | result |
|---|---|---|
| Mass-transfer coefficient: geometry = sphere, u = 0.1 m/s, L = 0.001 m, D = 5e-09 m^2/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s | SOLVED | Re = 100, Sc = 200, Sh = 37.0882, kc = 0.000185441 m/s |
| Mass-transfer coefficient: geometry = pipe, u = 1.0 m/s, L = 0.025 m, D = 1.5e-09 m^2/s, rho = 998.0 kg/m^3, mu = 0.001 Pa*s | SOLVED | Re = 24950, Sc = 668.003, Sh = 897.2, kc = 5.3832e-05 m/s |
| Mass-transfer coefficient: u = 0.1 m/s, L = 0.001 m, D = 5e-09 m^2/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s | NEEDS_INPUT | chemeng_mass_transfer_coefficient needs geometry (correlation: sphere (Ranz-Marshall), flat_plate (laminar, average) or |
| Mass-transfer coefficient: geometry = sphere, u = 0.1 m/s, L = 0.001 m, D = 5e-09 m^2/s, rho = 1000.0 kg/m^3, mu = 0.001 Pa*s, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |

| question | outcome | result |
|---|---|---|
| Mass-transfer coefficient: geometry = pipe, u = 0.01 m/s, L = 0.025 m, D = 1.5e-9 m^2/s, rho = 998 kg/m^3, mu = 0.001 Pa*s | INVALID_INPUT | Mass-transfer coefficient from a stated Sherwood correlation: Linton-Sherwood is stated for 2000 <= Re <= 70000 and 0.6 |

## Minimum reflux of a multicomponent column (Underwood) —

`chemeng_multicomponent_underwood`

The minimum reflux ratio of a multicomponent distillation with constant relative volatilities by Underwood's equations: the root theta between the key volatilities of sum alpha_i $z_i/(alpha_i - \theta) = 1 - q$, then R_min + 1 = sum alpha_i x_D,i/(alpha_i - theta).

$$\sum_i \frac{\alpha_i z_i}{\alpha_i - \theta} = 1 - q, \quad R_{min} + 1 = \sum_i \frac{\alpha_i x_{D,i}}{\alpha_i - \theta}$$

| input | meaning | condition |
|---|---|---|
| `alpha` | relative volatility of each component (any common reference, constant) | |
| `z` | feed mole fractions (sum to 1) | |
| `xD` | distillate mole fractions at minimum reflux (sum to 1) | |
| `q` | feed quality q (1 saturated liquid, 0 saturated vapour) | |
| `lk` | light key: its position in the lists (1 = first) | |
| `hk` | heavy key: its position in the lists | |

| output | meaning |
|---|---|
| `theta` | Underwood root between the key volatilities |
| `Rmin` | minimum reflux ratio L/D |

- **Method:** sum alpha_i $z_i/(alpha_i - \theta) = 1 - q$, theta between alpha_HK and alpha_LK (bisection); R_min = sum alpha_i $x_Di/(alpha_i - \theta) - 1$
- **Assumes:** constant relative volatilities and molar overflow; adjacent keys (no component between them in volatility); the distillate composition at minimum reflux is stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_multicomponent_underwood', alpha=[2.5, 1.0, 0.5], z=
[0.4, 0.3, 0.3], xD=[0.9, 0.1, 0.0], q=1.0, lk=1, hk=2)
# -> theta = 1.40263, Rmin = 0.802   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_multicomponent_underwood', alpha=[4.0, 2.0, 1.0, 0.6],
z=[0.1, 0.3, 0.3, 0.3], xD=[0.24, 0.7, 0.06, 0.0], q=0.5, lk=2, hk=3)
# -> theta = 1.48444, Rmin = 1.97326   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Multicomponent minimum reflux: alpha = [2.5, 1.0, 0.5], z = [0.4, 0.3, 0.3], xD = [0.9, 0.1, 0.0], q = 1.0, lk = 1, hk = 2 | SOLVED | theta = 1.40263, Rmin = 0.802 |
| Multicomponent minimum reflux: alpha = [4.0, 2.0, 1.0, 0.6], z = [0.1, 0.3, 0.3, 0.3], xD = [0.24, 0.7, 0.06, 0.0], q = 0.5, lk = 2, hk = 3 | SOLVED | theta = 1.48444, Rmin = 1.97326 |
| Multicomponent minimum reflux: z = [0.4, 0.3, 0.3], xD = [0.9, 0.1, 0.0], q = 1.0, lk = 1, hk = 2 | NEEDS_INPUT | chemeng_multicomponent_underwood needs alpha (relative volatility of each component (any common reference, constant)). E |
| Multicomponent minimum reflux: alpha = [2.5, 1.0, 0.5], z = [0.4, 0.3, 0.3], xD = [0.9, 0.1, 0.0], q = 1.0, lk = 1, hk = 2, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Multicomponent minimum reflux: alpha = [2.5, 1.0, 1.5], z = [0.4, 0.3, 0.3], xD = [0.9, 0.1, 0.0], q = 1, lk = 1, hk = 2 | INVALID_INPUT | Minimum reflux of a multicomponent column (Underwood): the keys must be adjacent in volatility (no component between the |

### CSTR volume for a conversion (second order) — `chemeng_cstr_volume_second_order`

Volume and space time of a CSTR that reaches conversion X when -r_A = k $C_A^2$ (liquid phase, constant density).

$$V = \frac{v_0 X}{k C_{A0} (1 - X)^2}$$

| input | meaning | condition |
|---|---|---|
| `v0` | volumetric feed flow | |
| `k` | second-order rate constant (-r_A = k C_A^2) | |
| `CA0` | feed (initial) concentration of A | |
| `X` | conversion of A wanted | |

| output | meaning |
|---|---|
| `V` | reactor volume |
| `tau` | space time $V/v_0$ |

- **Method:** design equation $V = F_{A0}$ X/(-r_A) at the exit concentration: $V = v_0$ X/(k C_A0 (1 - X)^2)
- **Assumes:** $0 \le X < 1$; steady state, perfect mixing, constant density, $-r_A = k\,C_A^2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_cstr_volume_second_order', v0=0.01, k=1e-05, CA0=1000.0, X=0.8)
# -> V = 20 m^3, tau = 2000 s   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_cstr_volume_second_order', v0=0.002, k=0.0001,
CA0=200.0, X=0.5)
# -> V = 0.2 m^3, tau = 100 s   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Second-order CSTR volume: v0 = 0.01 m^3/s, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8 | SOLVED | V = 20 m^3, tau = 2000 s |
| Second-order CSTR volume: v0 = 0.002 m^3/s, k = 0.0001 m^3/(mol*s), CA0 = 200.0 mol/m^3, X = 0.5 | SOLVED | V = 0.2 m^3, tau = 100 s |
| Second-order CSTR volume: k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8 | NEEDS_INPUT | chemeng_cstr_volume_second_order needs v0 (volumetric feed flow). Example: solve('chemeng_cstr_volume_second_order', v0= |
| Second-order CSTR volume: v0 = 0.01 m^3/s, k = 1e-05 m^3/(mol*s), CA0 = 1000.0 mol/m^3, X = 0.8, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Second-order CSTR volume: v0 = 0.01 m^3/s, k = 1e-5 m^3/(mol*s), CA0 = 1000 mol/m^3, X = 100% | INVALID_INPUT | CSTR volume for a conversion (second order): the conversion X must be a fraction between 0 and 1. |

## Equimolar counterdiffusion flux — `chemeng_equimolar_counterdiffusion`

The molar flux of A through a film of thickness L in equimolar counterdiffusion (N_B = -N_A, ideal gas): $N_A = c$ D $(y_A1 - y_A2)/L$ with c = P/(R T).

$$N_A = \frac{P}{RT}\,\frac{D_{AB}\,(y_{A1} - y_{A2})}{L}$$

| input | meaning | condition |
|---|---|---|
| `D` | diffusivity of A in B | |
| `P` | total pressure | |
| `T` | temperature | |
| `L` | film (path) thickness | |
| `yA1` | mole fraction of A at $z = 0$ | |
| `yA2` | mole fraction of A at $z = L$ | |

| output | meaning |
|---|---|
| `NA` | molar flux of A |
| `NB` | molar flux of B (= -N_A) |

- **Method:** N_A = (P/(R T)) D $(y_A1 - y_A2)/L$

- **Assumes:** steady state, ideal gas, constant D; equimolar counterdiffusion $(N_B = -N_A)$; $yA1 > yA2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_equimolar_counterdiffusion', D=2.3e-05, P=101325.0,
T=298.15, L=0.1, yA1=0.2, yA2=0.05)
# -> NA = 0.00141015 mol/(m^2*s), NB = -0.00141015 mol/(m^2*s)   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_equimolar_counterdiffusion', D=1e-05, P=200000.0,
T=400.0, L=0.02, yA1=0.6, yA2=0.0)
# -> NA = 0.0180409 mol/(m^2*s), NB = -0.0180409 mol/(m^2*s)   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Equimolar counterdiffusion flux: D = 2.3e-05 m^2/s, P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.2, yA2 = 0.05 | SOLVED | NA = 0.00141015 mol/(m^2*s), NB = -0.00141015 mol/(m^2*s) |
| Equimolar counterdiffusion flux: D = 1e-05 m^2/s, P = 200000.0 Pa, T = 400.0 K, L = 0.02 m, yA1 = 0.6, yA2 = 0.0 | SOLVED | NA = 0.0180409 mol/(m^2*s), NB = -0.0180409 mol/(m^2*s) |
| Equimolar counterdiffusion flux: P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.2, yA2 = 0.05 | NEEDS_INPUT | chemeng_equimolar_counterdiffusion needs D (diffusivity of A in B). Example: solve('chemeng_equimolar_counterdiffusion', |
| Equimolar counterdiffusion flux: D = 2.3e-05 m^2/s, P = 101325.0 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.2, yA2 = 0.05, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Equimolar counterdiffusion flux: D = 2.3e-5 m^2/s, P = 101325 Pa, T = 298.15 K, L = 0.1 m, yA1 = 0.05, yA2 = 0.2 | INVALID_INPUT | Equimolar counterdiffusion flux: A diffuses from yA1 to yA2: yA1 must exceed yA2. |

## Bubble-point pressure by the modified Raoult's law (activity coefficients) —

`chemeng_modified_raoult_bubble_pressure`

The bubble-point pressure P = sum x_i gamma_i P_i^sat of a non-ideal liquid with stated activity coefficients and vapour pressures (low pressure, ideal vapour), and the vapour composition $y_i = x_i$ gamma_i P_i^sat/P.

$$P = \sum_i x_i \gamma_i P_i^{sat}, \quad y_i = \frac{x_i \gamma_i P_i^{sat}}{P}$$

| input | meaning | condition |
|---|---|---|
| `x` | liquid mole fractions (sum to 1) | |
| `gamma` | activity coefficient of each component at this composition | |
| `Psat` | vapour pressure of each pure component at the temperature, in the unit p_unit | |

| input | meaning | condition |
|---|---|---|
| `p_unit` | unit of the vapour pressures: Pa, kPa, MPa, bar, atm or psi | |

| output | meaning |
|---|---|
| `P` | bubble-point pressure |
| `y1` | vapour mole fraction of the first component |
| `y` | vapour mole fractions, in the order of x |

- **Method:** P = sum x_i gamma_i P_i^sat, $y_i = x_i$ gamma_i P_i^sat/P
- **Assumes:** low pressure (ideal vapour, Poynting factor 1); activity coefficients stated for this liquid composition and temperature
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
chemical_engineering_ai999x.solve('chemeng_modified_raoult_bubble_pressure', x=[0.4, 0.6], gamma=
[1.8, 1.2], Psat=[55.0, 30.0], p_unit='kPa')
# -> P = 61200 Pa, y1 = 0.647059, y = [0.647058823529, 0.352941176471]   (SOLVED)
```

```
chemical_engineering_ai999x.solve('chemeng_modified_raoult_bubble_pressure', x=[0.2, 0.3, 0.5],
gamma=[2.0, 1.1, 1.05], Psat=[1.2, 0.8, 0.3], p_unit='bar')
# -> P = 90150 Pa, y1 = 0.532446, y = [0.532445923461, 0.292845257903, 0.174708818636]   (SOLVED)
```

**Questions in words (documented forms), as executed:**

| question | outcome | result |
|---|---|---|
| Modified Raoult bubble pressure: x = [0.4, 0.6], gamma = [1.8, 1.2], Psat = [55.0, 30.0], p_unit = kPa | SOLVED | P = 61200 Pa, y1 = 0.647059, y = [0.647058823529, 0.352941176471] |
| Modified Raoult bubble pressure: x = [0.2, 0.3, 0.5], gamma = [2.0, 1.1, 1.05], Psat = [1.2, 0.8, 0.3], p_unit = bar | SOLVED | P = 90150 Pa, y1 = 0.532446, y = [0.532445923461, 0.292845257903, 0.174708818636] |
| Modified Raoult bubble pressure: gamma = [1.8, 1.2], Psat = [55.0, 30.0], p_unit = kPa | NEEDS_INPUT | chemeng_modified_raoult_bubble_pressure needs x (liquid mole fractions (sum to 1)). Example: solve('chemeng_modified_rao |
| Modified Raoult bubble pressure: x = [0.4, 0.6], gamma = [1.8, 1.2], Psat = [55.0, 30.0], p_unit = kPa, zz = 1 | UNRECOGNIZED_FORMAT | I could not match this question to a documented form, so I will not guess. Every form: chemical_engineering_ai999x.templ |
| Modified Raoult bubble pressure: x = [0.4, 0.7], gamma = [1.8, 1.2], Psat = [55, 30], p_unit = kPa | INVALID_INPUT | Bubble-point pressure by the modified Raoult's law (activity coefficients): x: the fractions must sum to 1. |